Saxa7-Final-Project.R

liula

2025-10-17

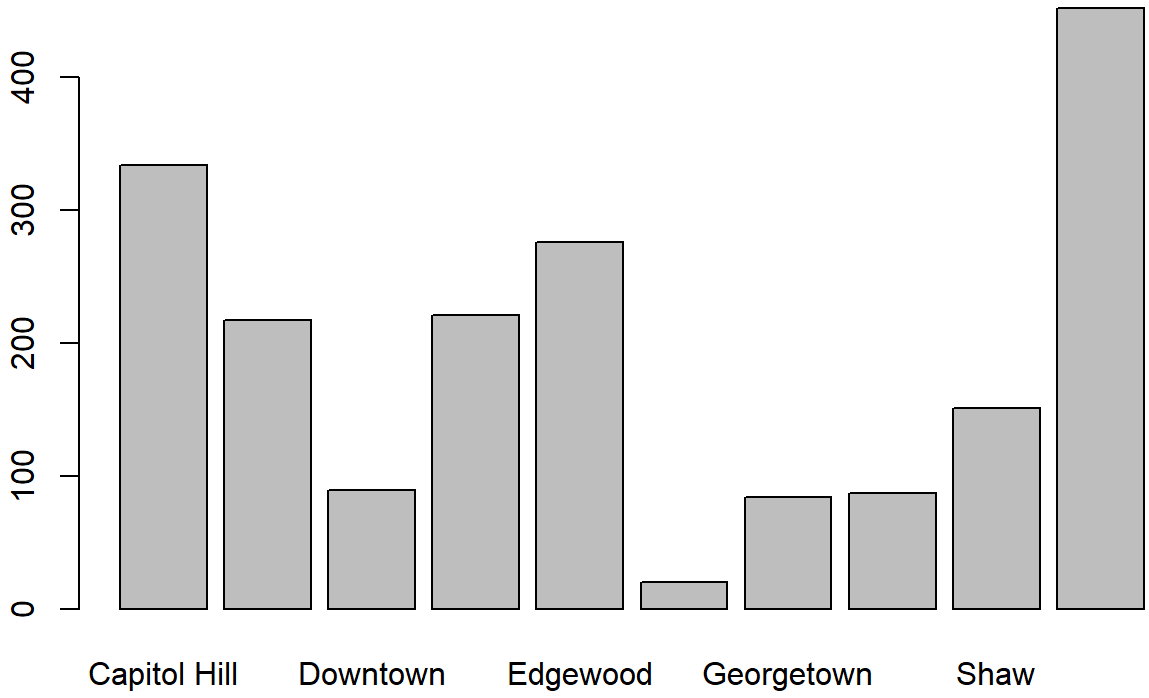
```
## QUESTION 1 ##
# GOAL: Exploratory Data Analysis
# check for missing values, replace if needed
# check data types of all the columns in the data frame
# determine insights

library(tidyverse)
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✓ dplyr     1.1.4     ✓ readr     2.1.5
## ✓ forcats   1.0.0     ✓ stringr   1.5.1
## ✓ ggplot2   3.5.2     ✓ tibble    3.3.0
## ✓ lubridate 1.9.4     ✓ tidyr     1.3.1
## ✓ purrr     1.1.0
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
e errors
```

```
Listings = read.csv("Listings.csv")
Reviews = read.csv("Reviews.csv")
# names(Listings)
# names(Reviews)
# glimpse(Listings)
# glimpse(Reviews)


## use a left join to make sure that the data frame has all the rows from Listing and the additi
onal info from Reviews
df = left_join( Listings , Reviews , by = c("id" = "listing_id"))
## check to see if there are any NaNs in the data set

# apply( df, 2, FUN = anyNA)

## find the index for where the NaNs are located and store in missing beds vector
missing_beds = which(is.na(df$beds))
## using missing beds vector to replace the NaNs with the mean
## beds should not double -> use ceiling to round 1.92 up to 2
df$beds[missing_beds] = ceiling(mean(df$beds, na.rm = TRUE))
## do the same thing with avg rating
missing_rating = which(is.na(df$avg_rating))
# df$avg_rating[missing_rating] = which(is.na(df$avg_rating))
df$avg_rating[missing_rating] = mean(df$avg_rating, na.rm = TRUE)

## since there are blank values in room type decided to replace these blanks with "unknown"
df$room_type[df$room_type == ""] <- "Unknown"

## now that the data frame has been cleaned of NaNs and Blanks
df = df %>%
  select(-id) %>%
  mutate(neighborhood = as.factor(neighborhood) , host_since = as.Date(host_since, format = "%
m/%d/%Y")
         , room_type = as.factor(room_type) , bathrooms = as.factor(bathrooms),
         bedrooms_f = as.factor(bedrooms))
glimpse(df)
```

```
## Rows: 1,931
## Columns: 15
## $ neighborhood         <fct> Shaw, Capitol Hill, Edgewood, Edgewood, Shaw, Col…
## $ host_since           <date> 2009-10-31, 2011-09-01, 2012-06-27, 2010-12-10, …
## $ host_acceptance_rate <dbl> 0.89, 0.80, 0.99, 0.76, 0.83, 0.98, 0.83, 0.40, 0…
## $ superhost            <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, …
## $ host_total_listings  <int> 7, 4, 11, 91, 2, 2, 2, 1, 1376, 91, 13, 1, 1, 4, …
## $ room_type            <fct> Entire home/apt, Entire home/apt, Entire home/apt…
## $ accommodates         <int> 4, 3, 6, 1, 3, 2, 6, 2, 3, 1, 6, 2, 2, 2, 2, 5, 2…
## $ bathrooms            <fct> 1 bath, 1 bath, 1 bath, 2 shared baths, 1 bath, 1…
## $ bedrooms             <int> 2, 1, 1, 0, 1, 1, 3, 1, 1, 1, 2, 1, 1, 1, 1, 3, 0…
## $ beds                 <dbl> 2, 1, 4, 1, 2, 2, 3, 1, 2, 1, 3, 1, 2, 1, 1, 3, 1…
## $ price                <dbl> 72, 205, 93, 30, 200, 96, 224, 130, 338, 55, 272,…
## $ min_nights           <int> 31, 2, 2, 31, 3, 2, 4, 31, 1, 31, 3, 2, 2, 2, 2, …
## $ total_reviews        <int> 191, 156, 377, 4, 14, 268, 22, 26, 35, 12, 206, 5…
## $ avg_rating           <dbl> 4.31, 4.80, 4.75, 3.25, 4.86, 4.85, 4.95, 4.76, 4…
## $ bedrooms_f           <fct> 2, 1, 1, 0, 1, 1, 3, 1, 1, 1, 2, 1, 1, 1, 1, 3, 0…
```

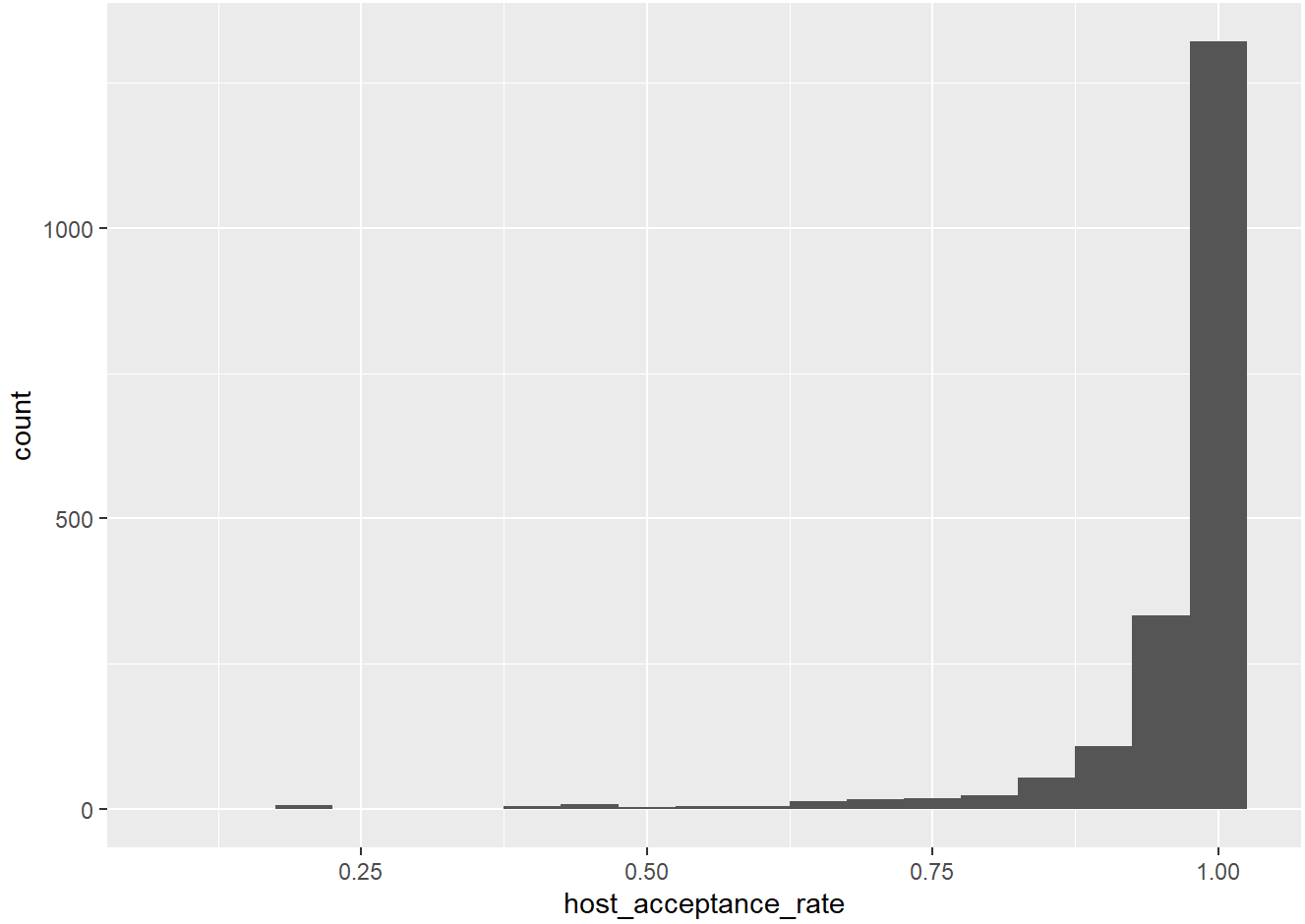
```
## let us explore each column

# 1) Neighborhood: Character data type -> convert to factor to make more robust for categorical analysis

neigh_freq=table(df$neighborhood)
## it was found that Union Station, Capital Hill, Columia heights, dupont, and edgewood were the most popular
barplot(neigh_freq)
```



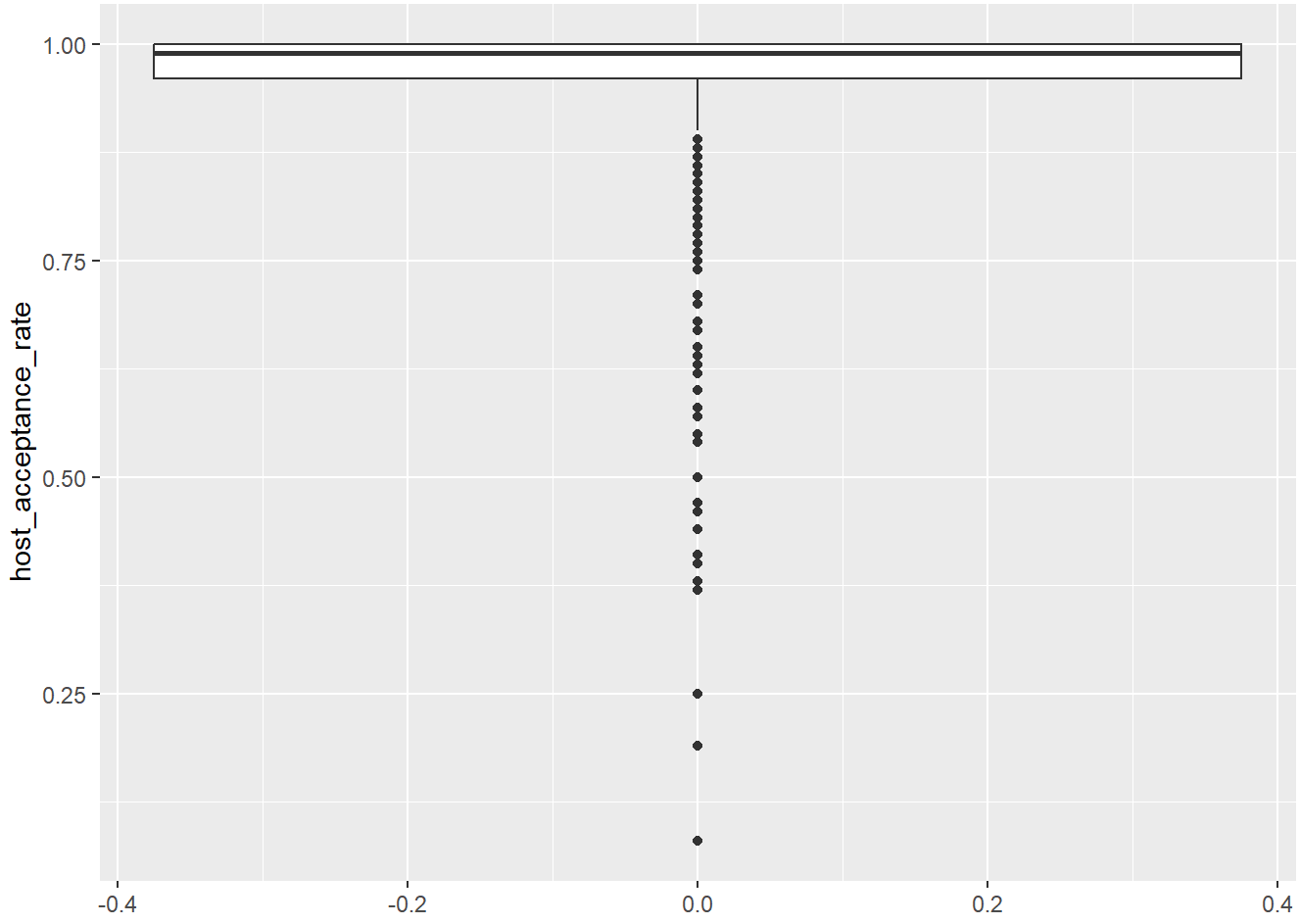

```
# 2) host_since is a character data type -> convert to a date
## decided to use the IQR method of removing outliers since the data was skewed with an extreme
outlier

interquartilerange= IQR(df$price)
upper = quantile(df$price, 0.75)
lower = quantile(df$price , 0.25)
upper_outlier = upper + 1.5 * interquartilerange
lower_outlier = lower - 1.5 * interquartilerange
df_price= df %>%
  filter ( price > lower_outlier & price < upper_outlier)

# ggplot(df, aes(x=host_since, y = price))  + geom_line() +labs(y = "Price" , x = "Host_since",
# title = "Line Plot of Raw Price V Host_since")
# ggplot(df_price, aes(x=host_since, y = price))  + geom_line() +labs(y = "Price" , x = "Host_si
nce",
# title = "Line Plot of Filtered Price V Host_since")

# 3) host_acceptance_rate is a double
## data type makes sense, no need to change . explore values

summary(df$host_acceptance_rate)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.0800  0.9600  0.9900  0.9563  1.0000  1.0000
```

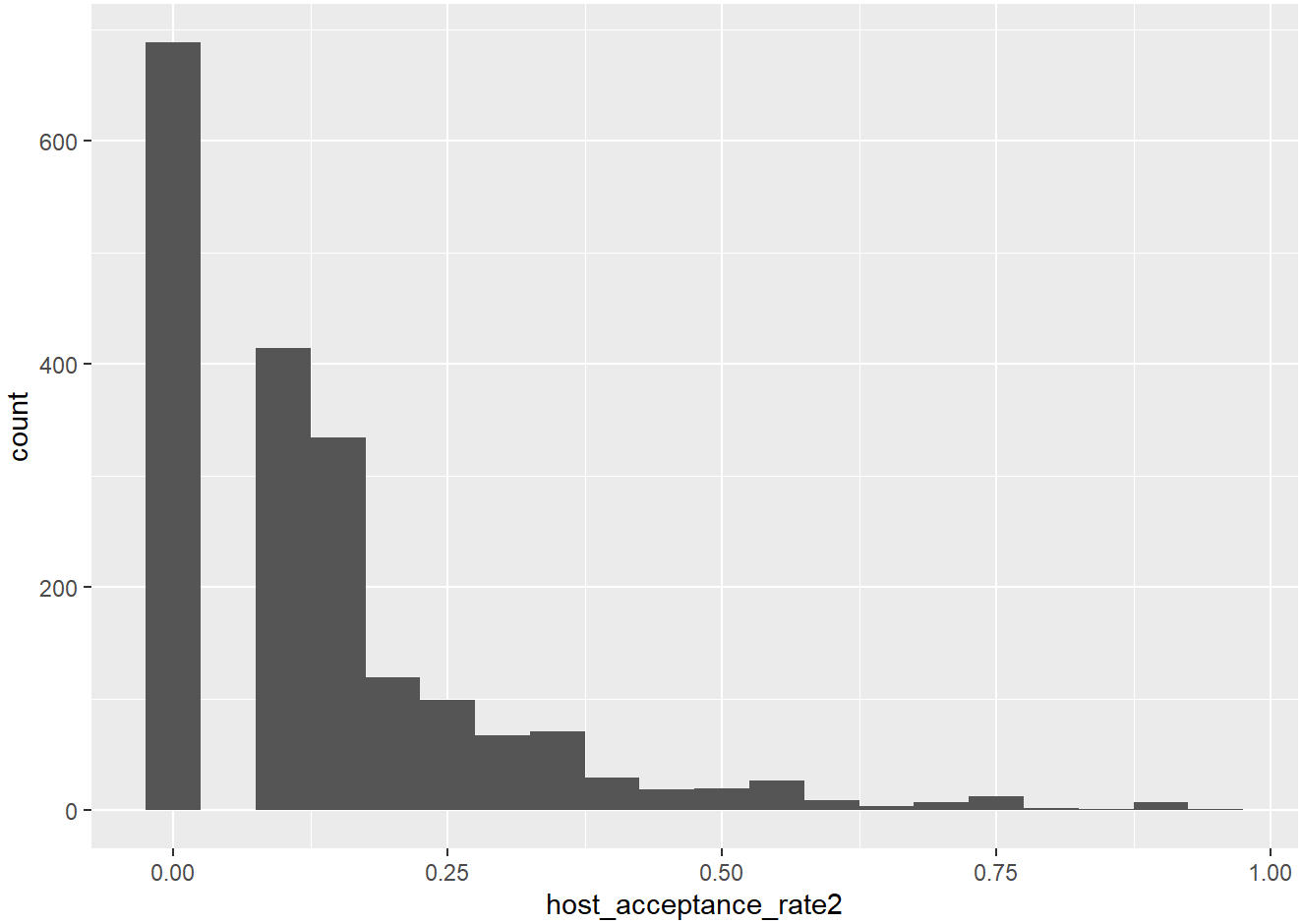
```
## summary stats show possible left skewness
ggplot(df, aes(host_acceptance_rate)) + geom_histogram(binwidth = 0.05)
```



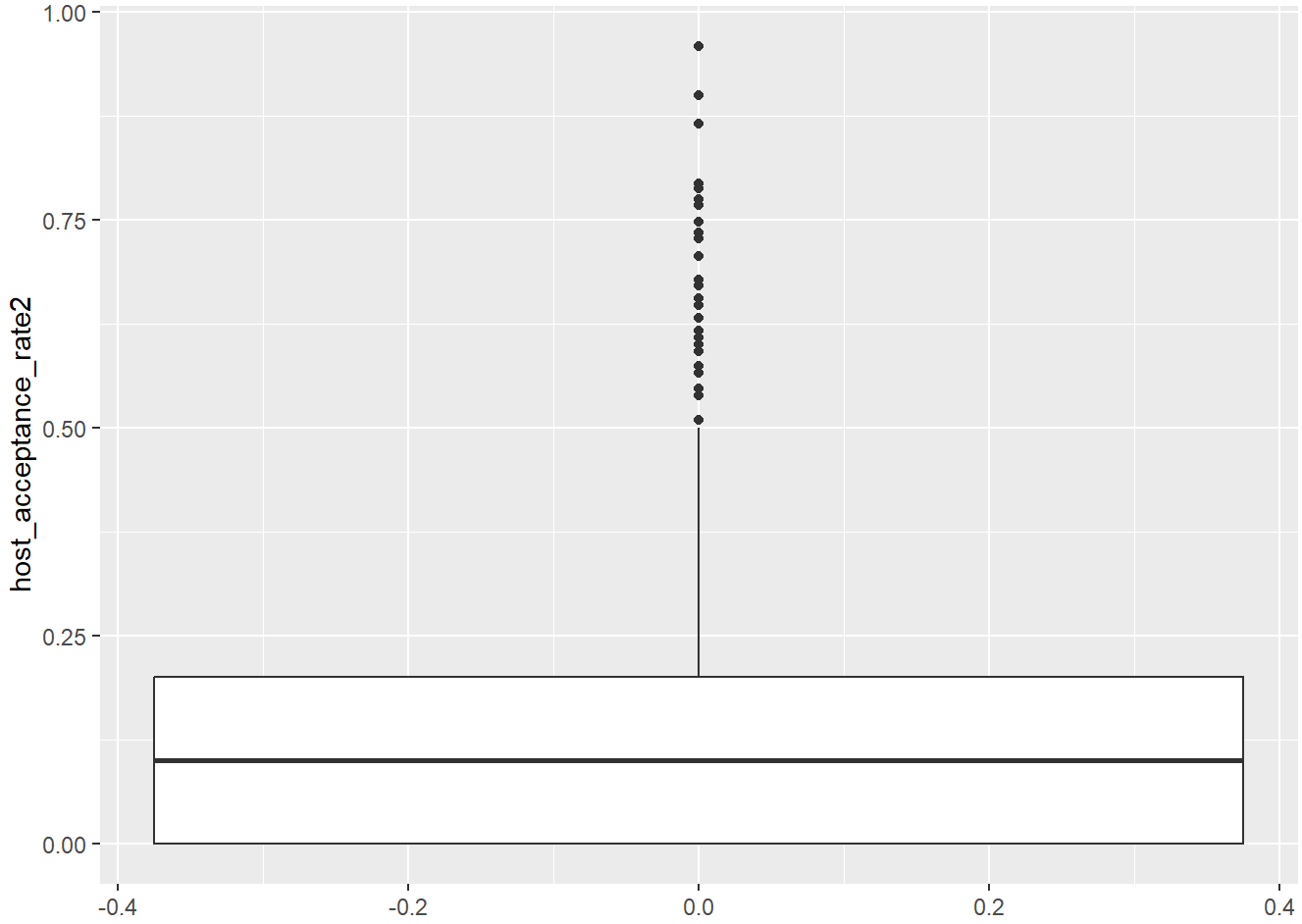

```
ggplot(df, aes(host_acceptance_rate)) + geom_boxplot() + coord_flip()
```



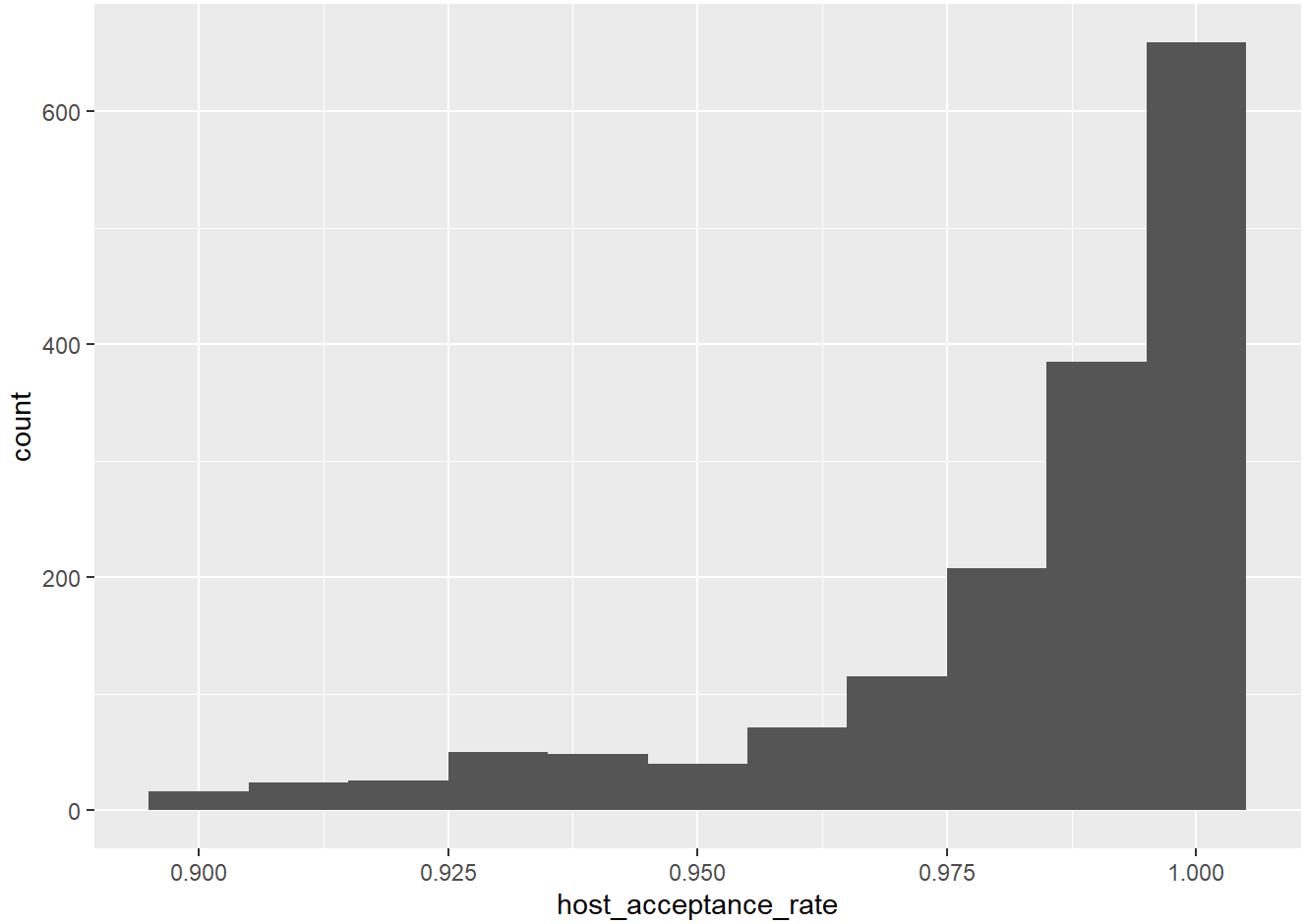

```
## box plot and histogram confirm left skewness
df_host_test = df %>%
  mutate( host_acceptance_rate2 = ((1-(host_acceptance_rate))^(1/2)))
## since data is left-skewed, can manipulate
ggplot(df_host_test, aes(host_acceptance_rate2)) + geom_histogram( binwidth = 0.05)
```




```
ggplot(df_host_test, aes(host_acceptance_rate2)) + geom_boxplot() + coord_flip()
```



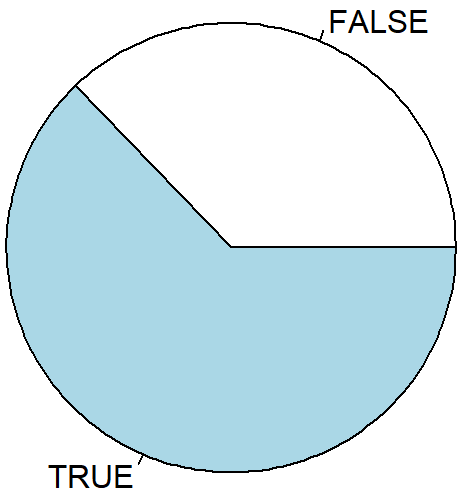

```
## Transform was not great, Outlier analysis performed
## data is very skewed so IQR outlier analysis is the best approach
IQR_host_rate = IQR(df_price$host_acceptance_rate)
upper_host_price = quantile(df_price$host_acceptance_rate , 0.75)
lower_host_price = quantile(df_price$host_acceptance_rate , 0.25)
upper_outlier_host_price = upper_host_price + 1.5 * IQR_host_rate
lower_outlier_host_price  = lower_host_price - 1.5 * IQR_host_rate

df_host_price = df_price %>%
  filter(host_acceptance_rate > lower_outlier_host_price & host_acceptance_rate < upper_outlier_
host_price)

ggplot(df_host_price , aes(host_acceptance_rate)) + geom_histogram(binwidth = 0.01)
```




```
# 4) Superhost is a logical data type -> this works but for analysis might be better to have 0 a
nd 1
# use if else for a vectorized approach of replacing all values
superhost2=ifelse(df_price$superhost == FALSE, 0 , 1)
## append the new vector to the existing data frame df
df_price = data.frame(df_price, superhost2)

superhost_freq = table(df_price$superhost)
superhost2_freq = table(df_price$superhost2)
prop.table(superhost_freq)
```

```
##
##     FALSE      TRUE
## 0.3718784 0.6281216
```

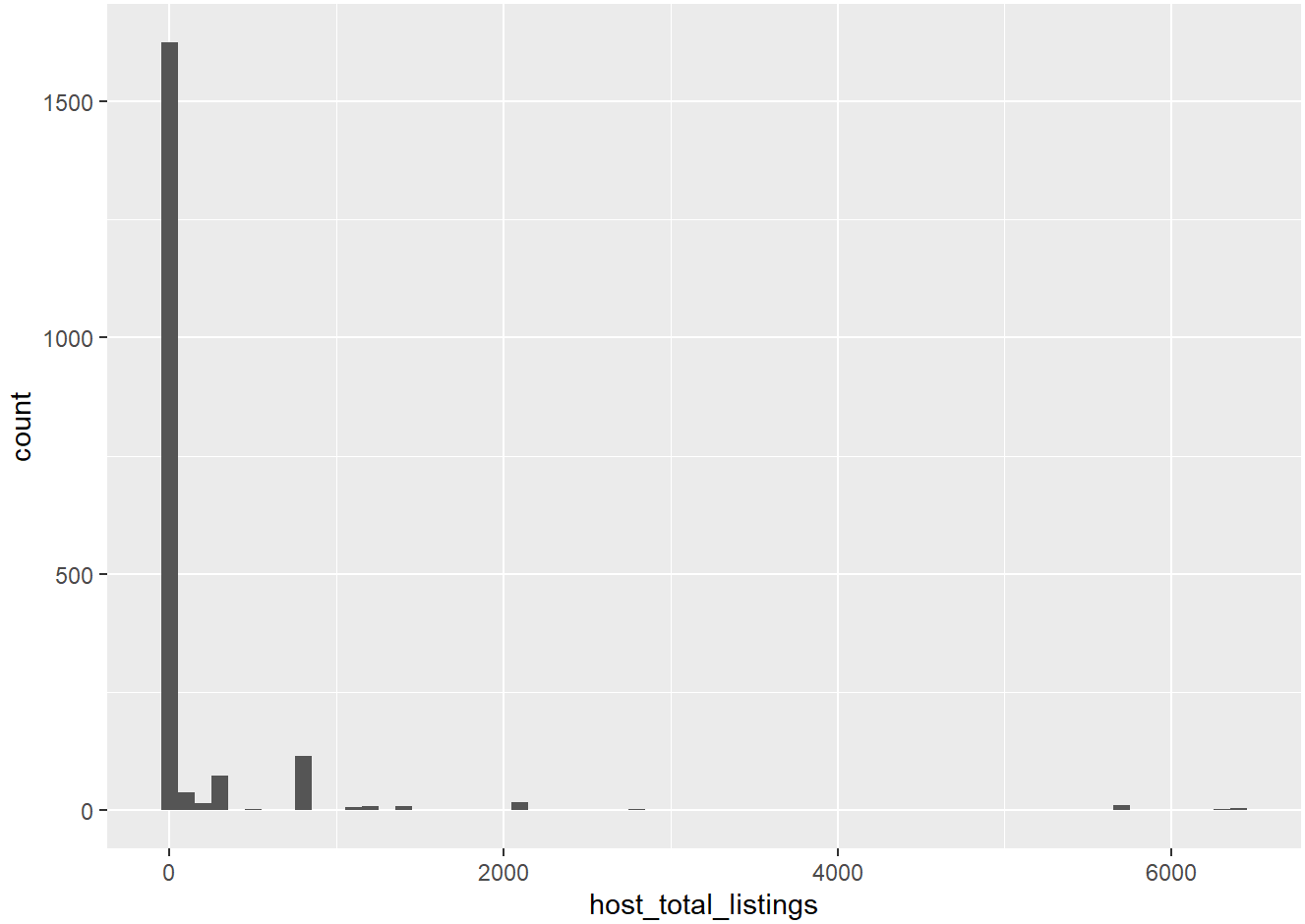
```
pie(superhost_freq)
```



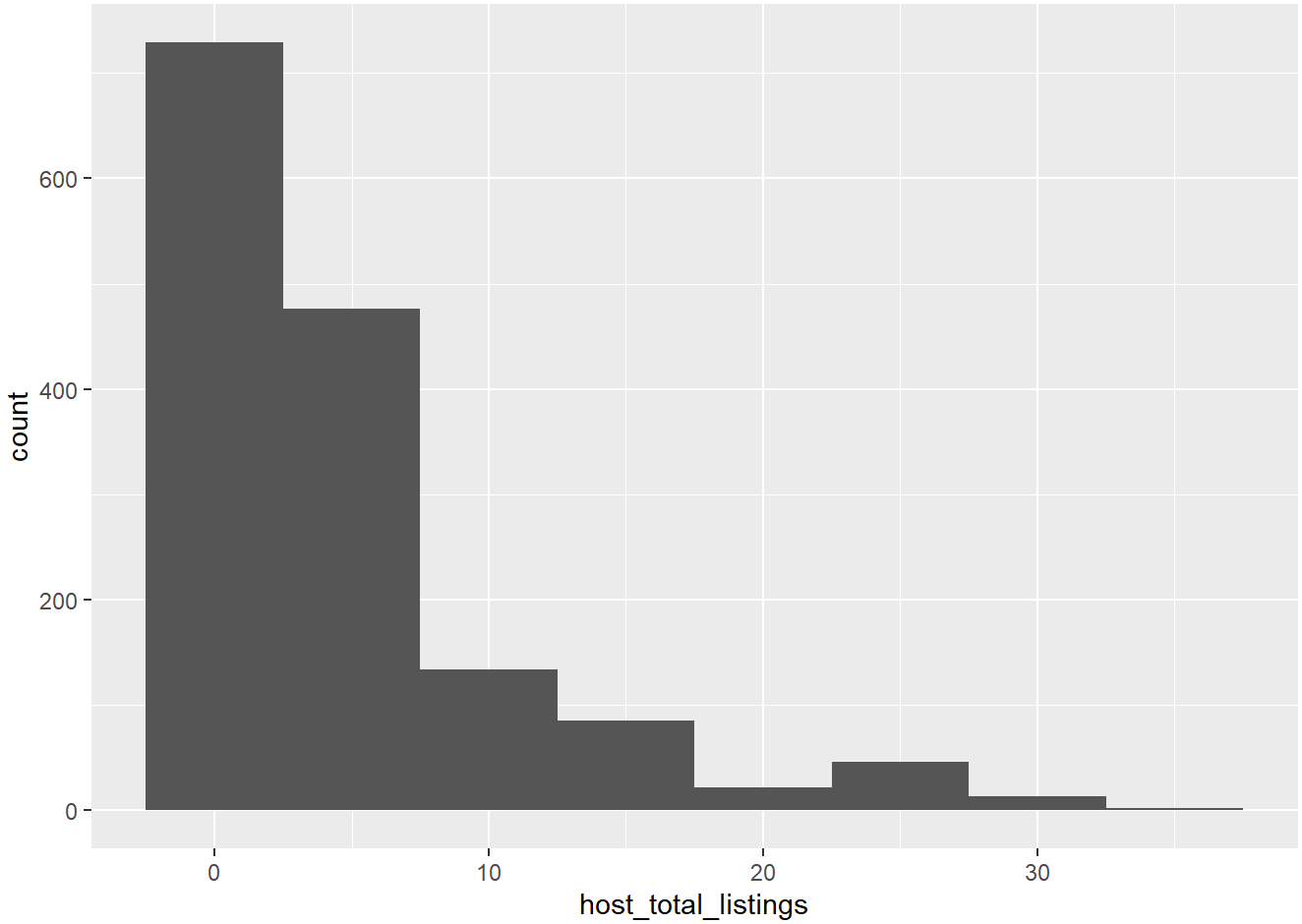

```
# pie(super host2_freq)
## approximately 2/3 of users are super hosts
## makes sense given that the host acceptance rate is so high

# 5) host_total_listing is an integer data type
## first find summary statistics
summary(df$host_total_listings)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     1.0     1.0     4.0   158.1    15.0  6397.0
```

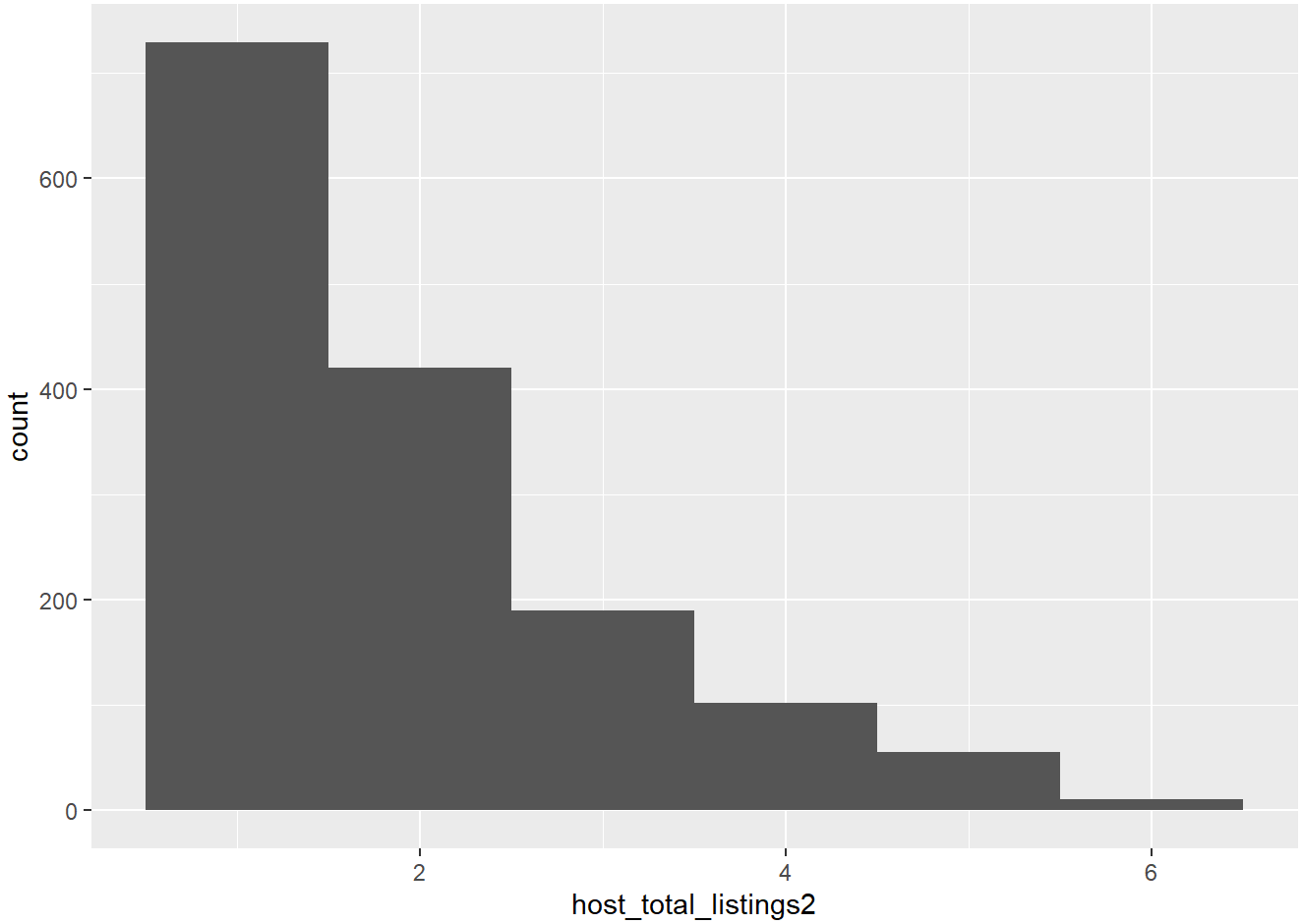
```
ggplot(df, aes(host_total_listings)) + geom_histogram(binwidth = 100)
```



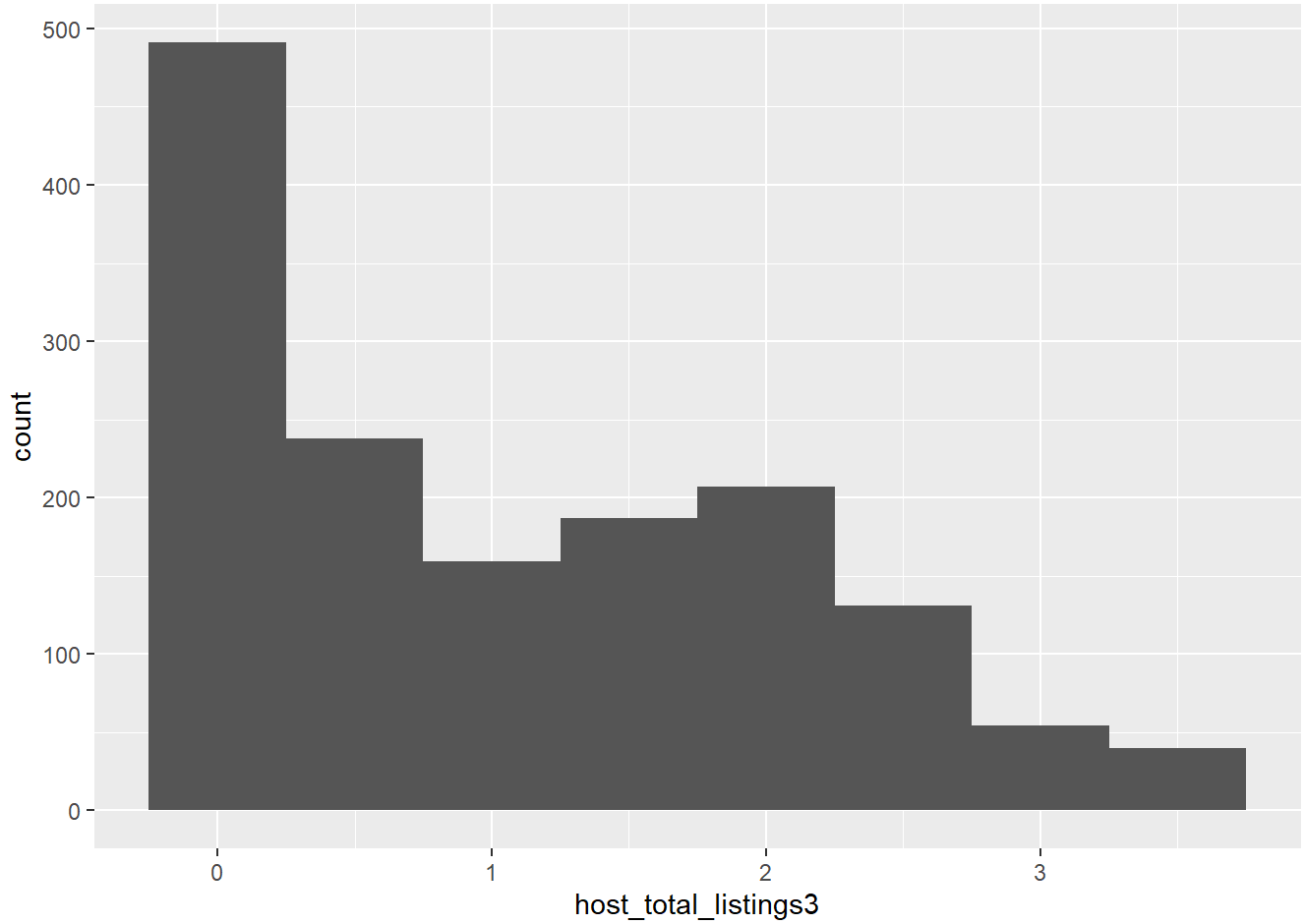

```
iqr_total_listings = IQR(df_price$host_total_listings)
upper_total_listing = quantile(df_price$host_total_listings, 0.75)
lower_total_listing = quantile(df_price$host_total_listings, 0.25)
upper_outlier_total_listings = upper_total_listing + 1.5 * iqr_total_listings
lower_outlier_total_listings = lower_total_listing - 1.5 * iqr_total_listings

df_listings_price = df_price %>%
  filter ( host_total_listings > lower_outlier_total_listings & host_total_listings < upper_outl
ier_total_listings)

ggplot(df_listings_price, aes(host_total_listings)) + geom_histogram(binwidth = 5)
```



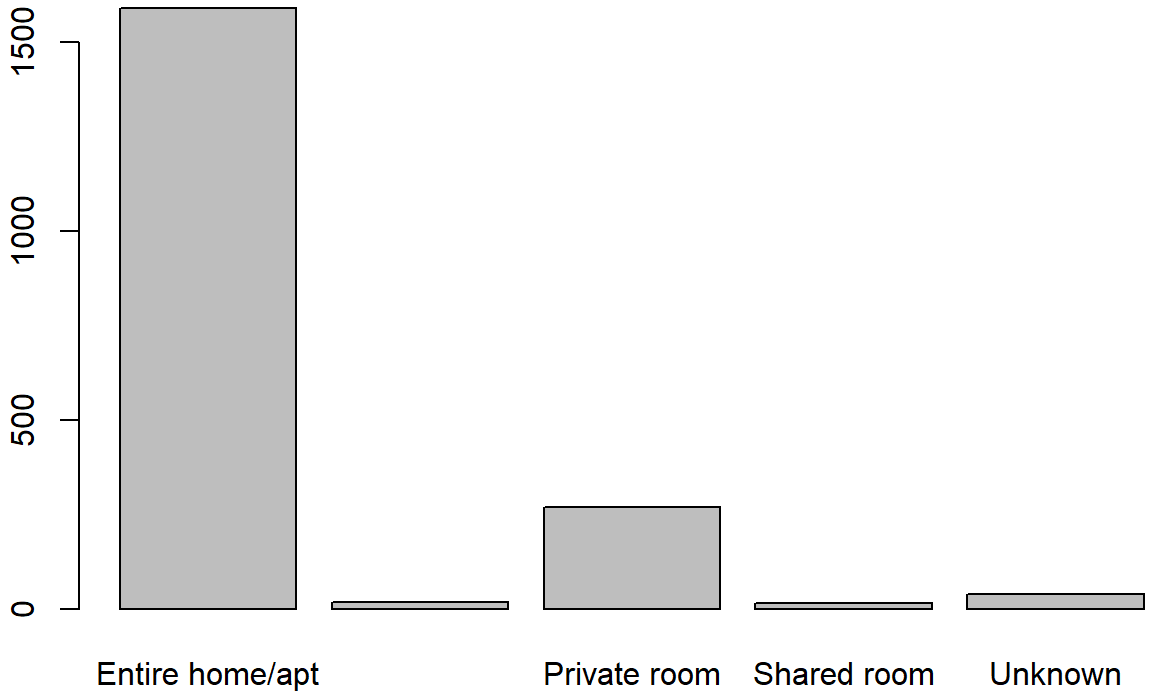

```
## data for host total listings is right skewed
## try sqrt transform

df3 = df_listings_price %>%
  mutate (host_total_listings2 = sqrt(host_total_listings),
          host_total_listings3 = log(host_total_listings))

ggplot(df3, aes(host_total_listings2)) + geom_histogram(binwidth = 1)
```




```
ggplot(df3, aes(host_total_listings3)) + geom_histogram(binwidth =0.5)
```



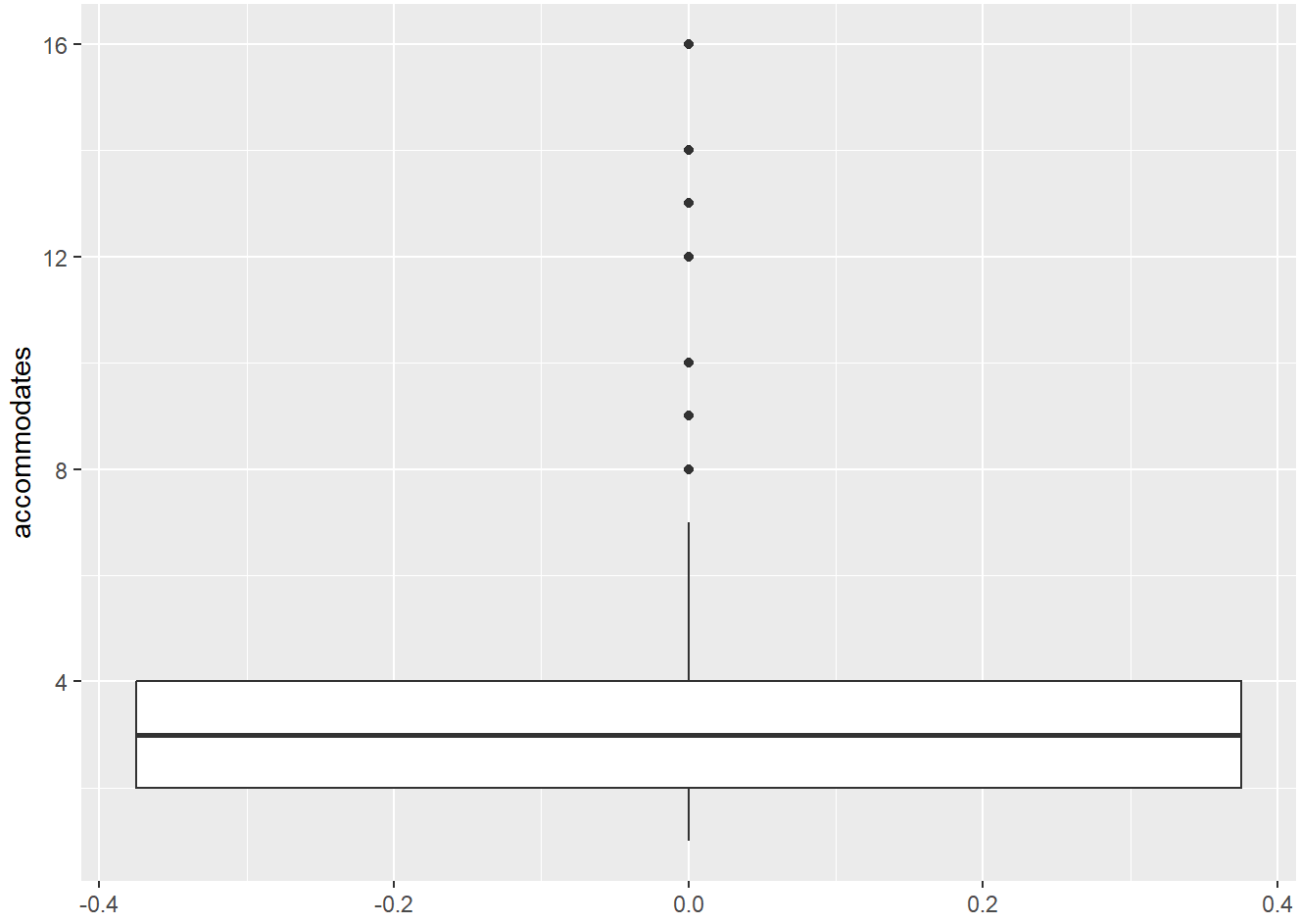

```
## log transform was a little better

# 6) room type (already took care of the blanks)
## data type is character -> lets convert this to a factor data type for better categorical anal
ysis

barplot(table(df$room_type))
```



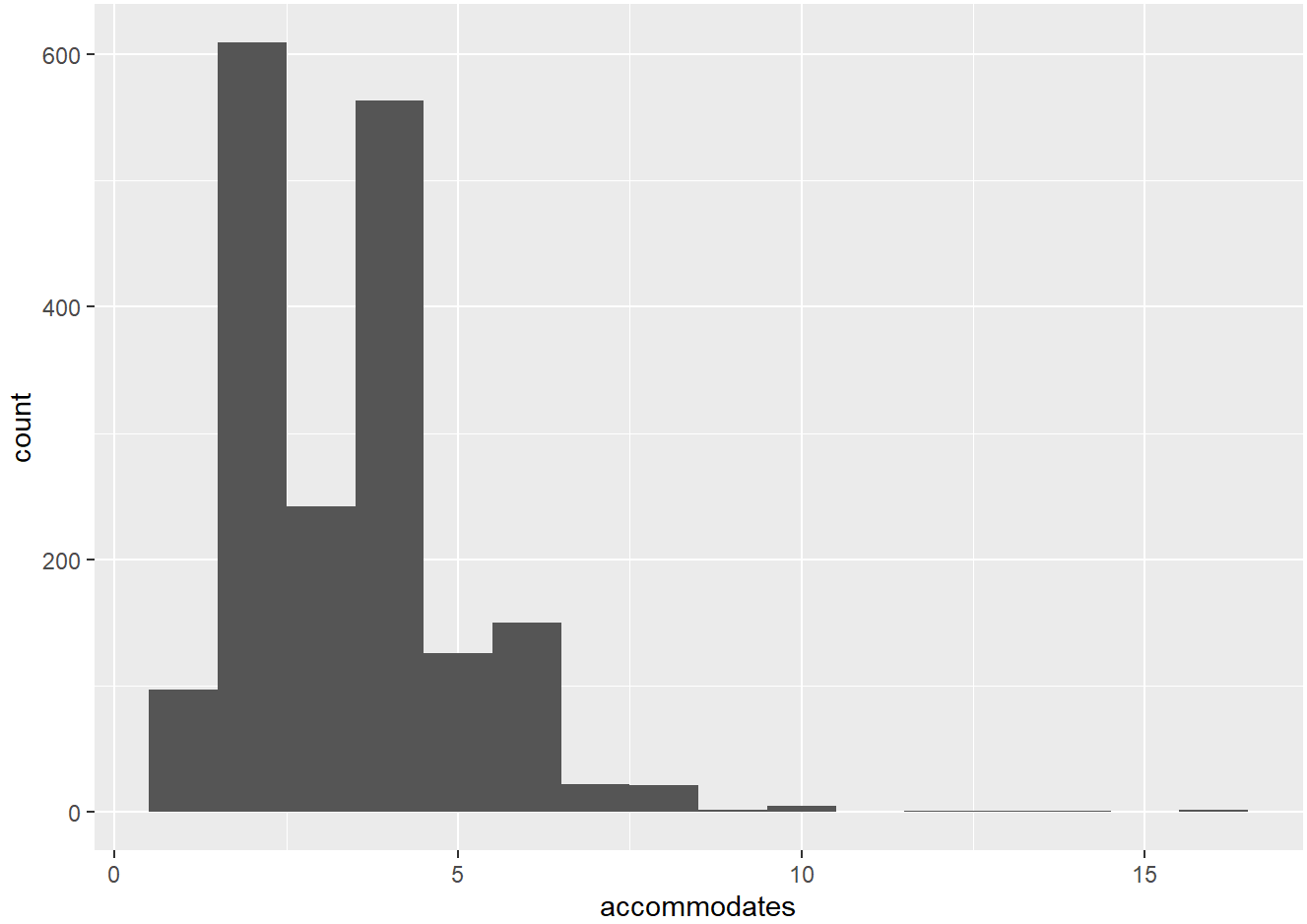

```
# some sampling error -> way to many entire home/apt compared to the others

# 7) accommodates -> is a integer data type
# first lets see the summary statistics

summary(df$accommodates)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.000   2.000   4.000   3.564   4.000  16.000
```

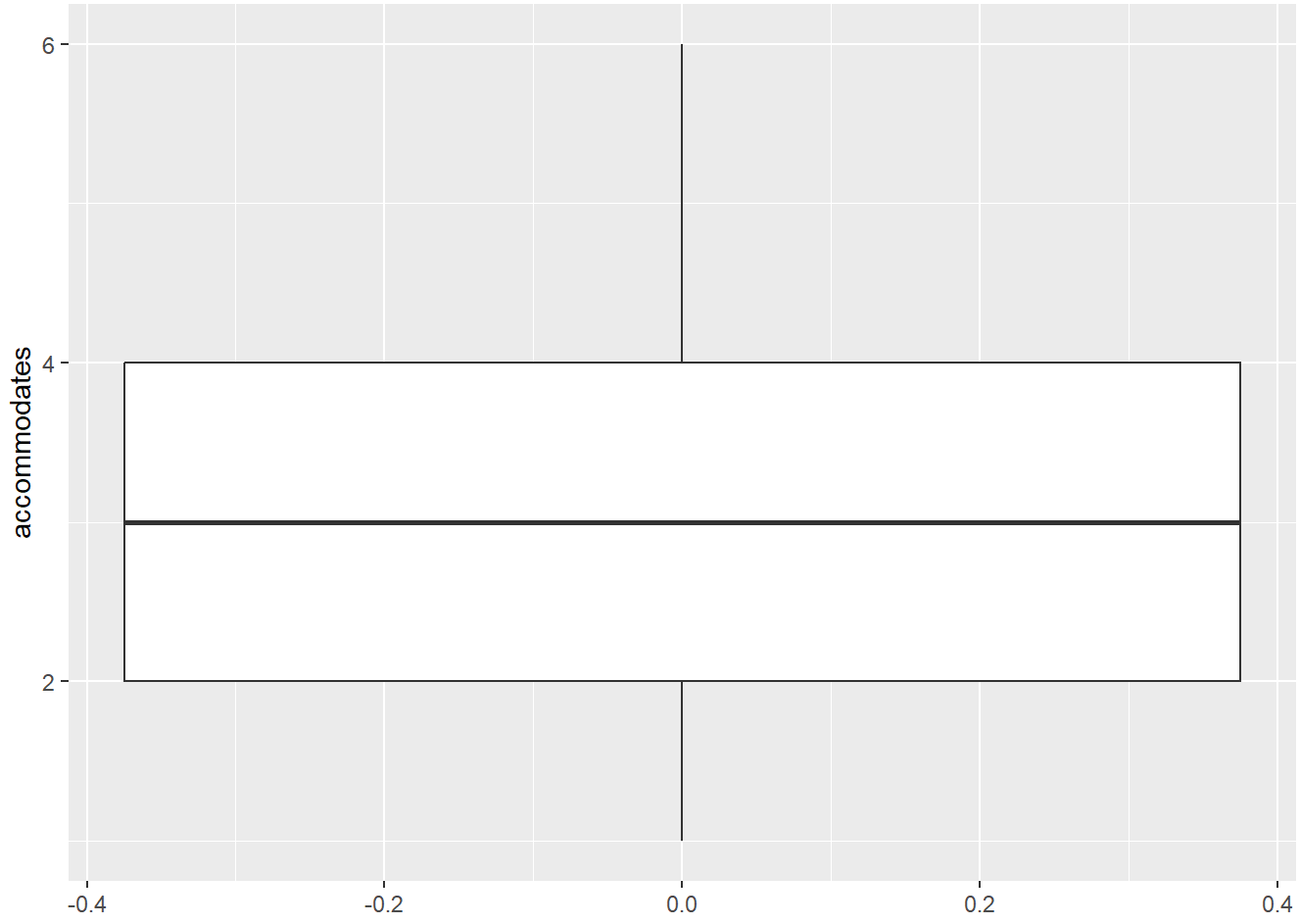
```
## check the distribution
df_price %>%
  ggplot(aes(accommodates)) + geom_boxplot() + coord_flip()
```



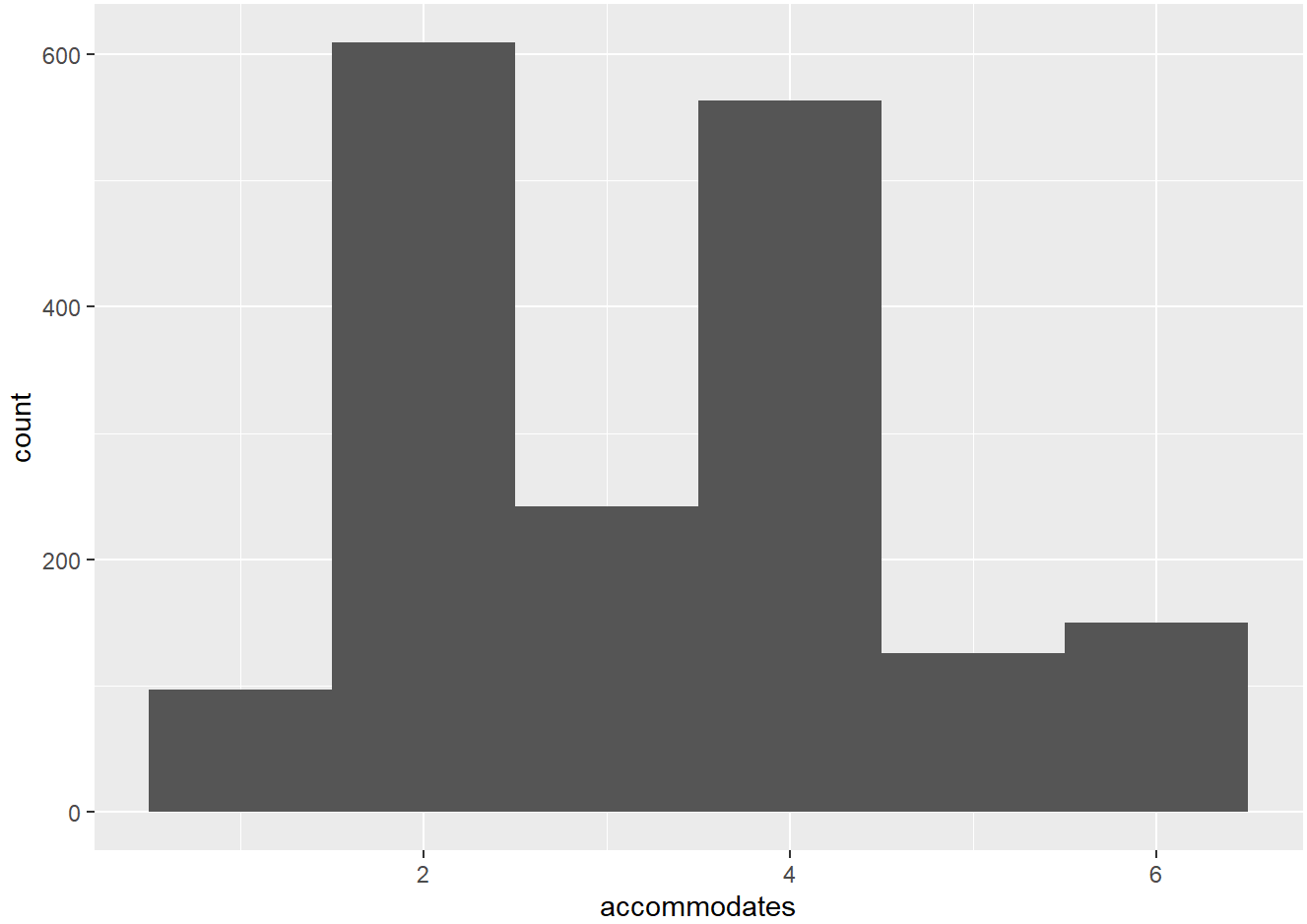

```
df_price %>%
  ggplot(aes(accommodates)) + geom_histogram(binwidth = 1)
```




```
# bimodal distribution, slightly skewed to the right
IQR_ap = IQR(df_price$accommodates)
upper_a_p = quantile(df_price$accommodates, 0.75)
lower_a_p = quantile(df_price$accommodates, 0.25)
upper_outlier_ap = upper_a_p + 1.5*IQR_ap
lower_outlier_ap = lower_a_p - 1.5*IQR_ap

df_a_p = df_price %>%
  filter(accommodates > lower_outlier_ap & accommodates < upper_outlier_ap)
df_a_p %>%
  ggplot(aes(accommodates)) + geom_boxplot() + coord_flip()
```



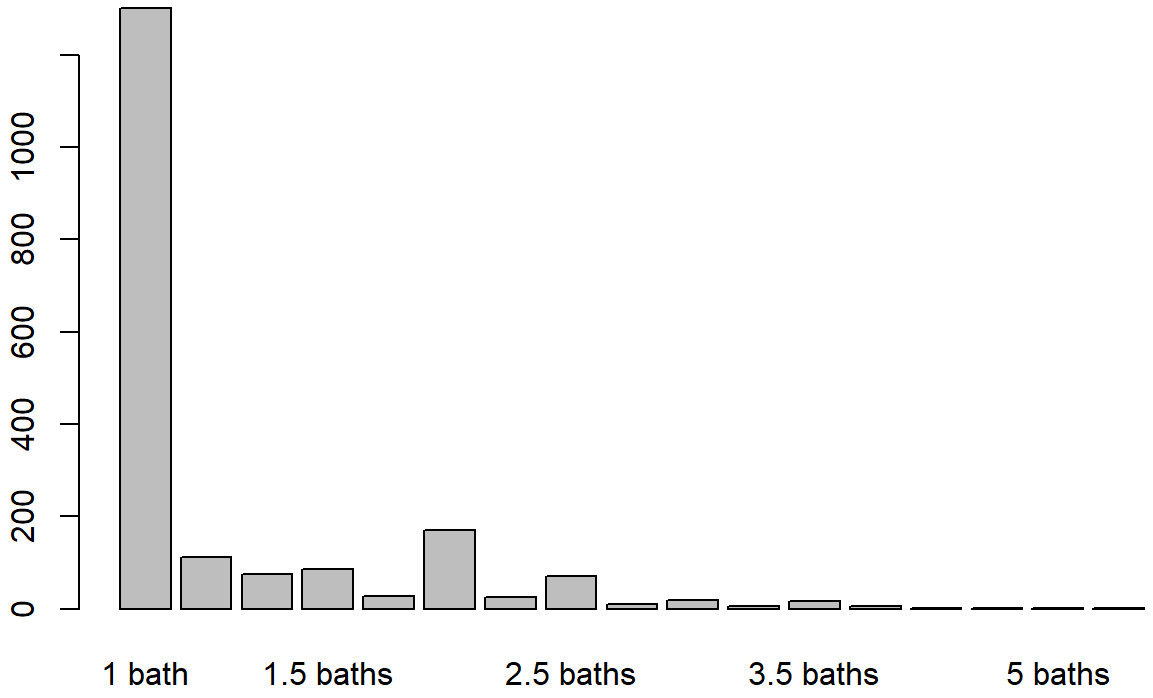

```
df_a_p %>%
  ggplot(aes(accommodates)) + geom_histogram(binwidth = 1)
```




```
# 8) bathrooms is character data type -> convert to factor

bathroom_freq = table(df$bathrooms)
prop.table(bathroom_freq)
```

```
##
##           1 bath   1 private bath    1 shared bath        1.5 baths
##     0.6737441740     0.0580010357     0.0383221129     0.0445365096
## 1.5 shared baths          2 baths   2 shared baths        2.5 baths
##     0.0145002589     0.0885551528     0.0134645262     0.0367685137
## 2.5 shared baths          3 baths   3 shared baths        3.5 baths
##     0.0046607975     0.0098394614     0.0031071983     0.0088037286
##          4 baths        4.5 baths 4.5 shared baths          5 baths
##     0.0031071983     0.0010357328     0.0005178664     0.0005178664
## Shared half-bath
##     0.0005178664
```

```
barplot(bathroom_freq)
```

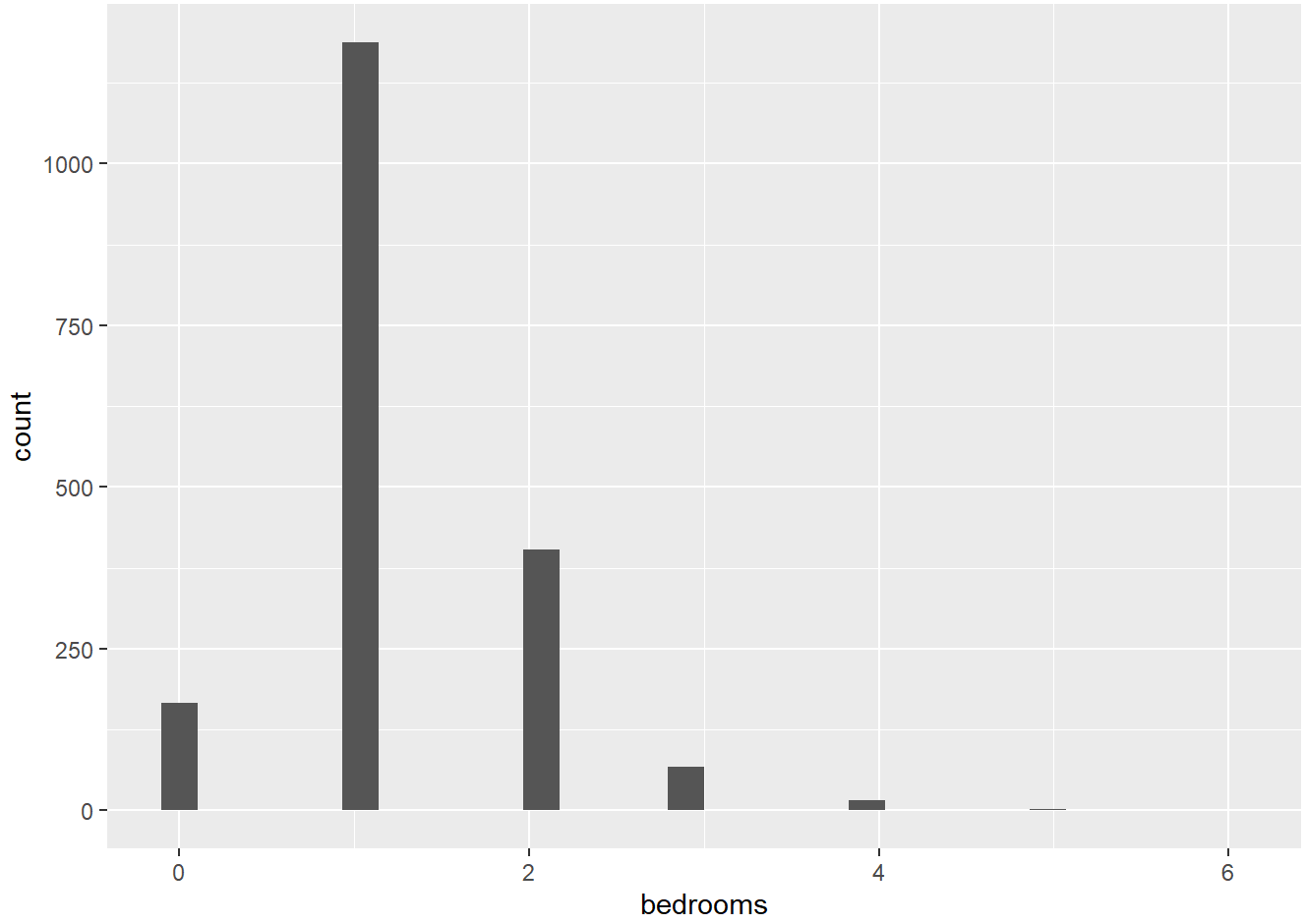



```
## lots of 1 bath -> highly imbalanced sampling

# 9) bedrooms -> int data type (makes sense, but could possibly use this as a categorical variab
le as well)

summary(df_price$bedrooms)
```

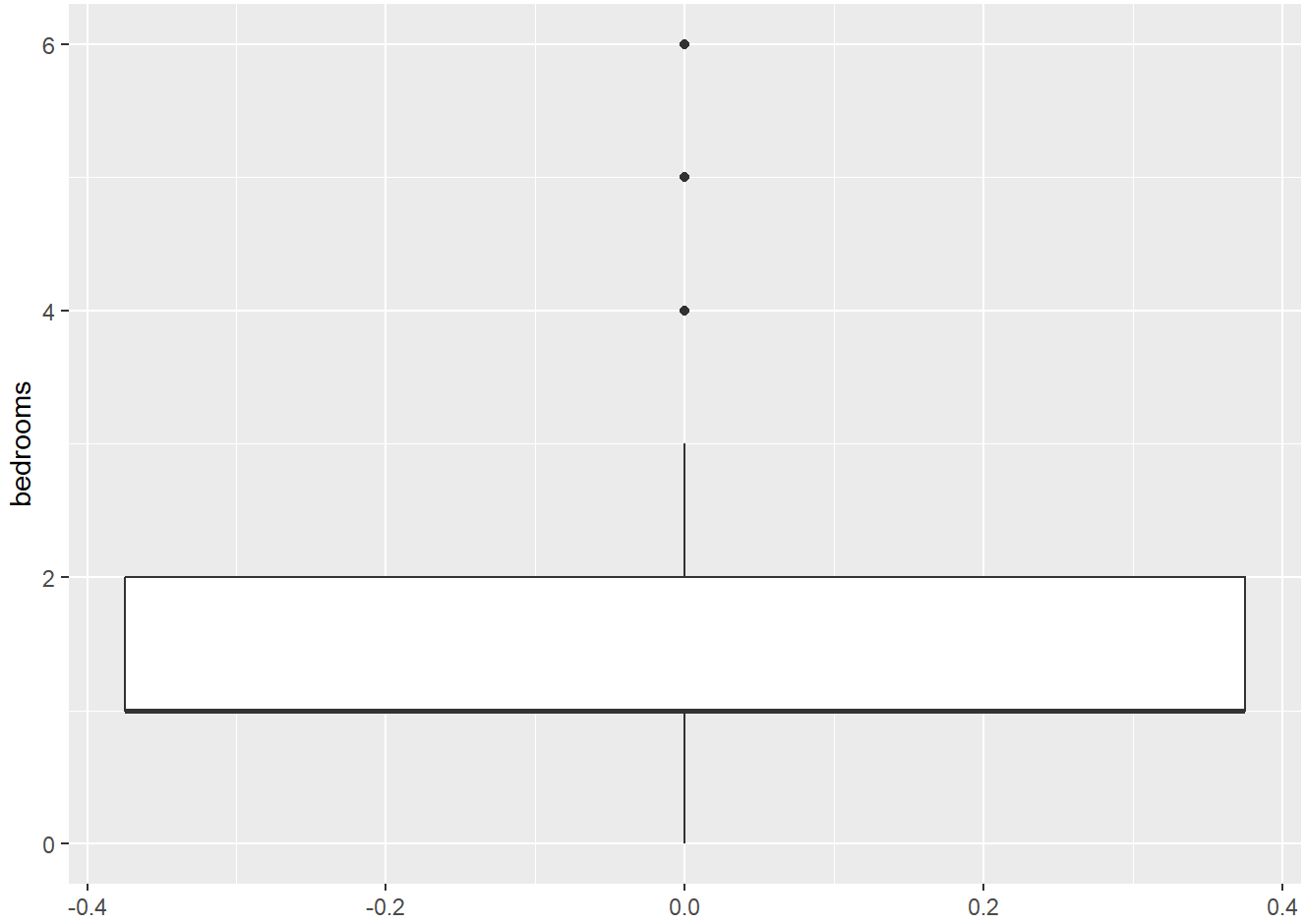
```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.000   1.000   1.000   1.234   2.000   6.000
```

```
ggplot(df_price, aes(bedrooms)) + geom_histogram()
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



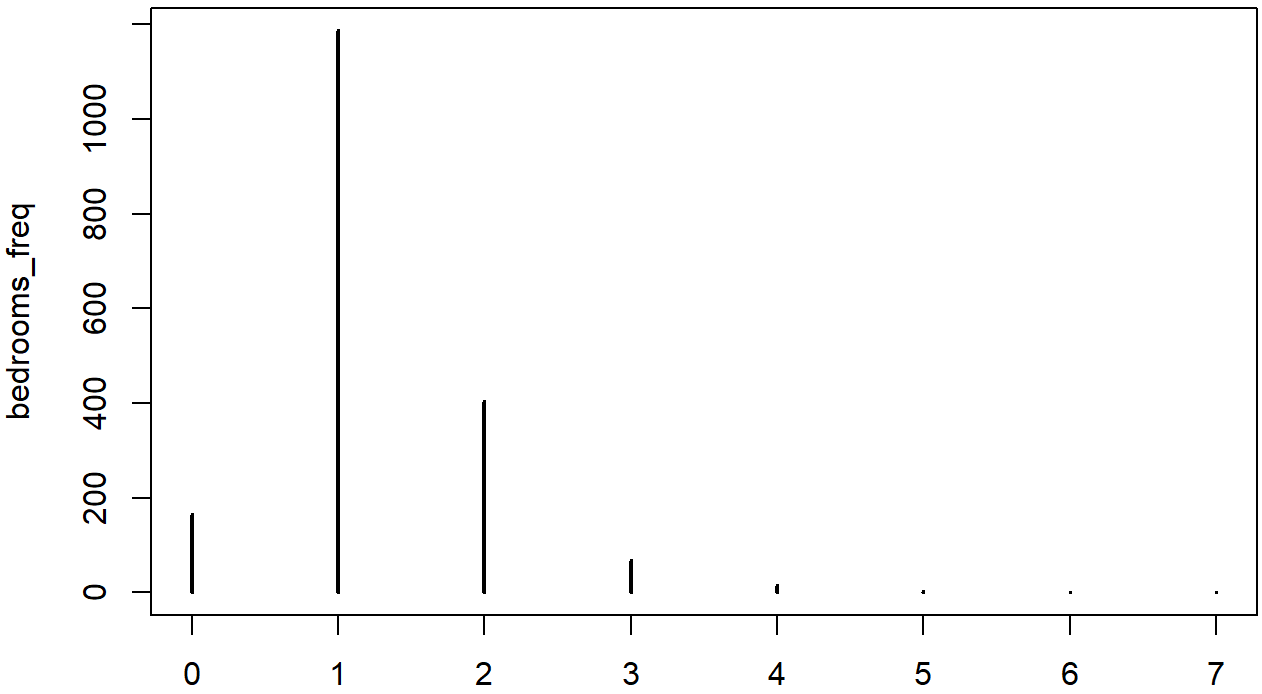

```
ggplot(df_price, aes(bedrooms)) + geom_boxplot() +coord_flip()
```



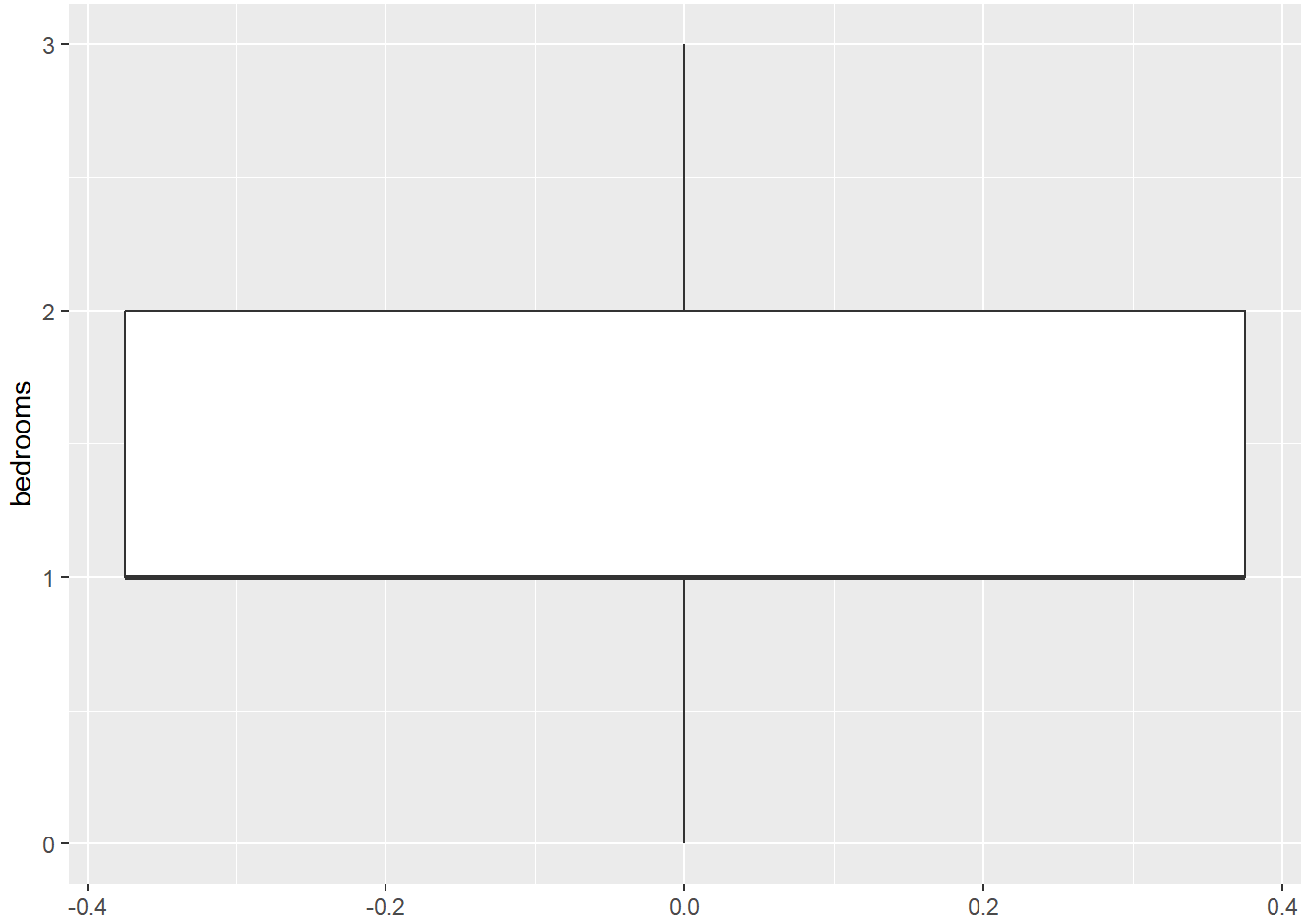

```
bedrooms_freq = table(df_price$bedrooms_f)
prop.table(bedrooms_freq)
```

```
##
##            0            1            2            3            4            5
## 0.0901194354 0.6444082519 0.2187839305 0.0369163952 0.0081433225 0.0010857763
##            6            7
## 0.0005428882 0.0000000000
```

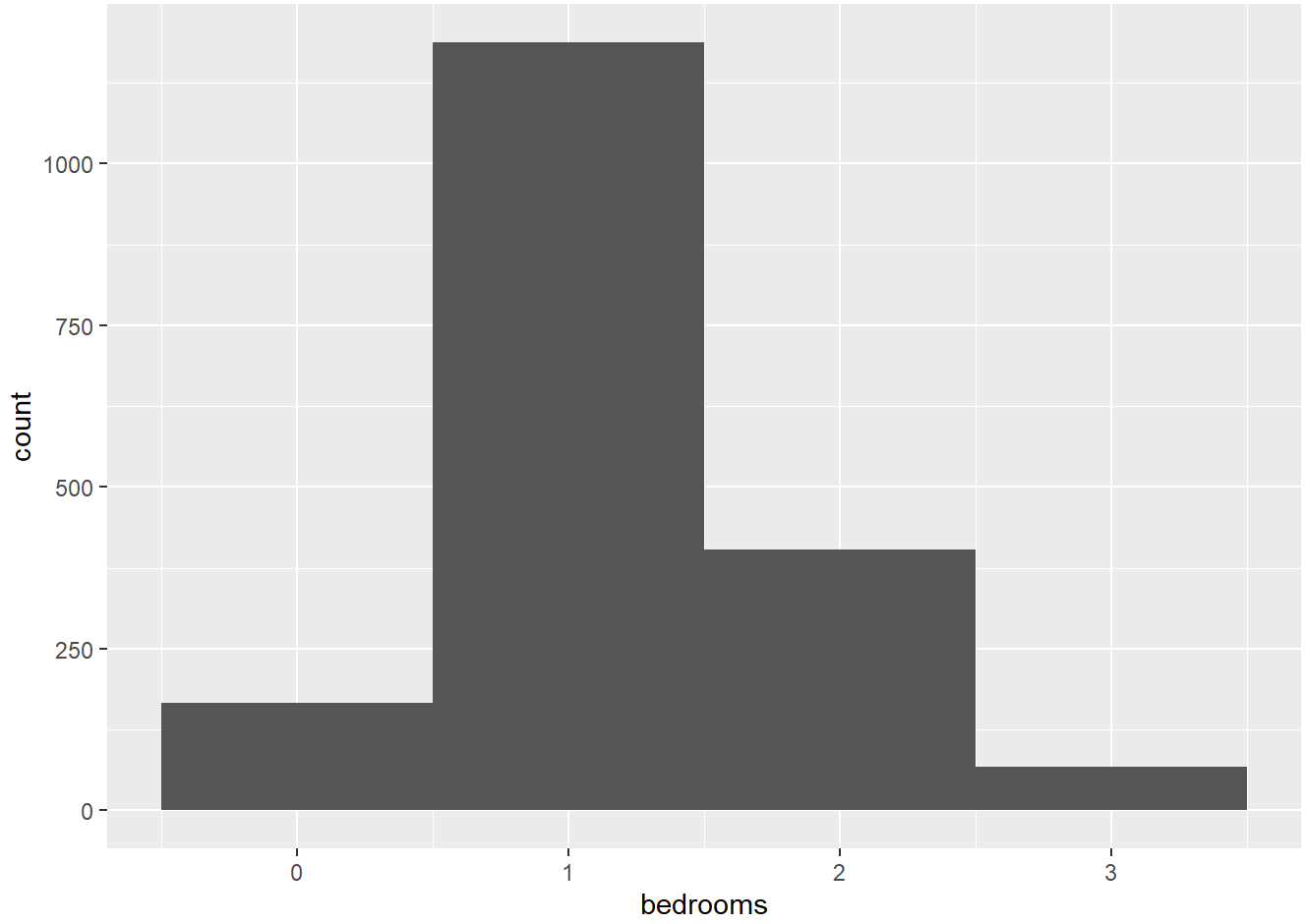
```
plot(bedrooms_freq)
```




```
IQR_pb = IQR(df_price$bedrooms)
upper_pb = quantile(df_price$bedrooms, 0.75)
lower_pb = quantile(df_price$bedrooms, 0.25)
upper_outlier_pb = upper_pb + 1.5*IQR_pb
lower_outlier_pb = lower_pb - 1.5*IQR_pb

df_b_p = df_price %>%
  filter( bedrooms > lower_outlier_pb & bedrooms < upper_outlier_pb )

df_b_p  %>%
  ggplot(aes(bedrooms)) + geom_boxplot() + coord_flip()
```



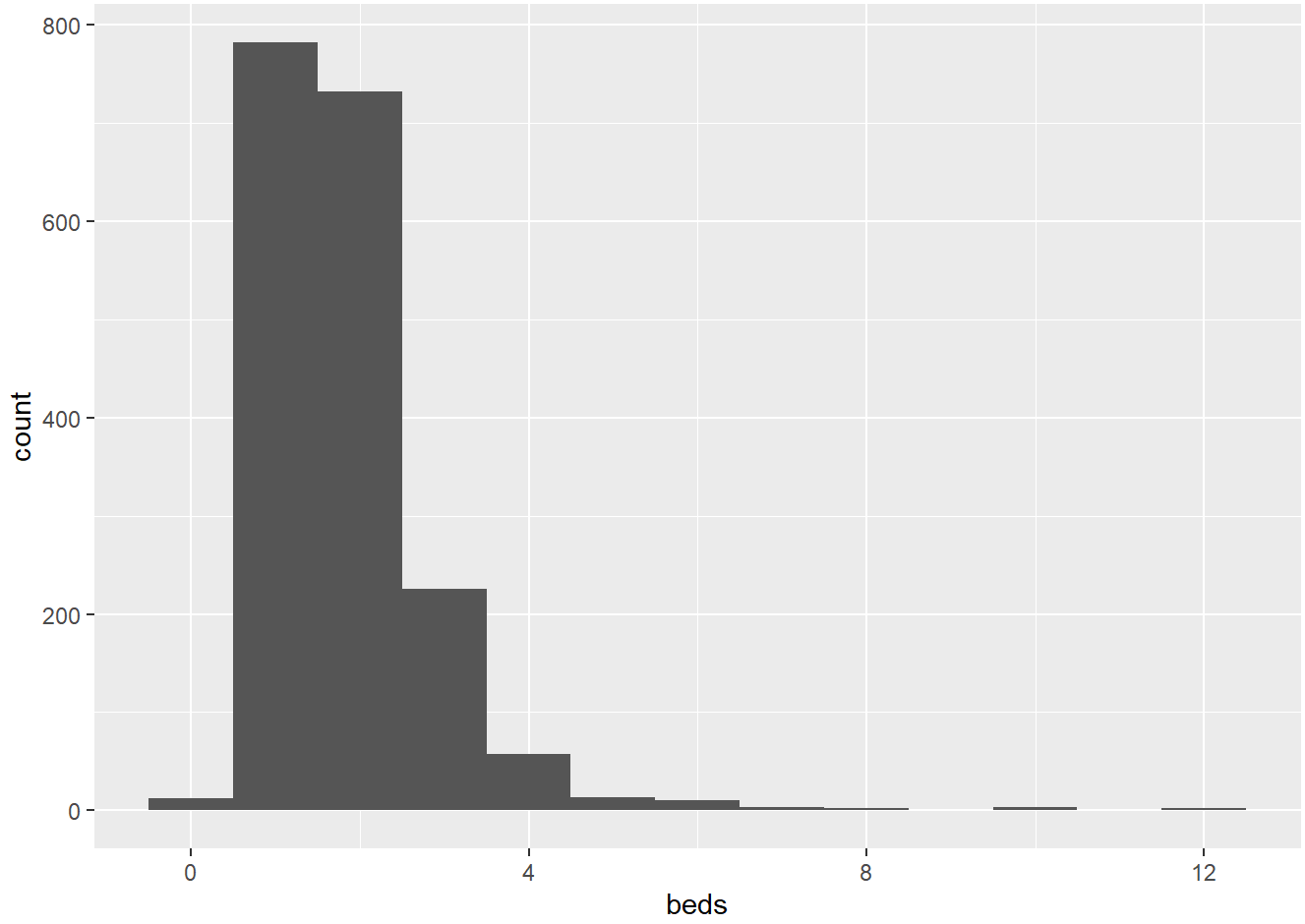

```
df_b_p  %>%
  ggplot(aes(bedrooms)) + geom_histogram(binwidth = 1)
```



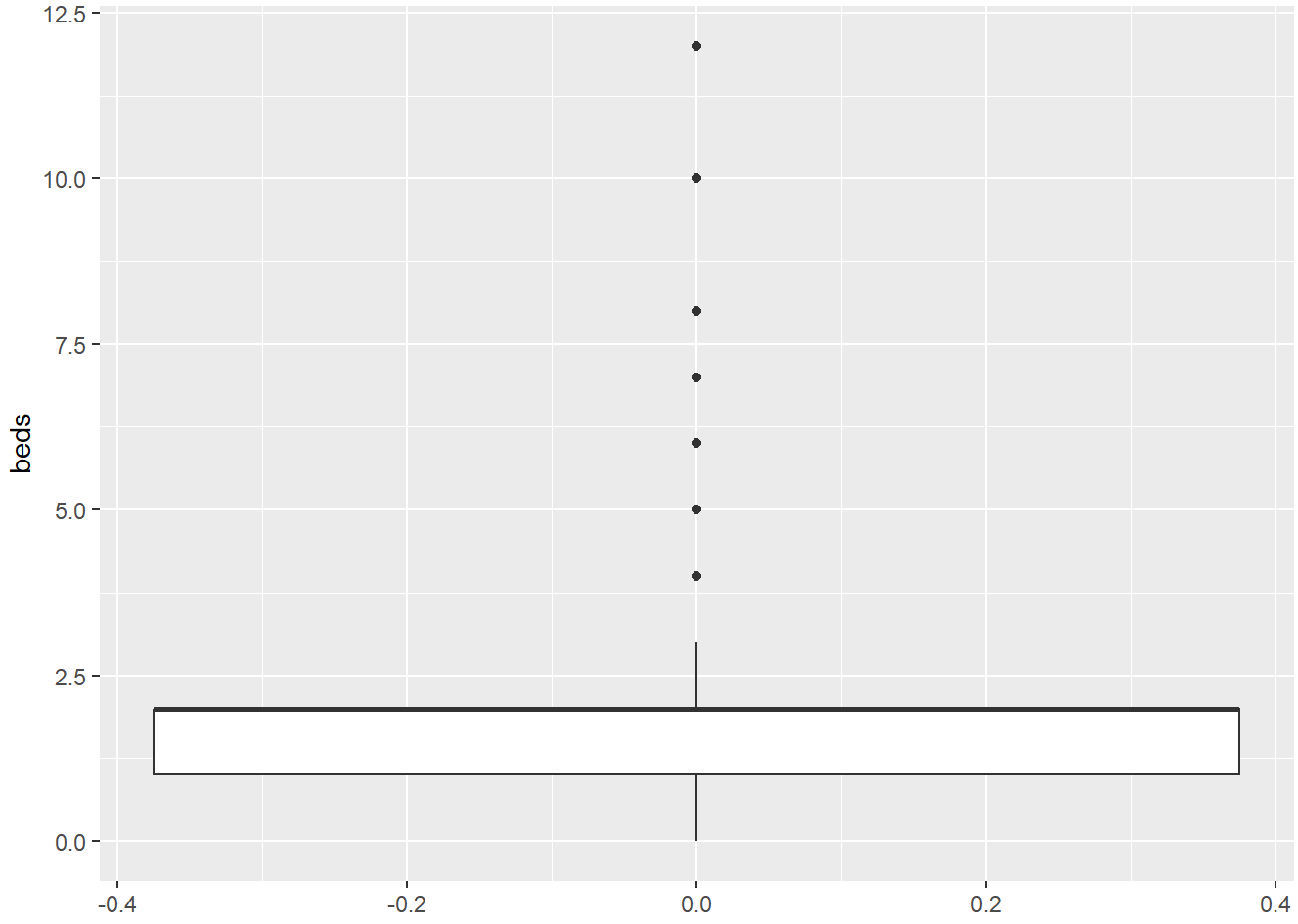

```
# 10) beds -> double data type

summary(df_price$beds)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.000   1.000   2.000   1.828   2.000  12.000
```

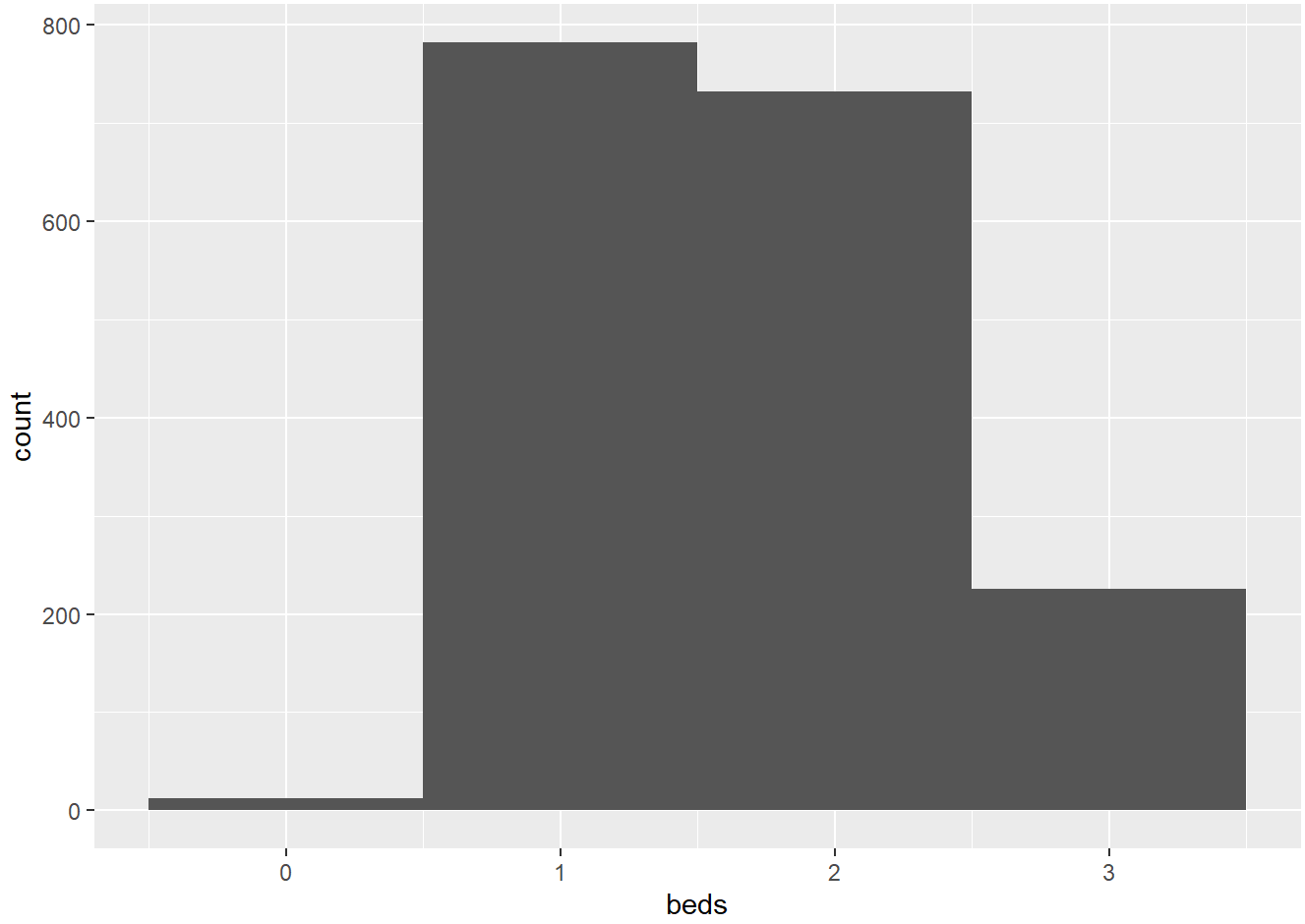
```
ggplot(df_price, aes(beds)) + geom_histogram(binwidth = 1)
```



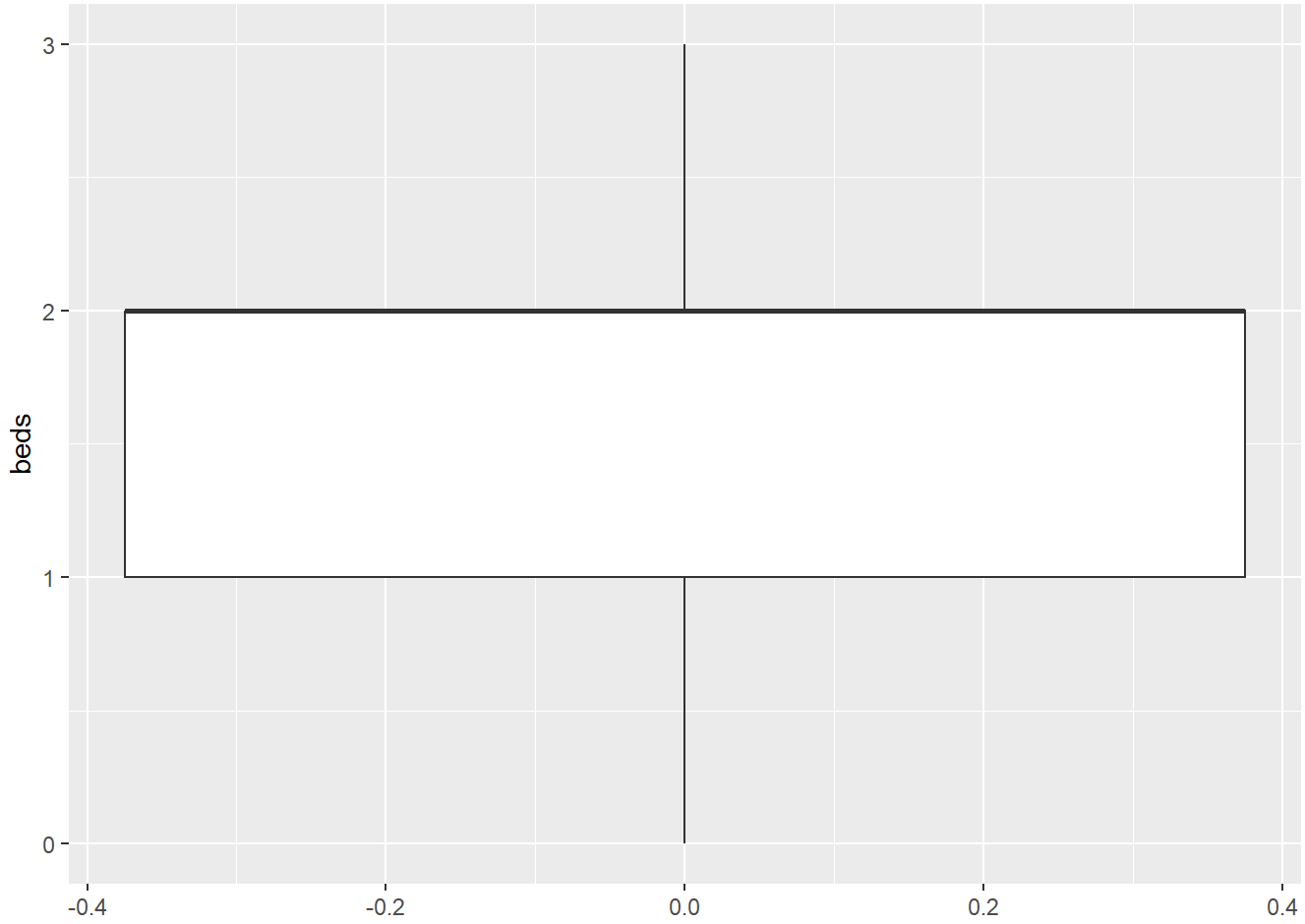

```
ggplot(df_price, aes(beds)) + geom_boxplot() + coord_flip()
```




```
IQR_bed_p = IQR(df_price$beds)
upper_bed_p = quantile(df_price$beds , 0.75)
lower_bed_p = quantile(df_price$beds, 0.25)
upper_outlier_bed_p= upper_bed_p + 1.5 * IQR_bed_p
lower_outlier_bed_p = lower_bed_p - 1.5 * IQR_bed_p

df_bed_p = df_price %>%
  filter(beds > lower_outlier_bed_p & beds < upper_outlier_bed_p)

df_bed_p %>%
  ggplot(aes(beds)) + geom_histogram(binwidth = 1)
```




```
df_bed_p %>%
  ggplot(aes(beds)) + geom_boxplot()+ coord_flip()
```

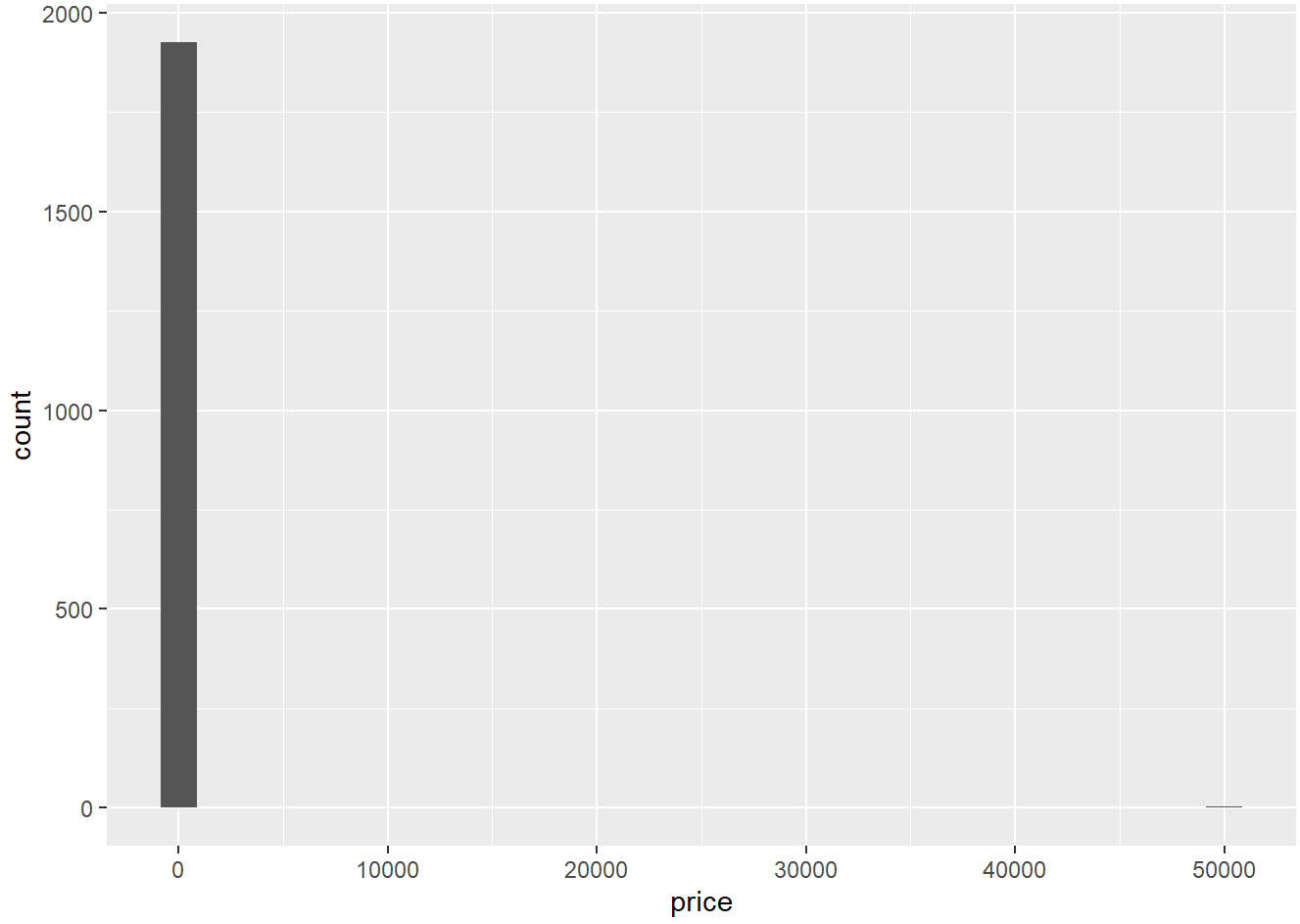



```
# 11) price is a double -> find summary statistics
summary(df$price)
```

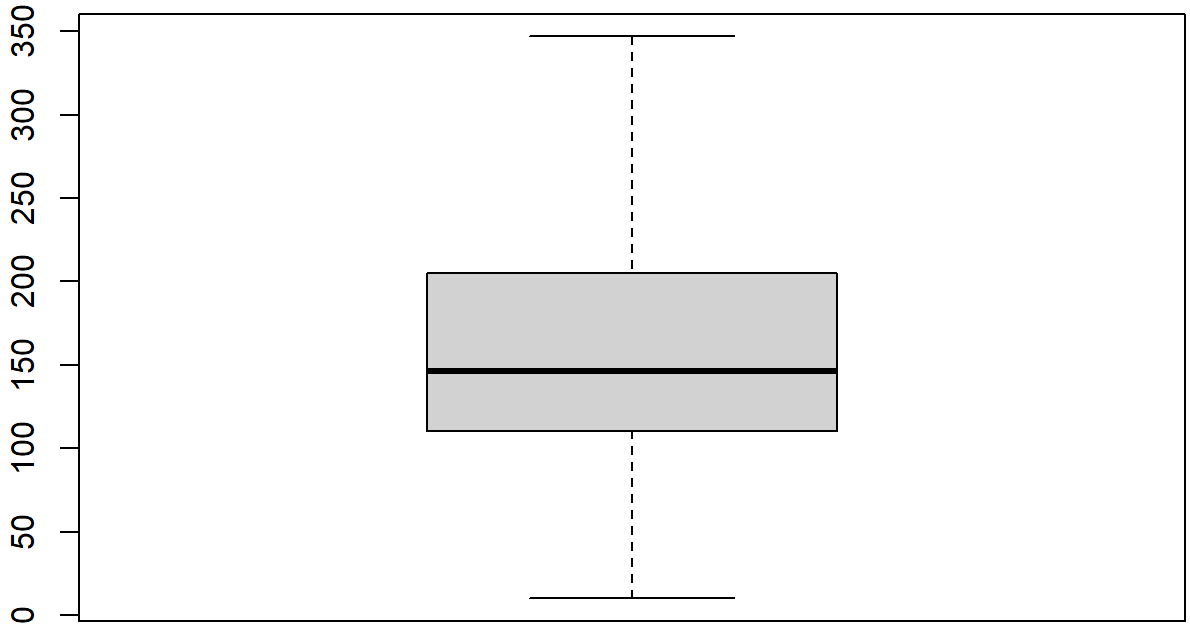
```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    10.0   110.0   146.0   272.4   205.0 50000.0
```

```
ggplot(df, aes(price)) + geom_histogram()
```

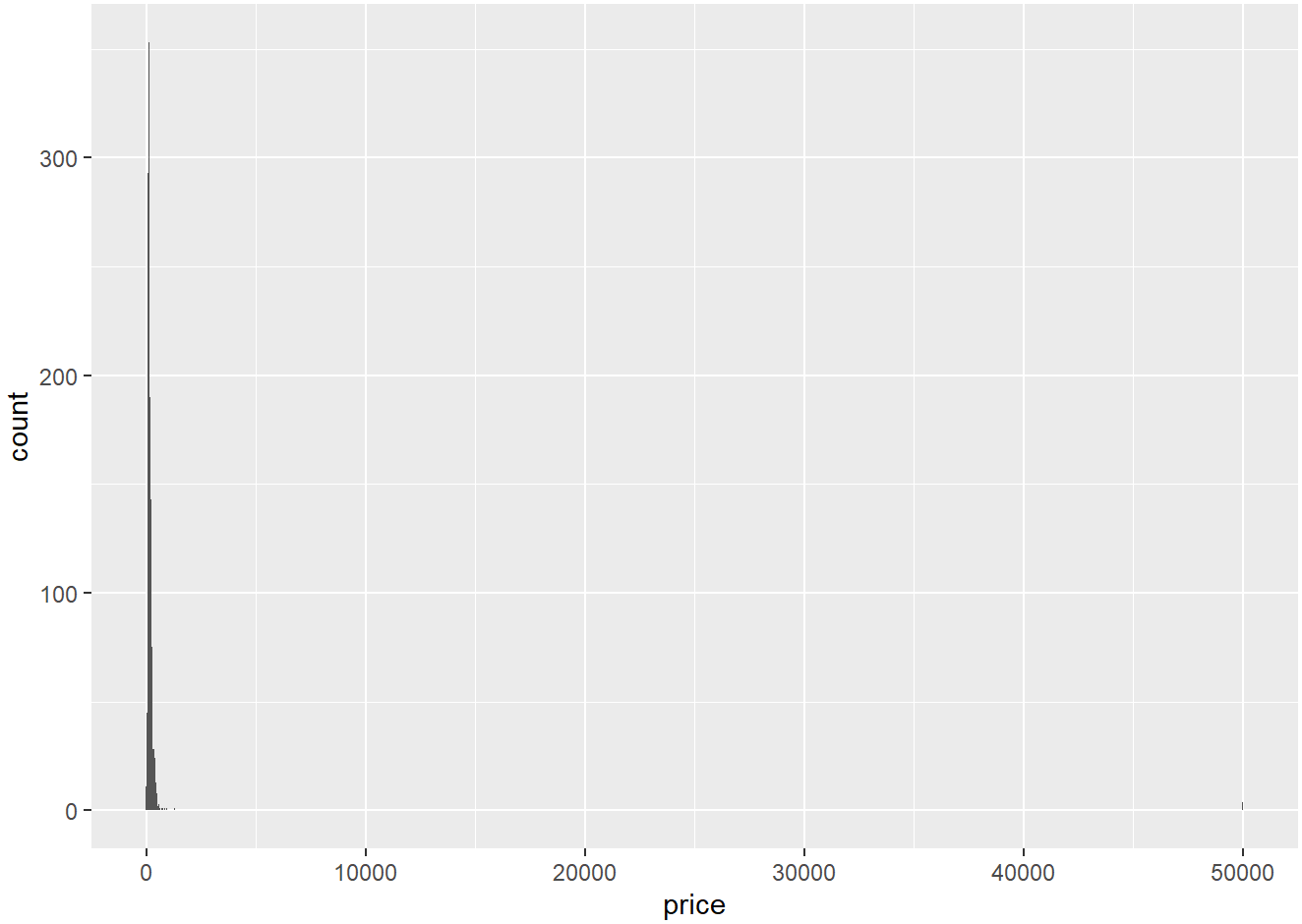
```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



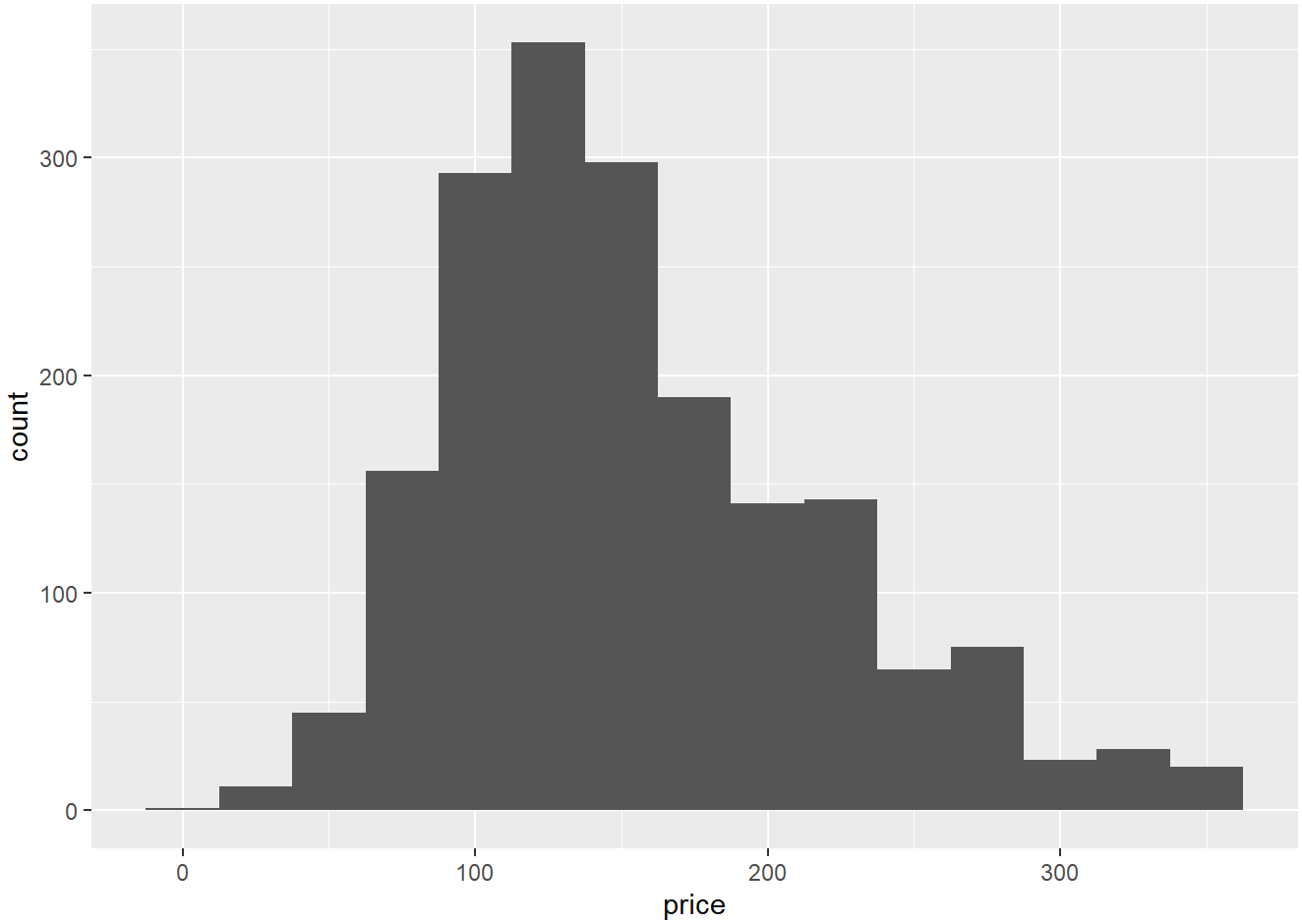

```
# there was a few data points that are clear outliers < 50000
# simple test to first visualize the data in a box plot with these outliers removed
boxplot(df$price , outline = F)
```



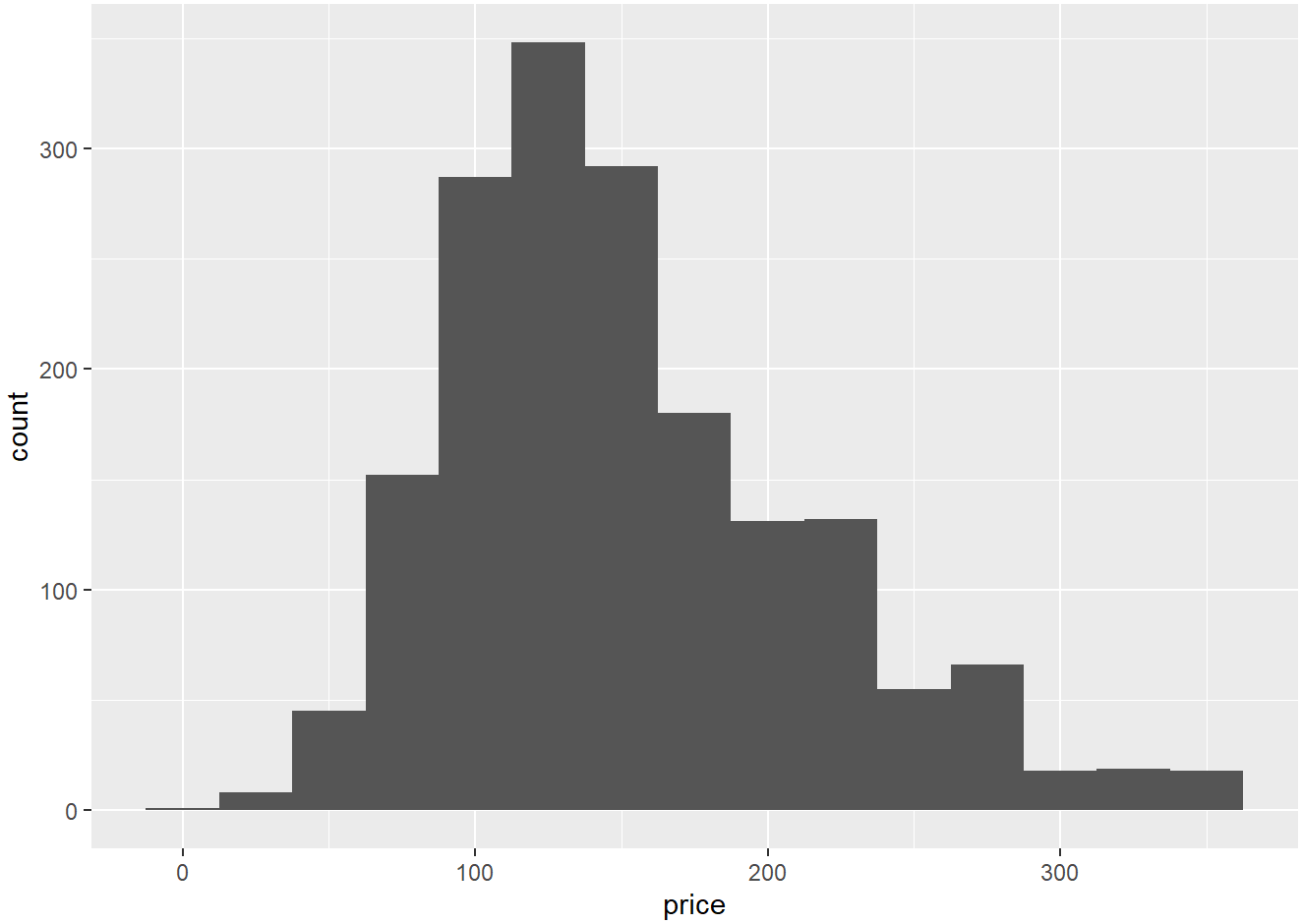

```
ggplot(df , aes(price)) + geom_histogram(binwidth = 25)
```



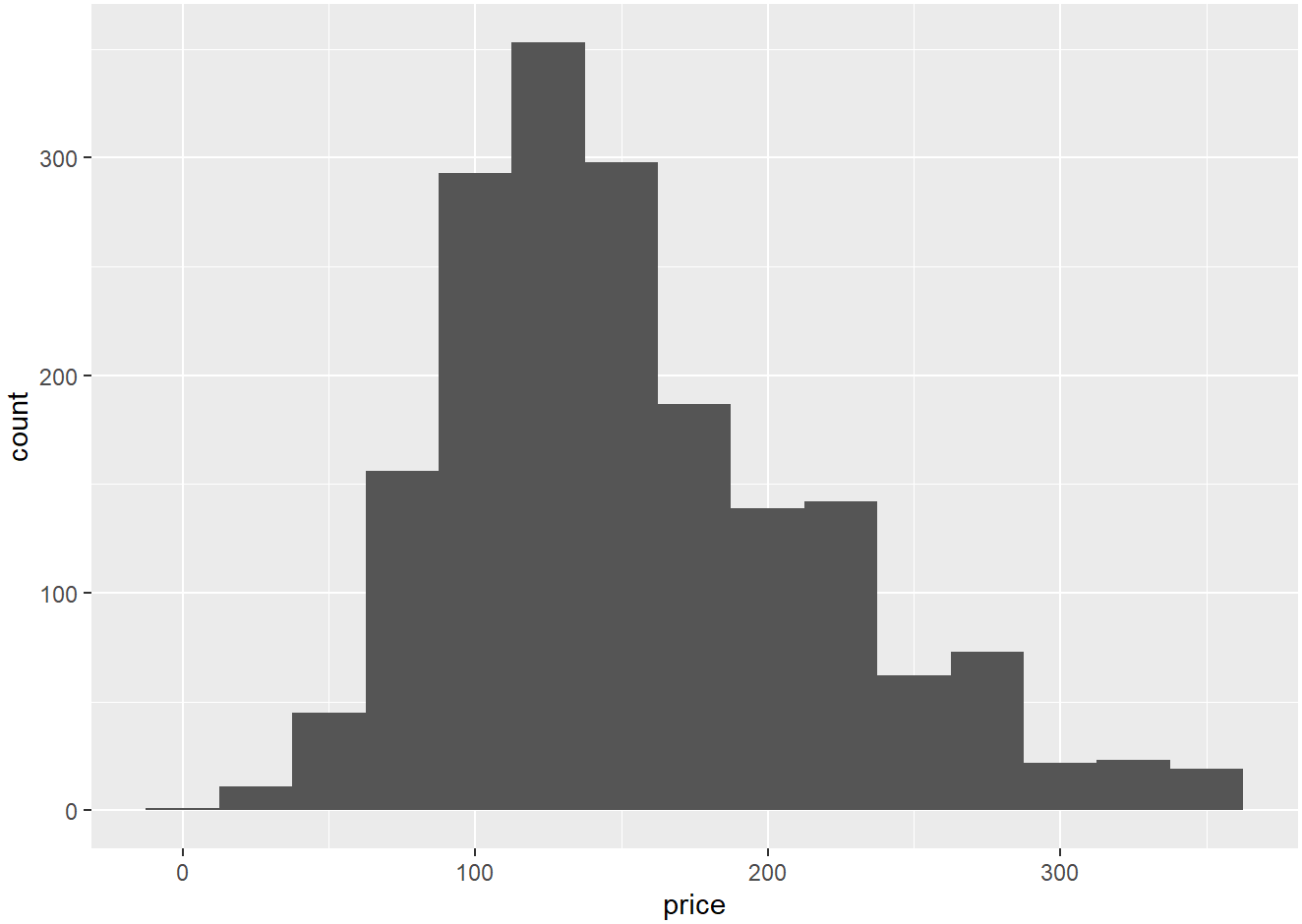

```
ggplot(df_price , aes(price)) + geom_histogram(binwidth = 25)
```



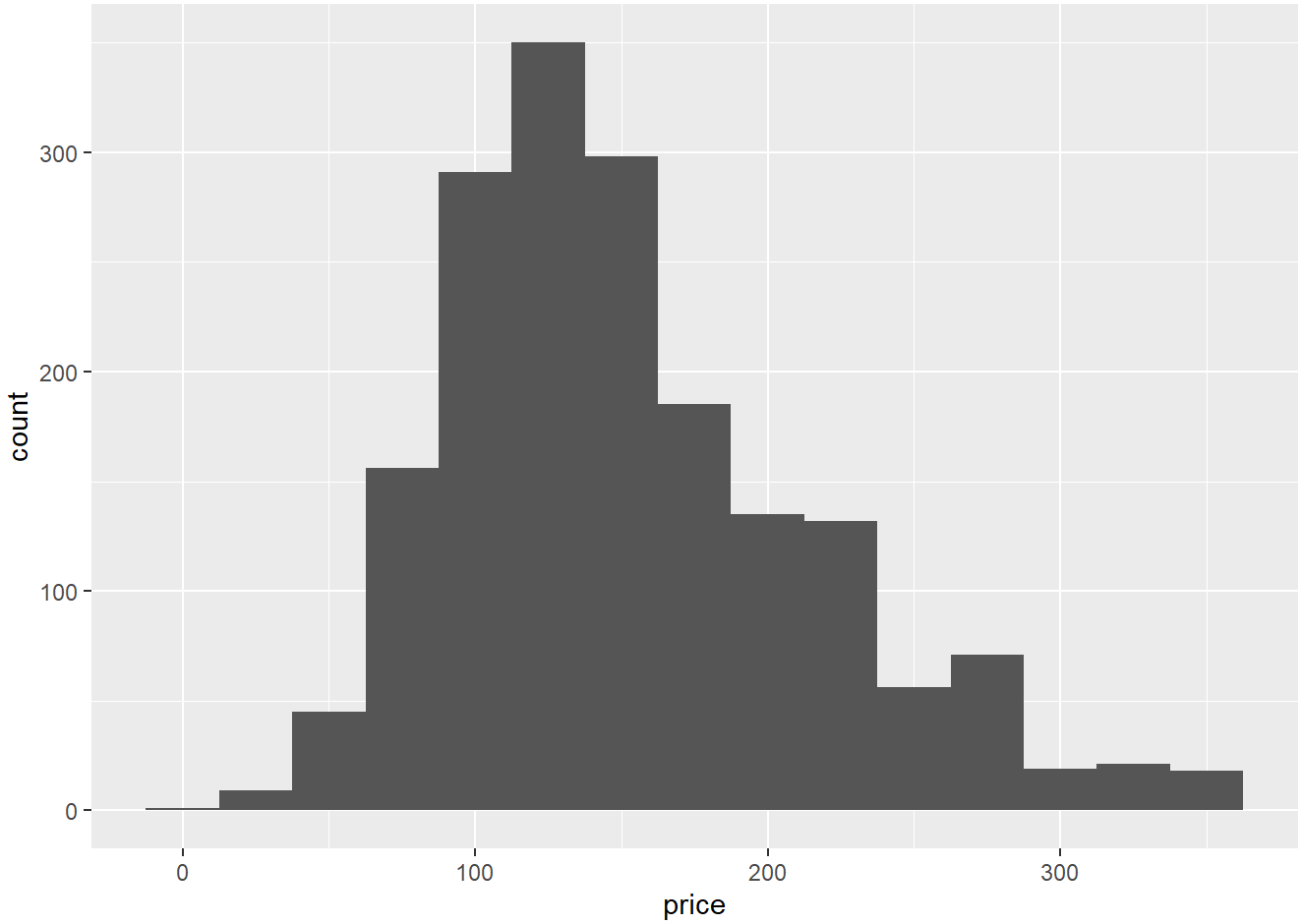

```
ggplot(df_bed_p, aes(price)) + geom_histogram(binwidth = 25)
```




```
ggplot(df_b_p, aes(price)) + geom_histogram(binwidth = 25)
```



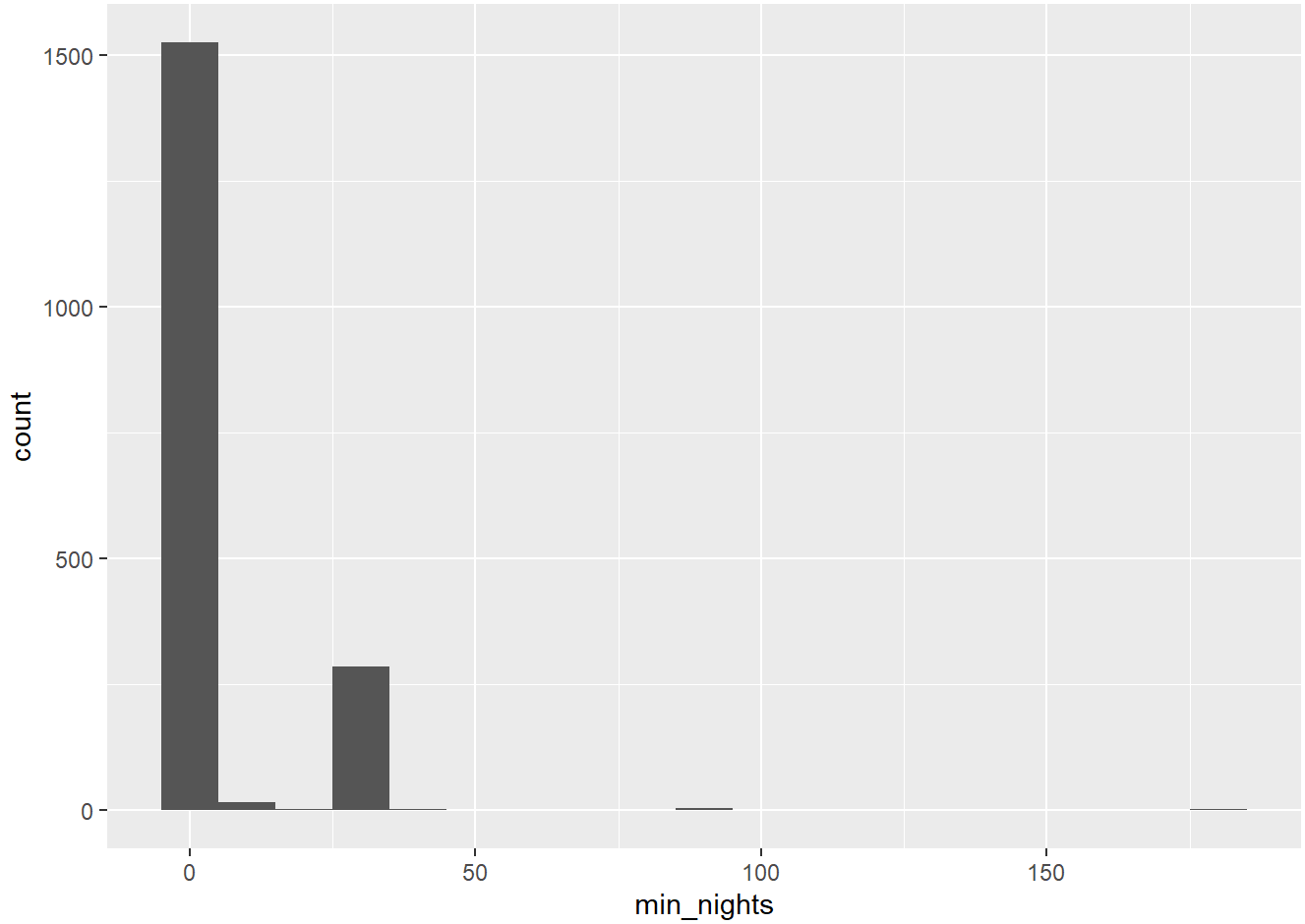

```
ggplot(df_a_p, aes(price)) + geom_histogram(binwidth = 25)
```



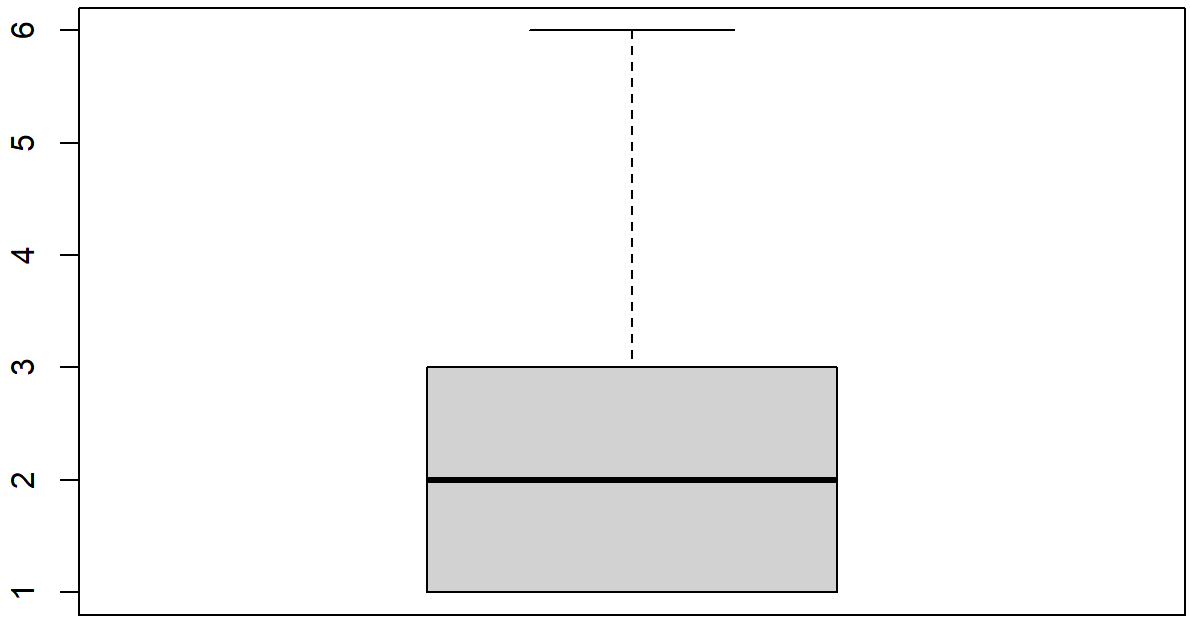

```
# 12) min_nights is an integer data type
summary(df_price$min_nights)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.000   1.000   2.000   7.104   3.000 182.000
```

```
ggplot(df_price, aes(min_nights)) + geom_histogram(binwidth = 10)
```



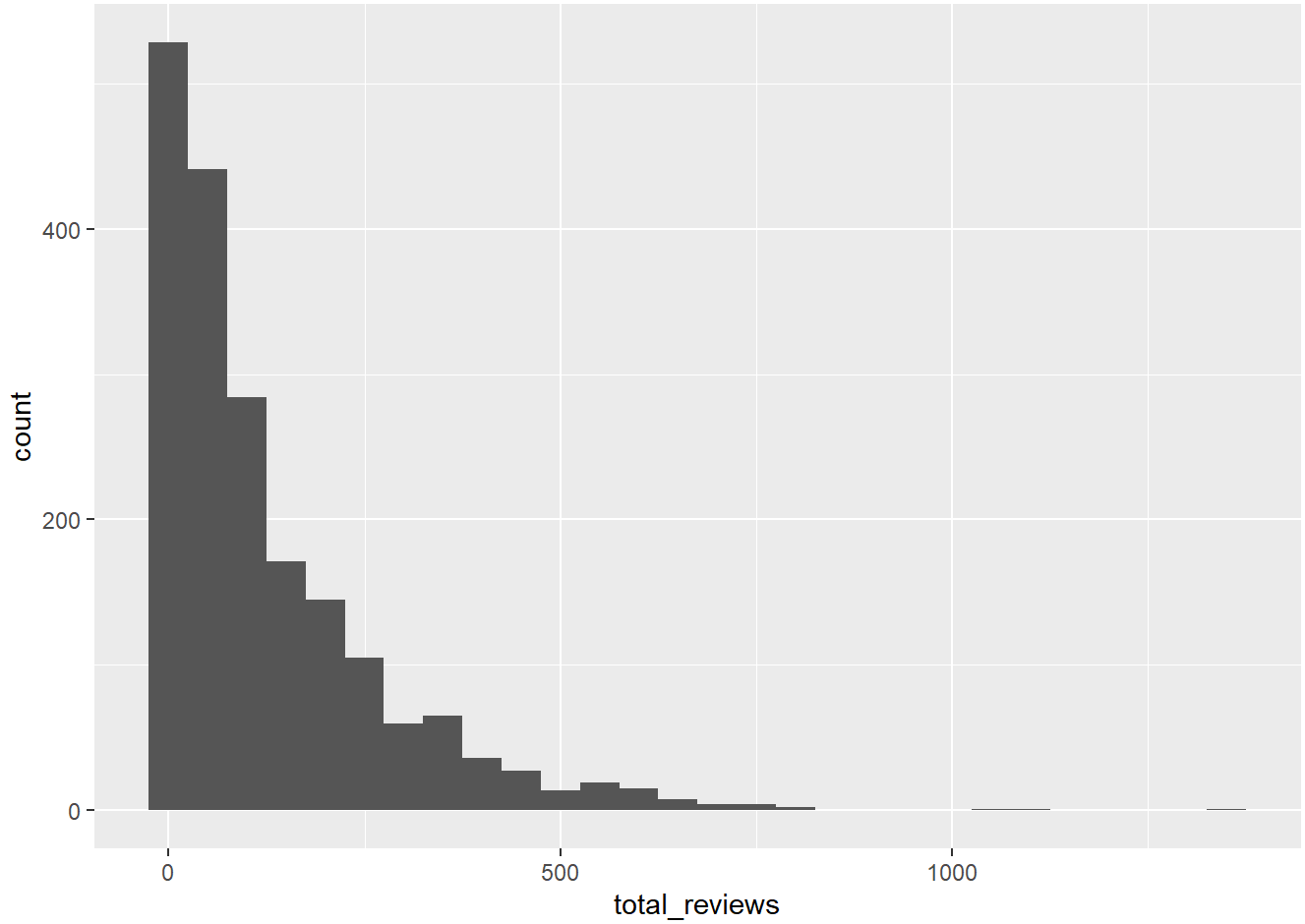

```
# same as price -> first lets visualize without the clear outliers
boxplot(df$min_nights, outline = F)
```



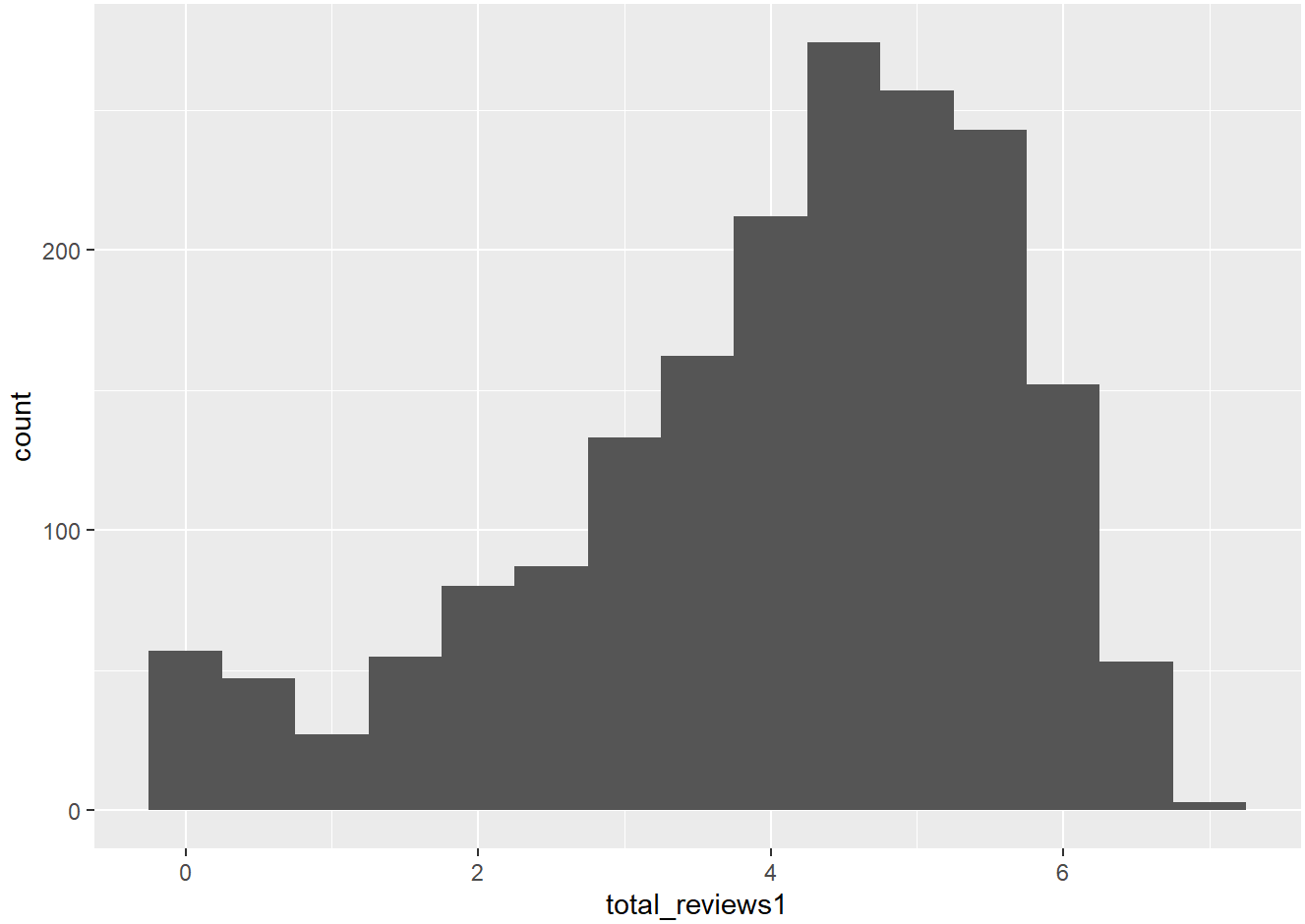

```
# 13) total reviews is an int data type
summary(df$total_reviews)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     1.0    22.5    75.0   126.6   184.0  1332.0
```

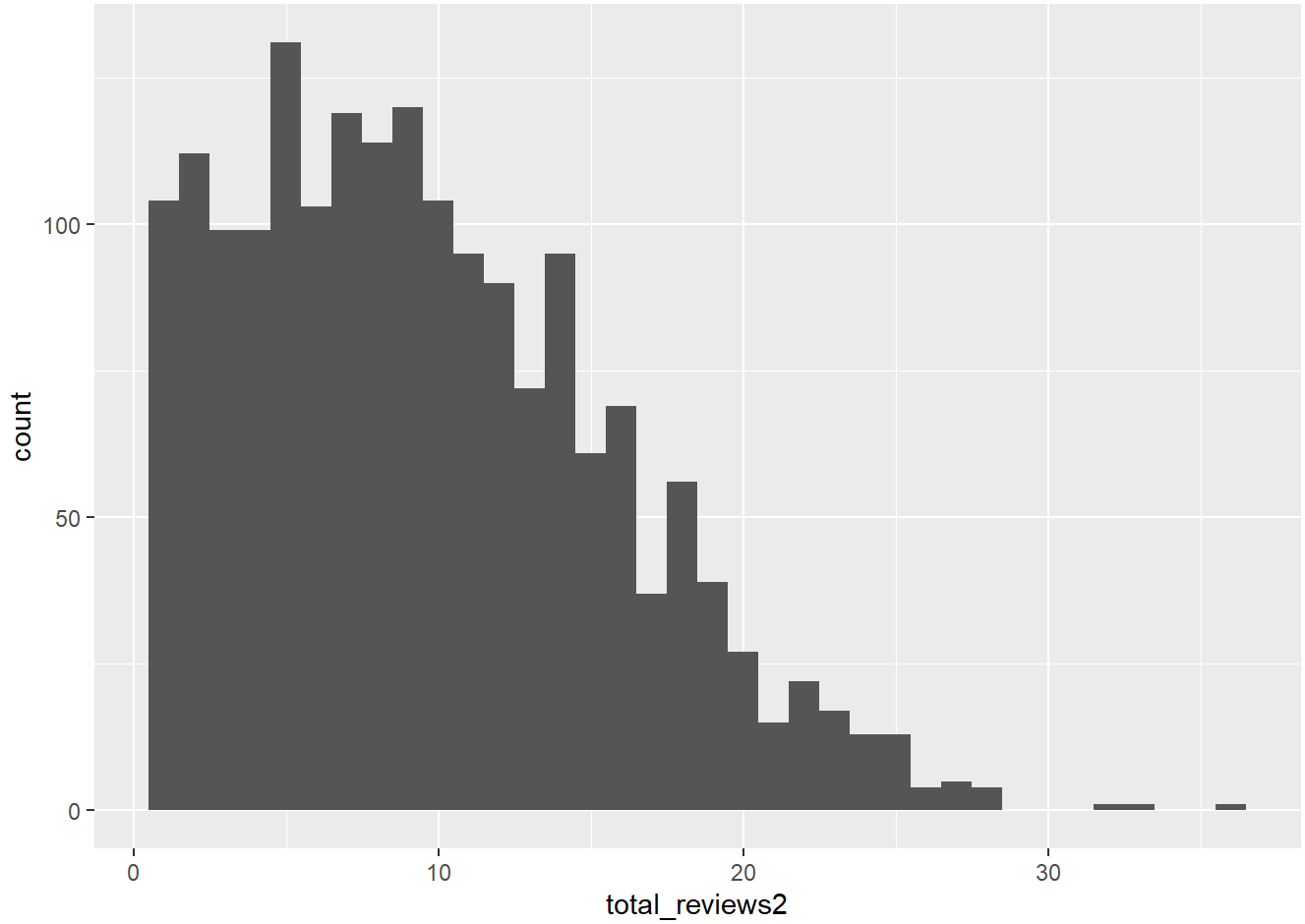
```
ggplot(df, aes(total_reviews)) + geom_histogram(binwidth=50)
```




```
df_tr_test = df_price %>%
  mutate(total_reviews1 = log(total_reviews),
         total_reviews2 = sqrt(total_reviews))


ggplot(df_tr_test, aes(total_reviews1)) + geom_histogram(binwidth = 0.5)
```



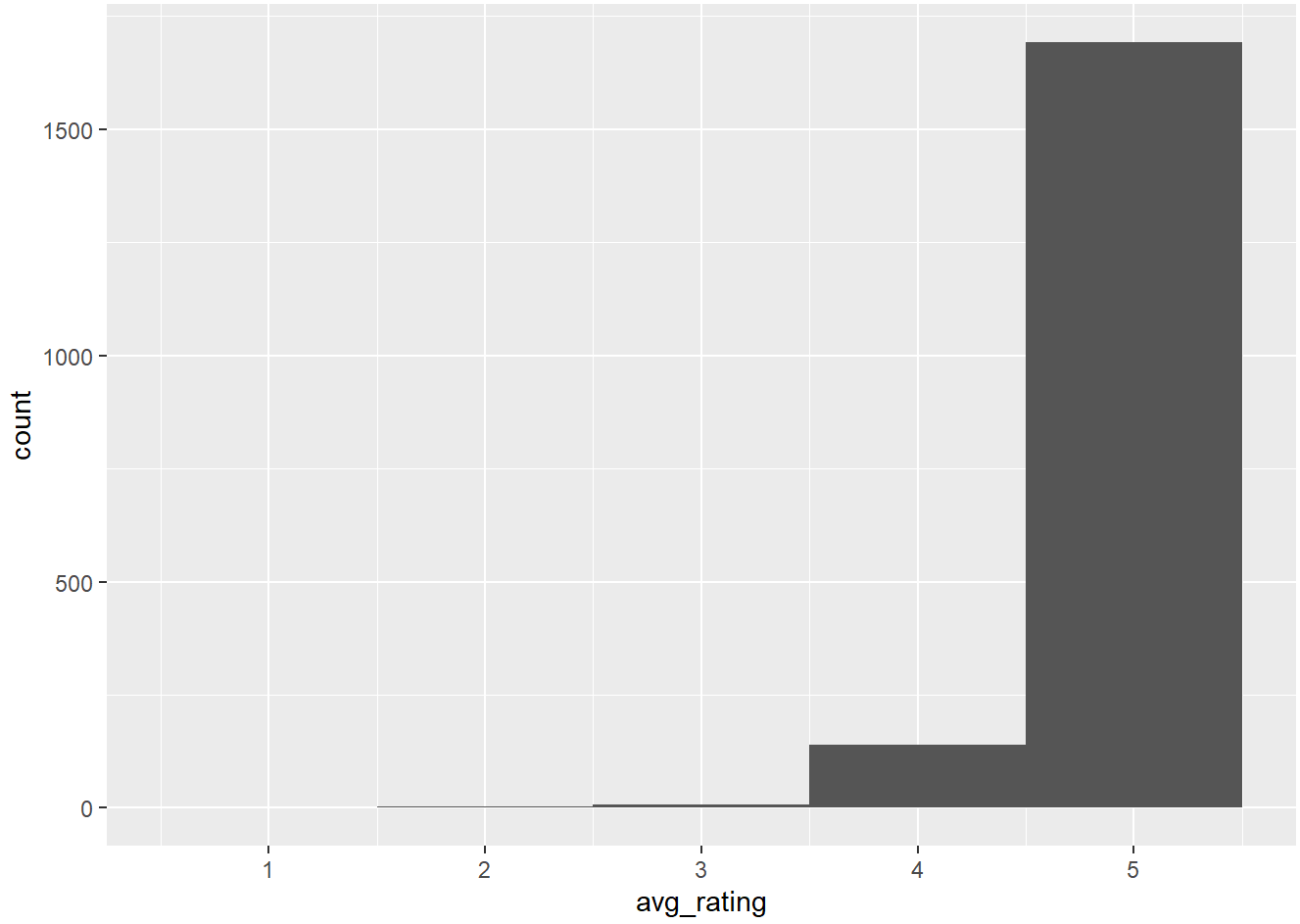

```
ggplot(df_tr_test, aes(total_reviews2)) + geom_histogram(binwidth = 1)
```



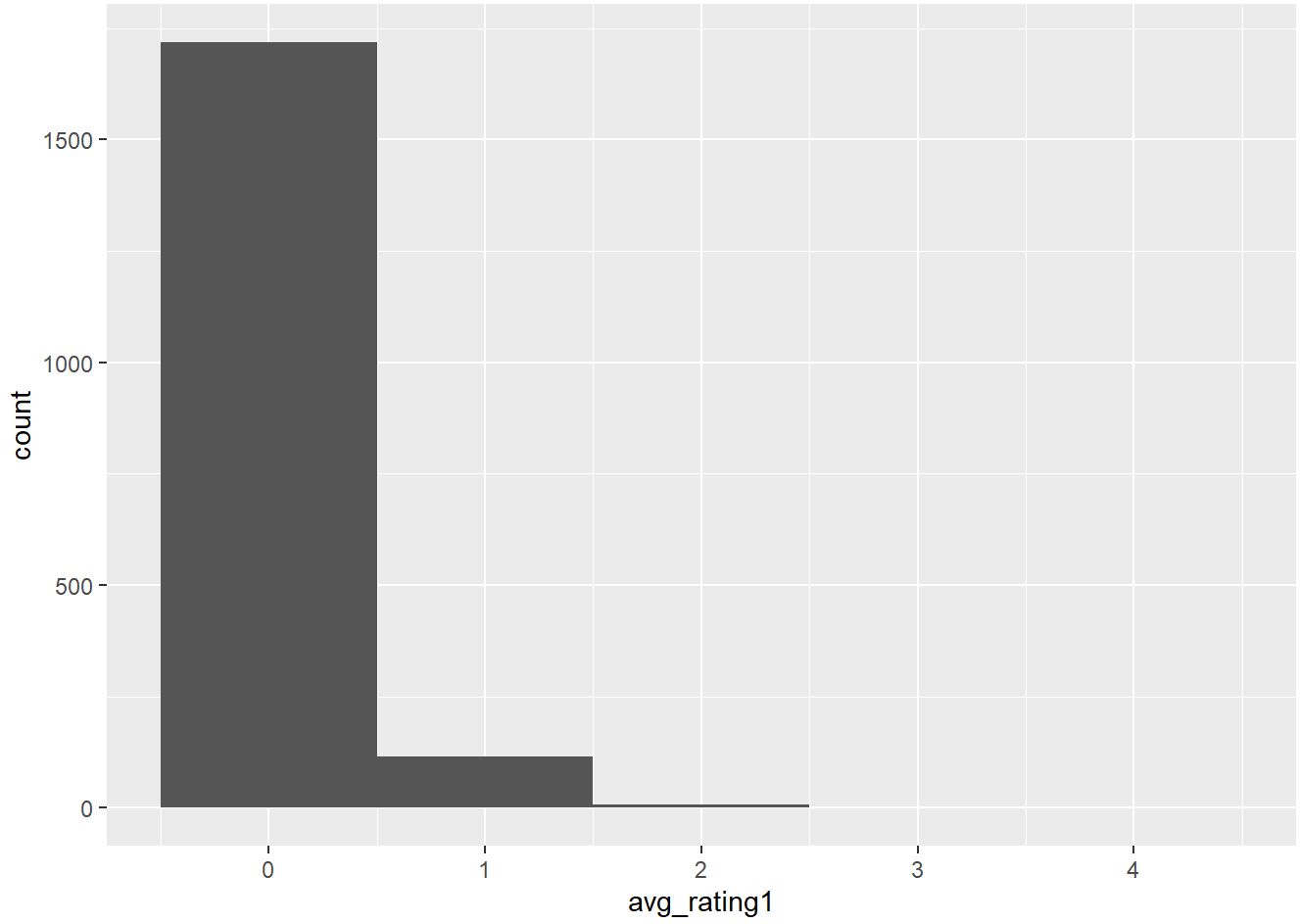

```
## log transform was better than sqrt transform

# 14) avg_rating is a double data type
summary(df$avg_rating)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.000   4.740   4.860   4.795   4.940   5.000
```

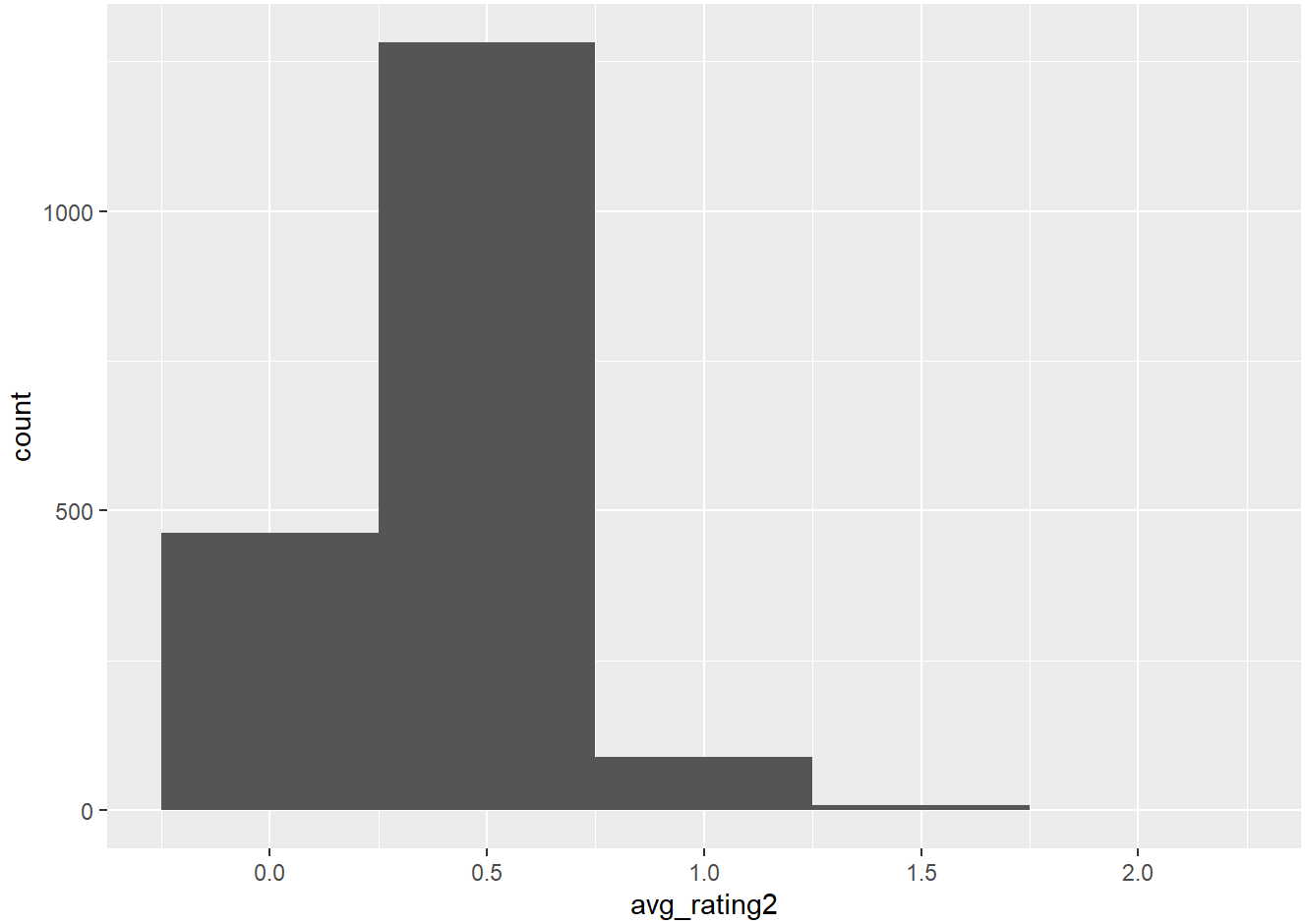
```
# left skewed data, with boundary conditions of 1 to 5
df_avg_rating = df_price %>%
  mutate( avg_rating1 = 5- avg_rating,
          avg_rating2 = sqrt(5- avg_rating),
          avg_rating3 = log(5- avg_rating),
          avg_rating4 = log((5- avg_rating)/ avg_rating))


ggplot(df_price, aes(avg_rating)) + geom_histogram(binwidth = 1)
```




```
ggplot(df_avg_rating, aes(avg_rating1)) + geom_histogram(binwidth = 1)
```




```
ggplot(df_avg_rating, aes(avg_rating2)) + geom_histogram(binwidth = 0.5)
```




```
## this worked well however 224 non-finite outside the scale range
# ggplot(df_avg_rating, aes(avg_rating3)) + geom_histogram(binwidth = 0.5)
# ggplot(df_avg_rating, aes(avg_rating4)) + geom_histogram(bindwith=.2)

## lets explore possible correlations between the numerical variables

correlation_matrix = cor(select_if(df, is.numeric))
diag(correlation_matrix) = NA
correlation_df = data.frame(correlation_matrix)
names(correlation_df[which.max(abs(correlation_df$price))])
```

```
## [1] "host_total_listings"
```

```
## before any filtering of data , host total listings is the most correlated with price

# now that the price outliers are filtered out
# let us retest the correlation matrix
correlation_matrix_clean = cor(select_if(df_price, is.numeric))
diag(correlation_matrix_clean) = NA
correlation_df_clean = data.frame(correlation_matrix_clean)

a=correlation_df_clean %>%
  arrange(desc(price)) %>%
  select(price)

correlation_matrix_clean2 = cor(select_if(df_a_p, is.numeric))
diag(correlation_matrix_clean2) = NA
correlation_df_clean2 = data.frame(correlation_matrix_clean2)

price_acc=correlation_df_clean2 %>%
  arrange(desc(price)) %>%
  select(price)

correlation_matrix_clean3 = cor(select_if(df_b_p, is.numeric))
diag(correlation_matrix_clean3) = NA
correlation_df_clean3 = data.frame(correlation_matrix_clean3)

price_br=correlation_df_clean3 %>%
  arrange(desc(price)) %>%
  select(price)

correlation_matrix_clean4 = cor(select_if(df_bed_p, is.numeric))
diag(correlation_matrix_clean4) = NA
correlation_df_clean4 = data.frame(correlation_matrix_clean4)

price_beds=correlation_df_clean4 %>%
  arrange(desc(price)) %>%
  select(price)

test = tibble(a , price_acc$price , price_br$price , price_beds$price)
test1 = data.frame(a , price_acc$price , price_br$price , price_beds$price)
test1
```

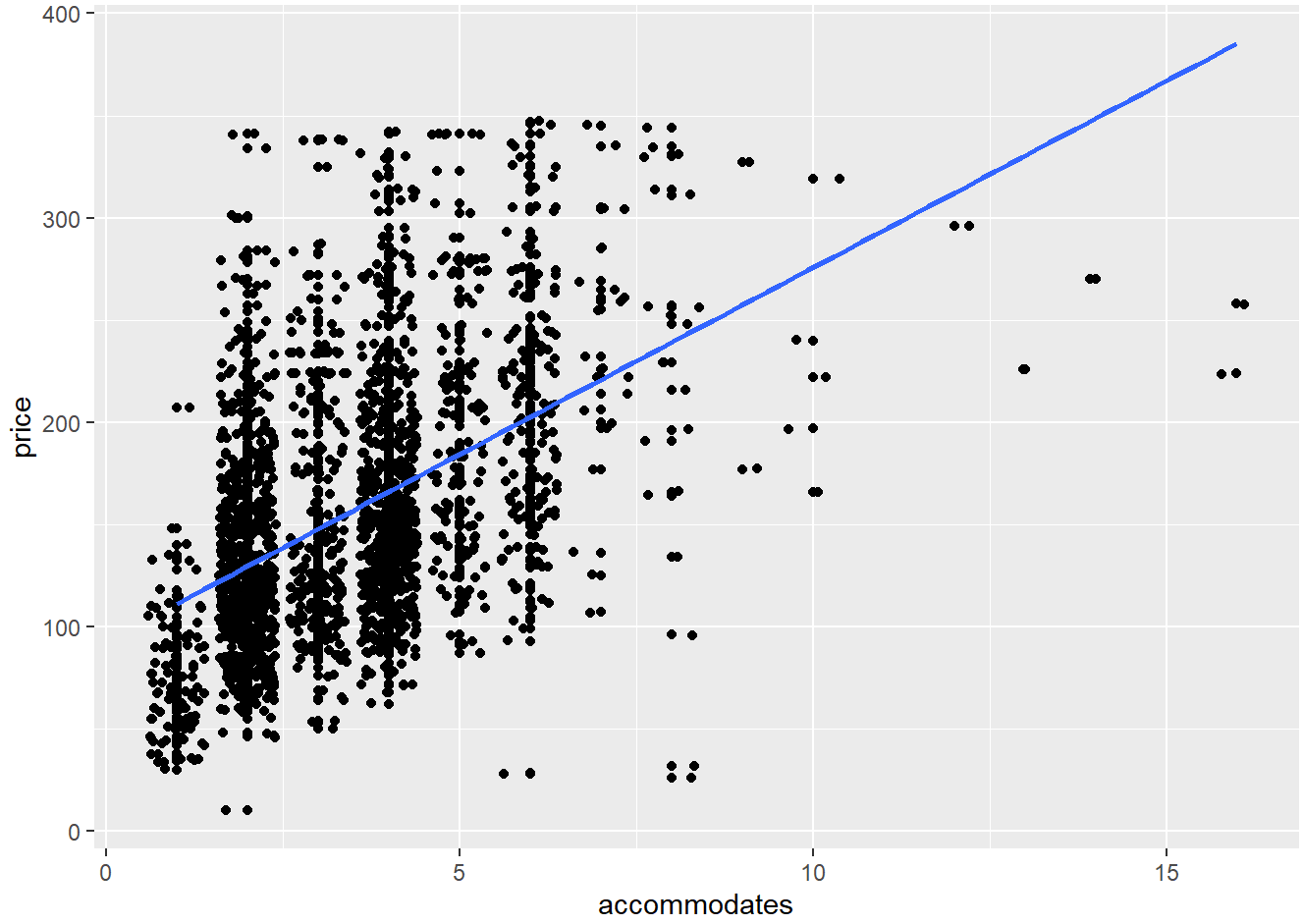
```
##                           price price_acc.price price_br.price
## accommodates          0.46533069      0.44933714     0.45391894
## bedrooms              0.44892186      0.40413748     0.42834523
## beds                  0.31551599      0.27103085     0.29084423
## host_total_listings   0.22599330      0.24145363     0.23263596
## host_acceptance_rate  0.16284419      0.15819767     0.16053715
## avg_rating           -0.01954975     -0.02751319    -0.01605669
## superhost2           -0.03688860     -0.04998552    -0.03449495
## total_reviews        -0.12580045     -0.12065275    -0.12234282
## min_nights           -0.13290305     -0.13778638    -0.13299262
## price                         NA              NA             NA
##                      price_beds.price
## accommodates               0.44014641
## bedrooms                   0.39223269
## beds                       0.34388009
## host_total_listings        0.24350299
## host_acceptance_rate       0.16368677
## avg_rating                -0.04212018
## superhost2                -0.06289038
## total_reviews             -0.13301302
## min_nights                -0.14199117
## price                              NA
```

```
test
```

```
## # A tibble: 10 × 4
##      price `price_acc$price` `price_br$price` `price_beds$price`
##      <dbl>             <dbl>            <dbl>              <dbl>
##  1  0.465             0.449            0.454              0.440
##  2  0.449             0.404            0.428              0.392
##  3  0.316             0.271            0.291              0.344
##  4  0.226             0.241            0.233              0.244
##  5  0.163             0.158            0.161              0.164
##  6 -0.0195           -0.0275          -0.0161            -0.0421
##  7 -0.0369           -0.0500          -0.0345            -0.0629
##  8 -0.126            -0.121           -0.122             -0.133
##  9 -0.133            -0.138           -0.133             -0.142
## 10 NA                NA               NA                 NA
```

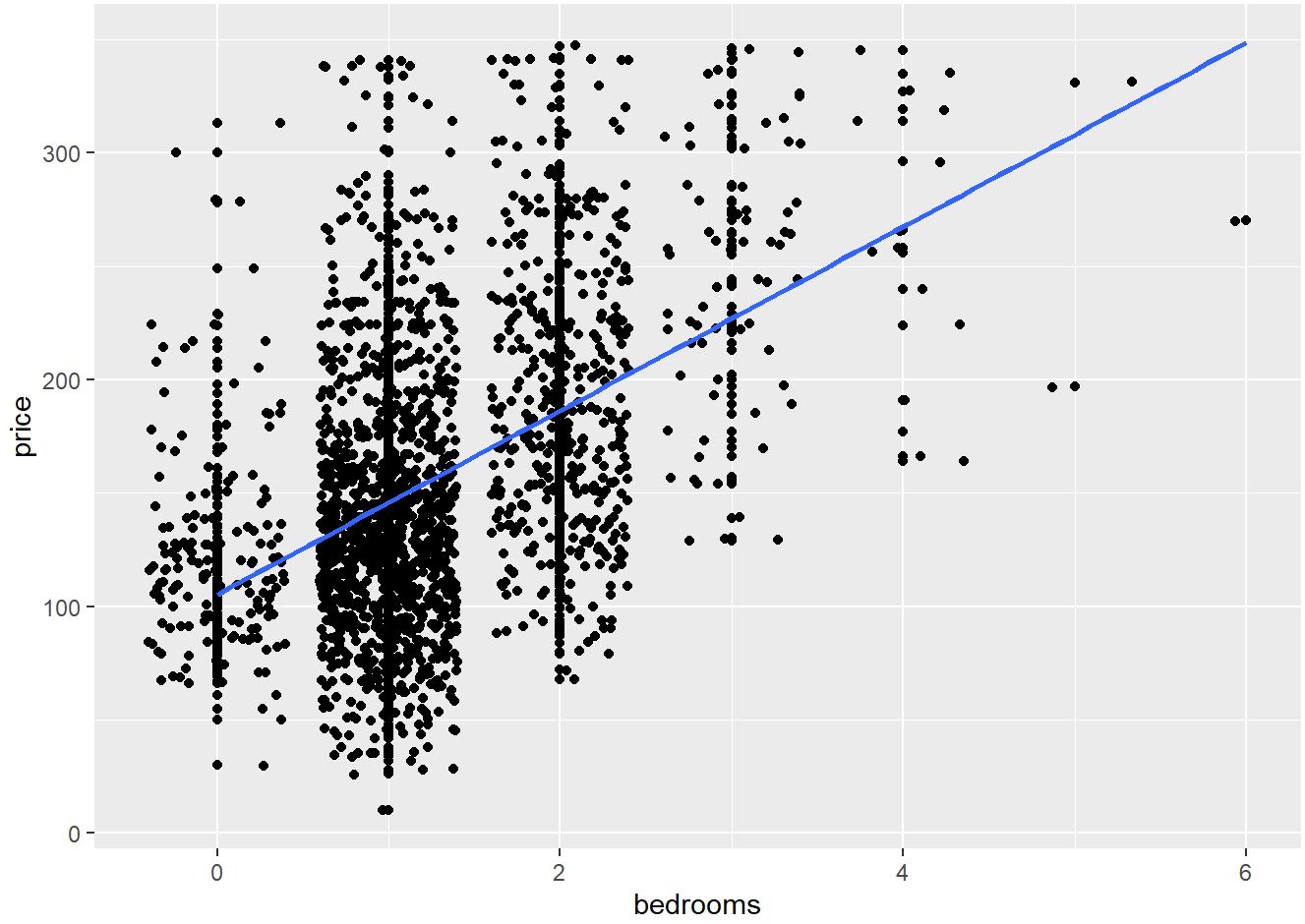
```
## based on the correlation matrix, only filtering out the price outliers was the most effective

## best three to study are accommodates, bedrooms , beds (stays true for all)

## lets start with visualizing price v accommodates, bedroom, bed
## price = dependent variable , the rest are independent variable

ggplot( df_price , aes(x = accommodates, y = price)) + geom_point() + geom_jitter() + geom_smoot
h(method = "lm" , se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



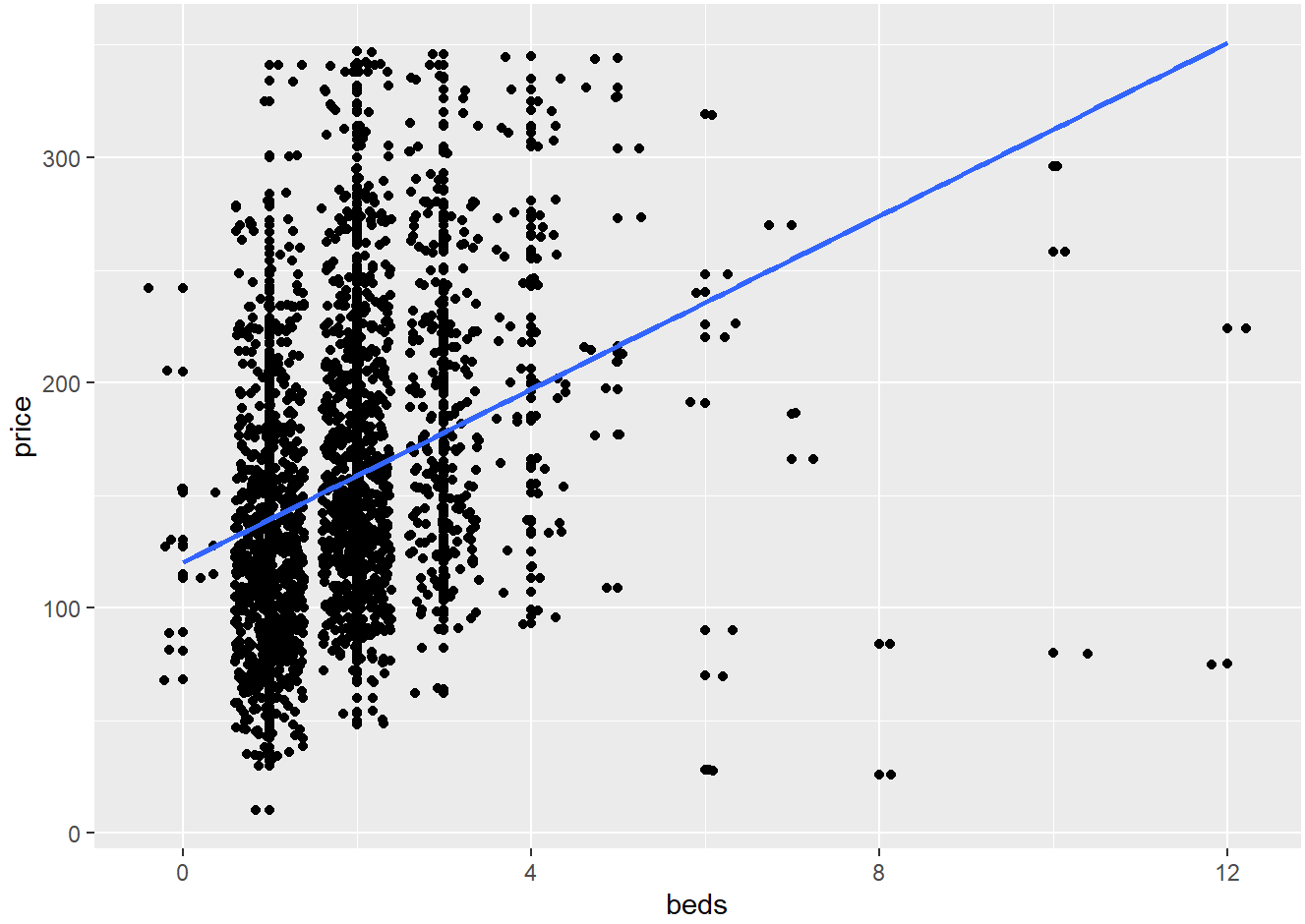

```
ggplot( df_price , aes(x = bedrooms, y = price)) + geom_point() + geom_jitter() + geom_smooth(me
thod = "lm" , se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



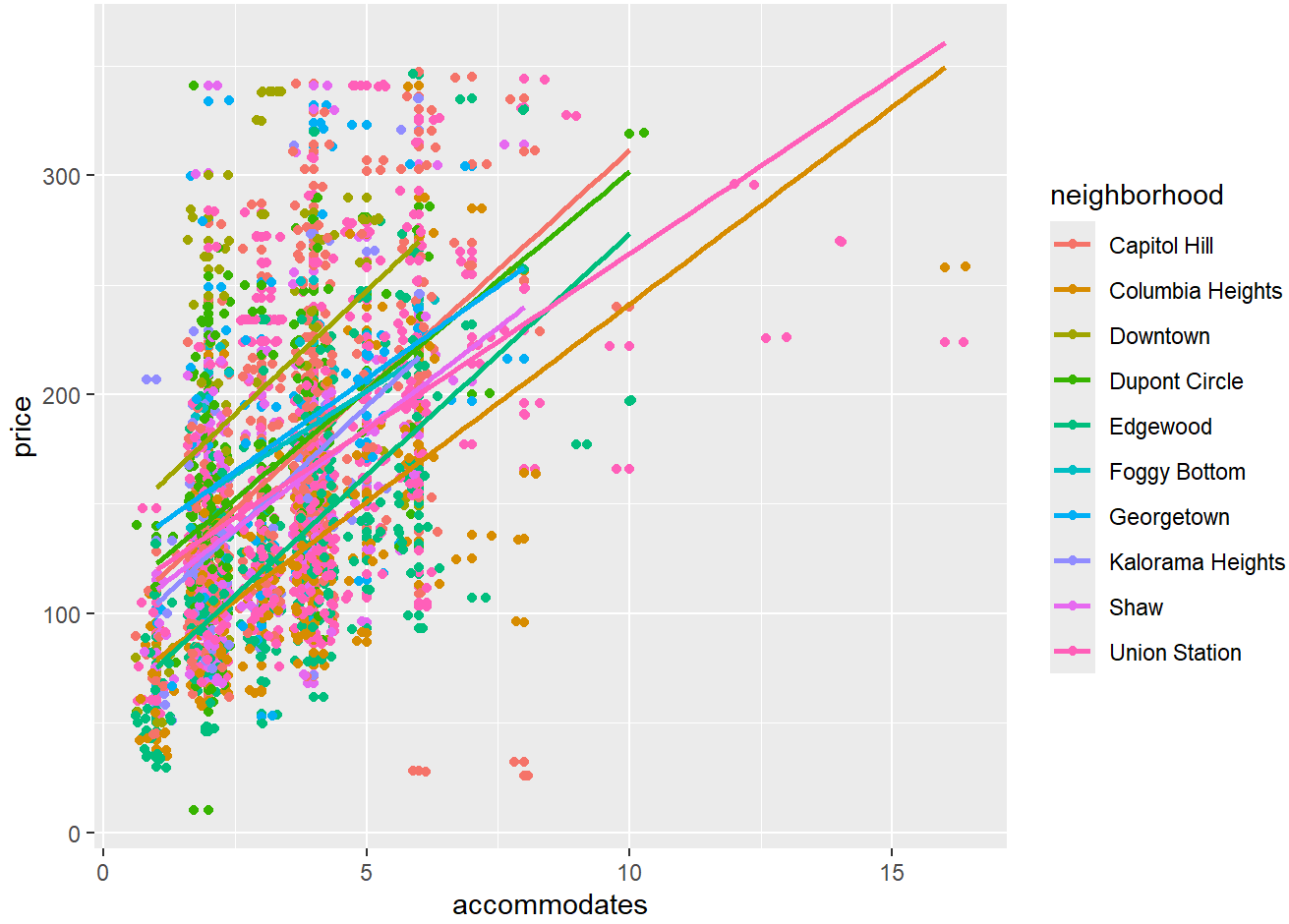

```
ggplot( df_price , aes(x = beds, y = price)) + geom_point() + geom_jitter() + geom_smooth(method = "lm" , se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



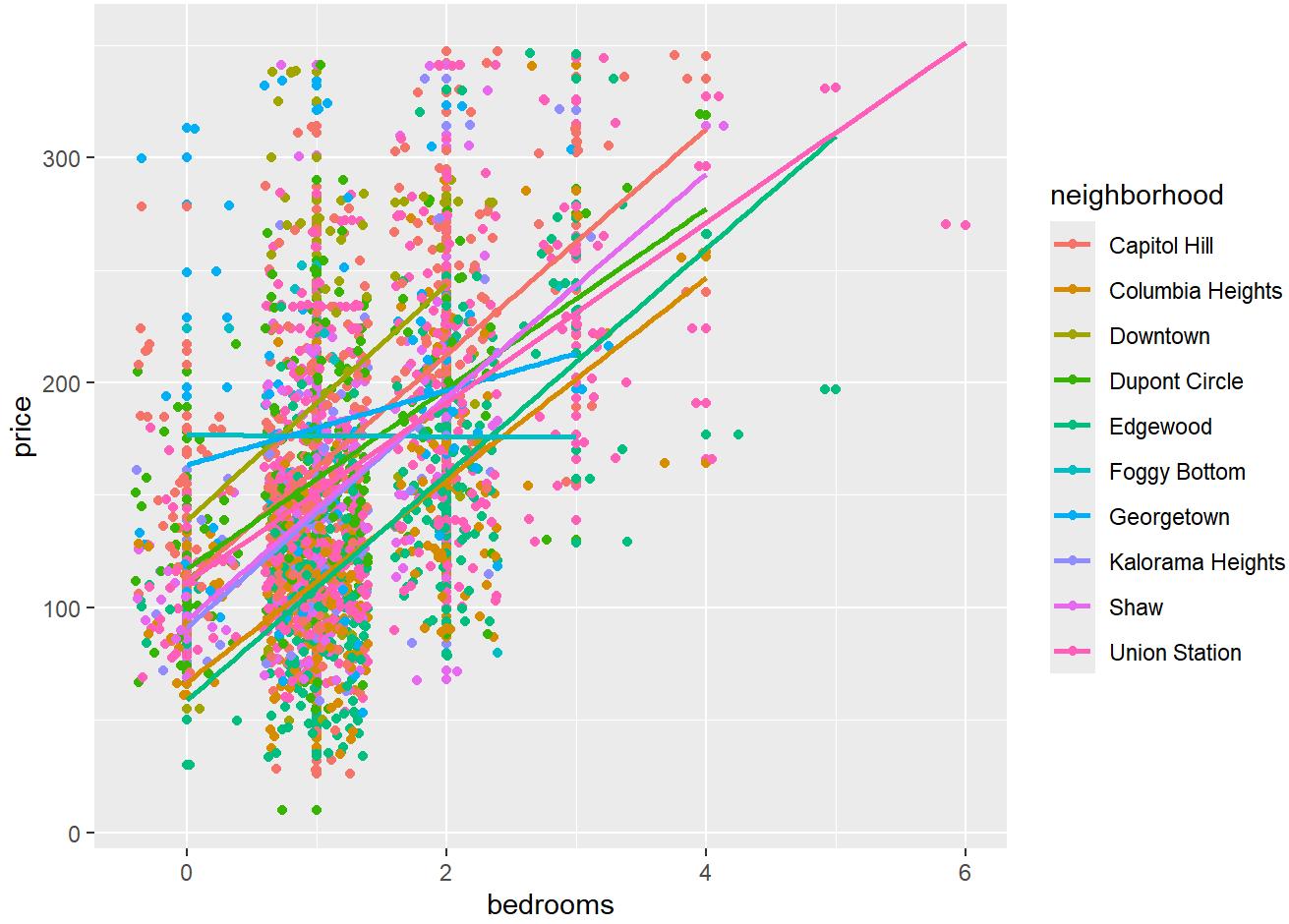

```
ggplot( df_price , aes(x = accommodates, y = price , color = neighborhood)) + geom_point() + geo
m_jitter() + geom_smooth(method = "lm" , se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



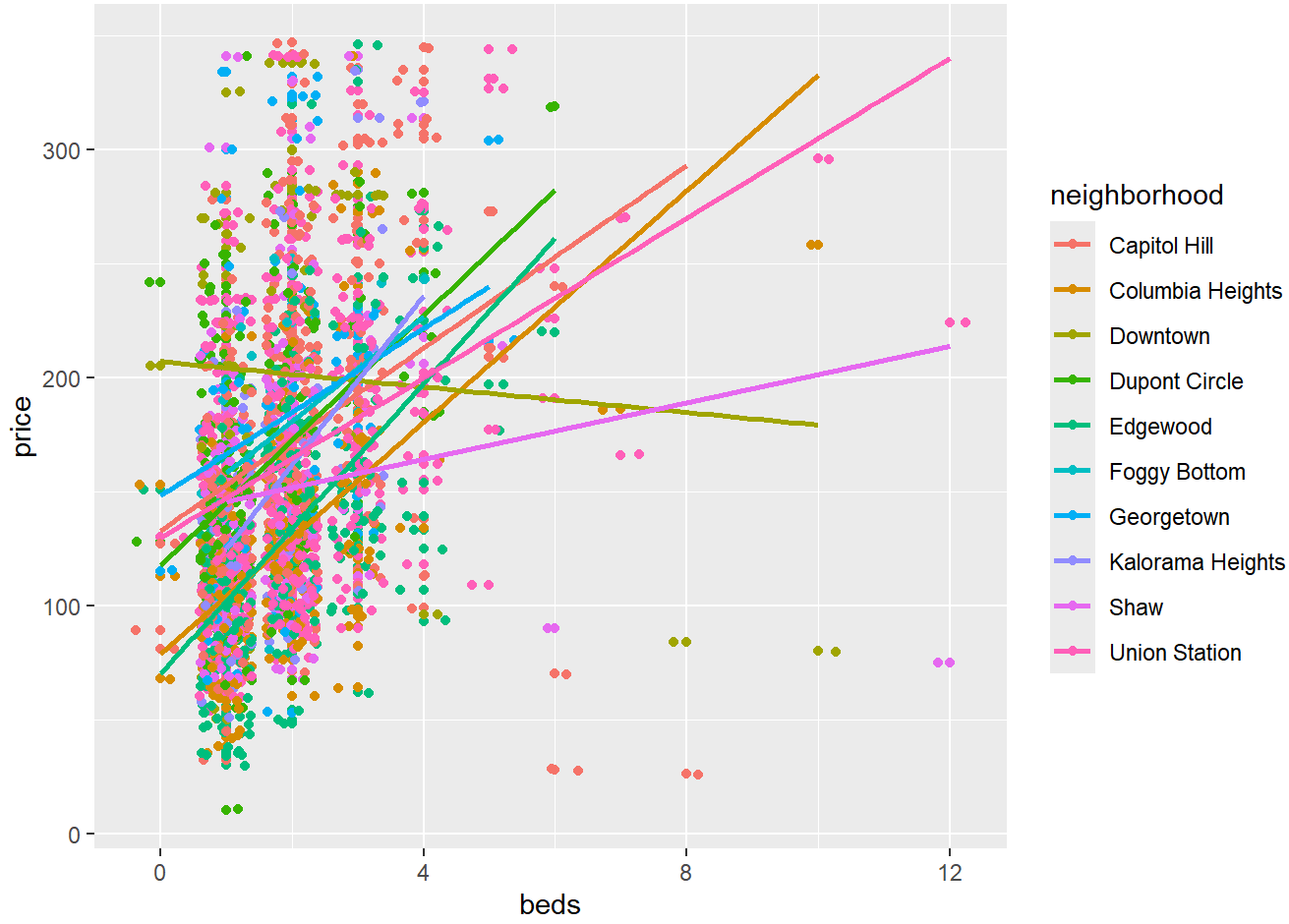

```
ggplot( df_price , aes(x = bedrooms, y = price , color = neighborhood)) + geom_point() + geom_ji
tter() + geom_smooth(method = "lm" , se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



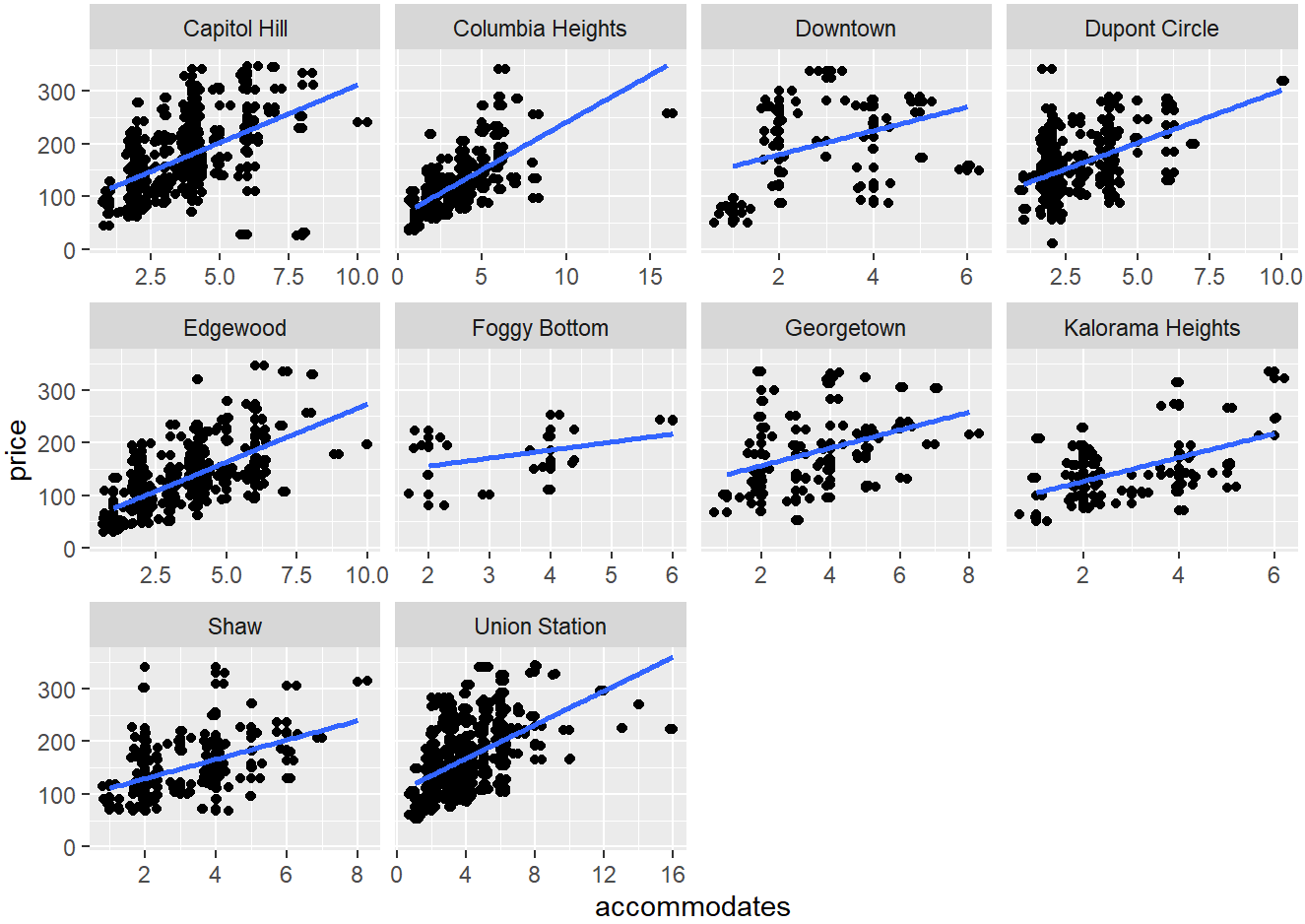

```
ggplot( df_price , aes(x = beds, y = price , color = neighborhood)) + geom_point() + geom_jitter
() + geom_smooth(method = "lm" , se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



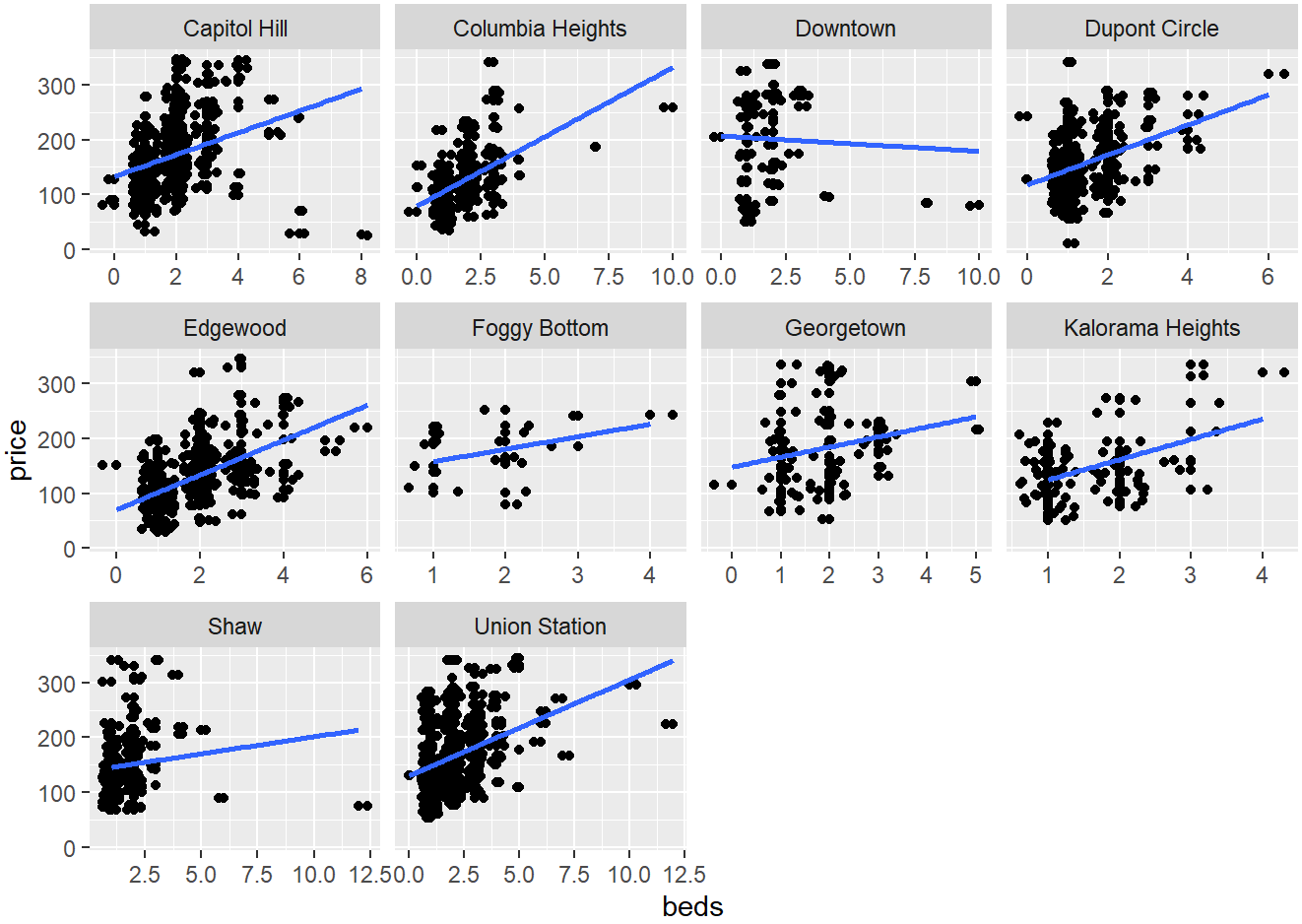

```
ggplot(df_price , aes(x = accommodates, y = price)) +
  geom_point() + geom_jitter() + geom_smooth(method = "lm" , se = FALSE) +
  facet_wrap(~neighborhood, scales = "free_x")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



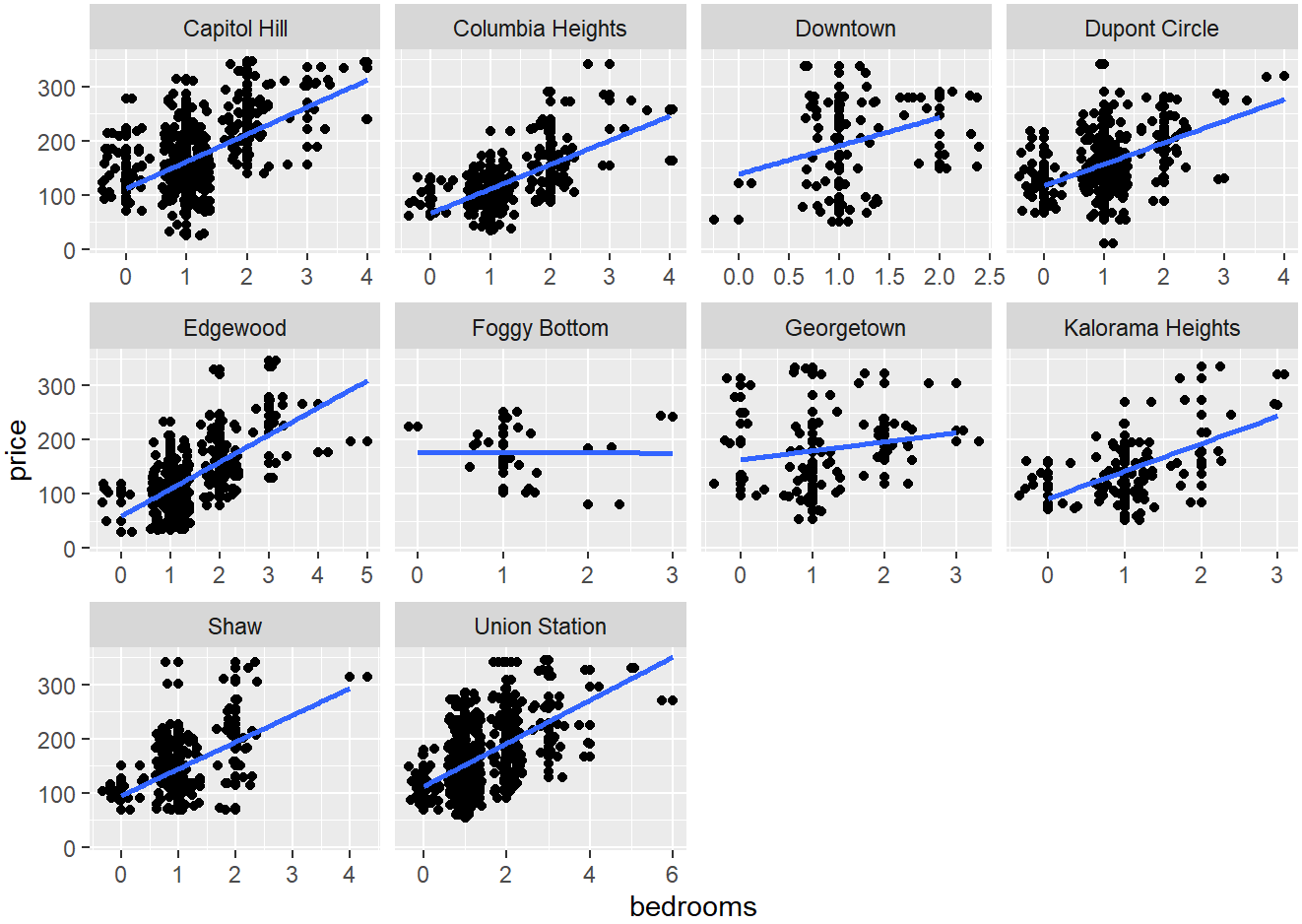

```
ggplot(df_price , aes(x = beds, y = price)) +
  geom_point() + geom_jitter() + geom_smooth(method = "lm" , se = FALSE) +
  facet_wrap(~neighborhood, scales = "free_x")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



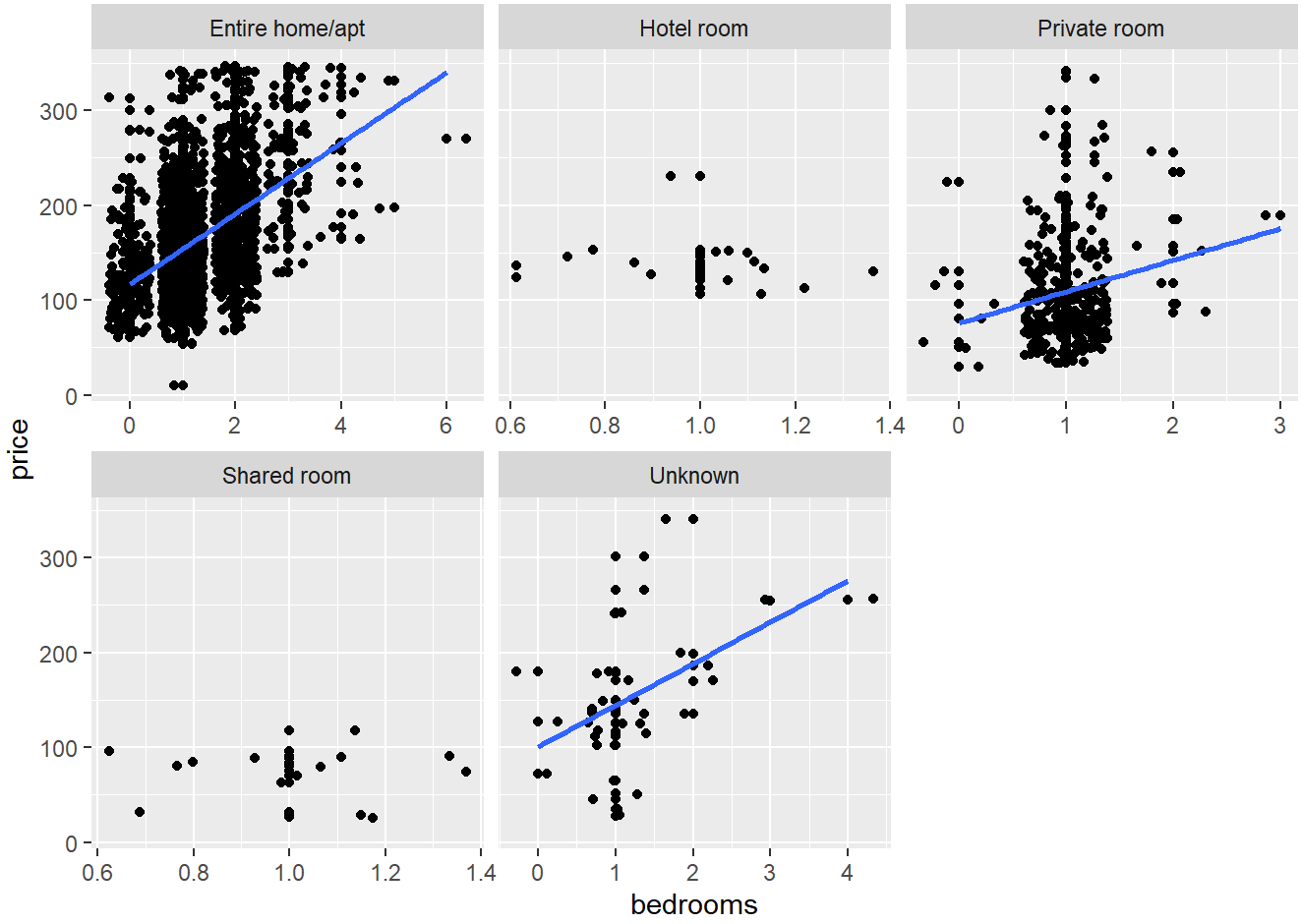

```
ggplot(df_price , aes(x = bedrooms, y = price)) +
  geom_point() + geom_jitter() + geom_smooth(method = "lm" , se = FALSE) +
  facet_wrap(~neighborhood, scales = "free_x")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```




```
ggplot(df_price , aes(x = bedrooms, y = price)) +
  geom_point() + geom_jitter() + geom_smooth(method = "lm" , se = FALSE) +
  facet_wrap(~room_type, scales = "free_x")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```




```
## Summary statistics, Group By Room_type , Bathrooms, Neighborhood

df_price %>%
  group_by(neighborhood) %>%
  summarize(avg_price = mean(price) , sd_price = sd(price) , median_price = quantile(price, 0.5)
, IQR_price = IQR(price))
```

```
## # A tibble: 10 × 5
##    neighborhood     avg_price sd_price median_price IQR_price
##    <fct>                <dbl>    <dbl>        <dbl>     <dbl>
##  1 Capitol Hill          171.     66.0         158.      89
##  2 Columbia Heights      125.     50.6         119       58
##  3 Downtown              202.     87.1         213      158
##  4 Dupont Circle         158.     54.6         150       69.8
##  5 Edgewood              128.     56.3         119       62
##  6 Foggy Bottom          176.     51.1         188.      67.2
##  7 Georgetown            183.     72.0         178       98.5
##  8 Kalorama Heights      147.     59.1         138.      66
##  9 Shaw                  150.     59.9         136.      77.5
## 10 Union Station         165.     63.0         152       95
```

```
df_price %>%
  group_by(room_type) %>%
  summarize(avg_price = mean(price) , sd_price = sd(price) , median_price = quantile(price, 0.5)
, IQR_price = IQR(price))
```

```
## # A tibble: 5 × 5
##   room_type       avg_price sd_price median_price IQR_price
##   <fct>               <dbl>    <dbl>        <dbl>     <dbl>
## 1 Entire home/apt     164.      61.8        151        85
## 2 Hotel room          141.      27.9        138        23.8
## 3 Private room        109.      56.1         97.5      60.2
## 4 Shared room          73.1     27.3         80.5      25
## 5 Unknown             153.      75.2        137        70
```

```
df_price %>%
  filter(room_type == "Entire home/apt") %>%
  group_by(neighborhood) %>%
  summarize(avg_price = mean(price) , sd_price = sd(price) , median_price = quantile(price, 0.5)
, IQR_price = IQR(price))
```

```
## # A tibble: 10 × 5
##    neighborhood     avg_price sd_price median_price IQR_price
##    <fct>                <dbl>    <dbl>        <dbl>     <dbl>
##  1 Capitol Hill          181.     61.9         168       81
##  2 Columbia Heights      134.     48.4         125       50
##  3 Downtown              226.     74.9         241      126
##  4 Dupont Circle         164.     57.6         160       82.5
##  5 Edgewood              144.     52.5         134.      59.2
##  6 Foggy Bottom          164.     48.4         166       64.5
##  7 Georgetown            189.     69.1         189       95.8
##  8 Kalorama Heights      152.     59.7         139       50.5
##  9 Shaw                  160.     57.4         150       74.8
## 10 Union Station         169.     61.6         156      102
```

```
df_price %>%
  filter(room_type == "Entire home/apt" & neighborhood == "Union Station") %>%
  group_by(bathrooms) %>%
  summarize(avg_price = mean(price) , sd_price = sd(price) , median_price = quantile(price, 0.5)
, IQR_price = IQR(price))
```

```
## # A tibble: 6 × 5
##   bathrooms avg_price sd_price median_price IQR_price
##   <fct>         <dbl>    <dbl>        <dbl>     <dbl>
## 1 1 bath         155.     52.1         142       71.5
## 2 1.5 baths      206.     61.3         185       86
## 3 2 baths        230.     67.3         226.      95.8
## 4 2.5 baths      229.     44.4         223       67.5
## 5 3 baths        274.     75.9         325      105
## 6 4 baths        247      32.5         247       23
```

```
## Question 2 ##


## lets also get some numbers:
table(df_price$room_type)
```

```
##
## Entire home/apt      Hotel room    Private room     Shared room         Unknown
##            1509              16             268              14              35
```

```
table(df_price$neighborhood)
```

```
##
##     Capitol Hill Columbia Heights         Downtown    Dupont Circle
##              310              213               69              216
##         Edgewood     Foggy Bottom       Georgetown Kalorama Heights
##              275               20               74               80
##             Shaw    Union Station
##              148              437
```

```
table(df_price$neighborhood, df_price$room_type)
```

```
##
##                   Entire home/apt Hotel room Private room Shared room Unknown
##   Capitol Hill                271          1           26           6       6
##   Columbia Heights            172          0           34           0       7
##   Downtown                     45          1           20           3       0
##   Dupont Circle               171         14           27           0       4
##   Edgewood                    212          0           60           0       3
##   Foggy Bottom                 15          0            4           0       1
##   Georgetown                   64          0            8           0       2
##   Kalorama Heights             59          0           17           1       3
##   Shaw                        102          0           39           4       3
##   Union Station               398          0           33           0       6
```

```
df %>%
  group_by(neighborhood, room_type) %>%
  summarize(avg_price = mean(price, na.rm = TRUE), standard_deviation = sd(price, na.rm = TRUE)
,
            Q_price25=quantile(price, 0.25), Q_price50=quantile(price, 0.5), Q_price75=quantile
(price, 0.75)
            , IQR = (quantile(price, 0.75) - quantile(price, 0.25))) %>%
  arrange(desc(avg_price))
```

```
## `summarise()` has grouped output by 'neighborhood'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 38 × 8
## # Groups:   neighborhood [10]
##    neighborhood     room_type   avg_price standard_deviation Q_price25 Q_price50
##    <fct>            <fct>           <dbl>              <dbl>     <dbl>     <dbl>
##  1 Kalorama Heights Hotel room     50000                0      50000     50000
##  2 Kalorama Heights Unknown        20052             27339.        72       137
##  3 Downtown         Unknown          579                NA        579       579
##  4 Downtown         Entire hom…      283.             109.       194.      280
##  5 Capitol Hill     Unknown          258              269.       109.      138.
##  6 Foggy Bottom     Unknown          241                NA        241       241
##  7 Georgetown       Entire hom…      241.             177.       136.      198.
##  8 Downtown         Hotel room       231                NA        231       231
##  9 Union Station    Unknown          216.              71.2      172.      190.
## 10 Foggy Bottom     Private ro…      206.              48.1      192.      217
## # ℹ 28 more rows
## # ℹ 2 more variables: Q_price75 <dbl>, IQR <dbl>
```

```
df %>%
  group_by(neighborhood, room_type) %>%
  summarize(avg_price = mean(price, na.rm = TRUE), standard_deviation = sd(price, na.rm = TRUE)
,
            Q_price25=quantile(price, 0.25), Q_price50=quantile(price, 0.5), Q_price75=quantile
(price, 0.75)
            , IQR = (quantile(price, 0.75) - quantile(price, 0.25))) %>%
  arrange((avg_price))
```

```
## `summarise()` has grouped output by 'neighborhood'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 38 × 8
## # Groups:   neighborhood [10]
##    neighborhood     room_type   avg_price standard_deviation Q_price25 Q_price50
##    <fct>            <fct>           <dbl>              <dbl>     <dbl>     <dbl>
##  1 Capitol Hill     Shared room      54.3              28.8       29        51
##  2 Kalorama Heights Shared room      63                NA         63        63
##  3 Edgewood         Private ro…      73.0              29.8       49.5      71
##  4 Columbia Heights Private ro…      79.6              25.4       64.2      78
##  5 Downtown         Shared room      86.7               8.33      82        84
##  6 Shaw             Shared room      93.5              17.9       86.2      90.5
##  7 Capitol Hill     Private ro…     102.               35.2       79        97.5
##  8 Union Station    Private ro…     104.               40.8       71        97
##  9 Capitol Hill     Hotel room      113                NA        113       113
## 10 Shaw             Private ro…     125.               56.5       92.5     110
## # ℹ 28 more rows
## # ℹ 2 more variables: Q_price75 <dbl>, IQR <dbl>
```

```
df %>%
  group_by(neighborhood, room_type) %>%
  summarize(avg_price = mean(price, na.rm = TRUE), standard_deviation = sd(price, na.rm = TRUE)
,
            Q_price25=quantile(price, 0.25), Q_price50=quantile(price, 0.5), Q_price75=quantile
(price, 0.75)
            , IQR = (quantile(price, 0.75) - quantile(price, 0.25))) %>%
  arrange(desc(standard_deviation))
```

```
## `summarise()` has grouped output by 'neighborhood'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 38 × 8
## # Groups:   neighborhood [10]
##    neighborhood     room_type   avg_price standard_deviation Q_price25 Q_price50
##    <fct>            <fct>           <dbl>              <dbl>     <dbl>     <dbl>
##  1 Kalorama Heights Unknown        20052             27339.       72        137
##  2 Capitol Hill     Unknown          258                269.      109.       138.
##  3 Georgetown       Entire hom…      241.               177.      136.       198.
##  4 Shaw             Entire hom…      176.               112.      121        150
##  5 Downtown         Entire hom…      283.               109.      194.       280
##  6 Kalorama Heights Entire hom…      171.               108.      115.       140.
##  7 Downtown         Private ro…      163.                95.1      71.8      172.
##  8 Capitol Hill     Entire hom…      201.                94.9     139        176
##  9 Shaw             Unknown          199.                90.9     148.       171
## 10 Georgetown       Private ro…      141                 88.4      87        100.
## # ℹ 28 more rows
## # ℹ 2 more variables: Q_price75 <dbl>, IQR <dbl>
```

```
df %>%
  group_by(neighborhood, room_type) %>%
  summarize(avg_price = mean(price, na.rm = TRUE), standard_deviation = sd(price, na.rm = TRUE)
,
            Q_price25=quantile(price, 0.25), Q_price50=quantile(price, 0.5), Q_price75=quantile
(price, 0.75)
            , IQR = (quantile(price, 0.75) - quantile(price, 0.25)))%>%
  arrange((standard_deviation))
```

```
## `summarise()` has grouped output by 'neighborhood'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 38 × 8
## # Groups:   neighborhood [10]
##    neighborhood     room_type   avg_price standard_deviation Q_price25 Q_price50
##    <fct>            <fct>           <dbl>              <dbl>     <dbl>     <dbl>
##  1 Kalorama Heights Hotel room    50000                 0      50000     50000
##  2 Downtown         Shared room      86.7                8.33      82        84
##  3 Edgewood         Unknown         131.                12.1      126       135
##  4 Dupont Circle    Hotel room      136.                13.8      128.      138
##  5 Shaw             Shared room      93.5               17.9       86.2      90.5
##  6 Columbia Heights Private ro…      79.6               25.4       64.2      78
##  7 Capitol Hill     Shared room      54.3               28.8       29        51
##  8 Edgewood         Private ro…      73.0               29.8       49.5      71
##  9 Capitol Hill     Private ro…     102.                35.2       79        97.5
## 10 Union Station    Private ro…     104.                40.8       71        97
## # ℹ 28 more rows
## # ℹ 2 more variables: Q_price75 <dbl>, IQR <dbl>
```

```
df %>%
  group_by(neighborhood, room_type) %>%
  summarize(avg_price = mean(price, na.rm = TRUE), standard_deviation = sd(price, na.rm = TRUE)
,
            Q_price25=quantile(price, 0.25), Q_price50=quantile(price, 0.5), Q_price75=quantile
(price, 0.75)
            , IQR = (quantile(price, 0.75) - quantile(price, 0.25))) %>%
  arrange(desc(IQR))
```

```
## `summarise()` has grouped output by 'neighborhood'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 38 × 8
## # Groups:   neighborhood [10]
##    neighborhood     room_type   avg_price standard_deviation Q_price25 Q_price50
##    <fct>            <fct>           <dbl>              <dbl>     <dbl>     <dbl>
##  1 Kalorama Heights Unknown         20052             27339.       72        137
##  2 Capitol Hill     Unknown           258               269.      109.       138.
##  3 Downtown         Entire hom…       283.              109.      194.       280
##  4 Downtown         Private ro…       163.               95.1      71.8      172.
##  5 Georgetown       Entire hom…       241.              177.      136.       198.
##  6 Union Station    Entire hom…       179.               79.9     121        159
##  7 Kalorama Heights Private ro…       142.               53.2     100        144
##  8 Capitol Hill     Entire hom…       201.               94.9     139        176
##  9 Georgetown       Private ro…       141                88.4      87        100.
## 10 Shaw             Unknown           199.               90.9     148.       171
## # ℹ 28 more rows
## # ℹ 2 more variables: Q_price75 <dbl>, IQR <dbl>
```

```
df %>%
  group_by(neighborhood, room_type) %>%
  summarize(avg_price = mean(price, na.rm = TRUE), standard_deviation = sd(price, na.rm = TRUE)
,
            Q_price25=quantile(price, 0.25), Q_price50=quantile(price, 0.5), Q_price75=quantile
(price, 0.75)
            , IQR = (quantile(price, 0.75) - quantile(price, 0.25))) %>%
  arrange((IQR))
```

```
## `summarise()` has grouped output by 'neighborhood'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 38 × 8
## # Groups:   neighborhood [10]
##    neighborhood     room_type   avg_price standard_deviation Q_price25 Q_price50
##    <fct>            <fct>           <dbl>              <dbl>     <dbl>     <dbl>
##  1 Capitol Hill     Hotel room      113                NA        113       113
##  2 Downtown         Hotel room      231                NA        231       231
##  3 Downtown         Unknown         579                NA        579       579
##  4 Foggy Bottom     Unknown         241                NA        241       241
##  5 Kalorama Heights Hotel room    50000                 0      50000     50000
##  6 Kalorama Heights Shared room      63                NA         63        63
##  7 Downtown         Shared room      86.7               8.33      82        84
##  8 Edgewood         Unknown         131.               12.1      126       135
##  9 Shaw             Shared room      93.5              17.9       86.2      90.5
## 10 Dupont Circle    Hotel room      136.               13.8      128.      138
## # ℹ 28 more rows
## # ℹ 2 more variables: Q_price75 <dbl>, IQR <dbl>
```

```
df_price %>%
  group_by(neighborhood, room_type) %>%
  summarize(avg_price = mean(price, na.rm = TRUE), standard_deviation = sd(price, na.rm = TRUE)
,
            Q_price25=quantile(price, 0.25), Q_price50=quantile(price, 0.5), Q_price75=quantile
(price, 0.75)
            , IQR = (quantile(price, 0.75) - quantile(price, 0.25))) %>%
  arrange(desc(avg_price))
```

```
## `summarise()` has grouped output by 'neighborhood'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 36 × 8
## # Groups:   neighborhood [10]
##    neighborhood  room_type      avg_price standard_deviation Q_price25 Q_price50
##    <fct>         <fct>              <dbl>              <dbl>     <dbl>     <dbl>
##  1 Foggy Bottom  Unknown              241                NA        241       241
##  2 Downtown      Hotel room           231                NA        231       231
##  3 Downtown      Entire home/a…       226.               74.9      154       241
##  4 Union Station Unknown              216.               71.2      172.      190.
##  5 Foggy Bottom  Private room         206.               48.1      192.      217
##  6 Shaw          Unknown              199.               90.9      148.      171
##  7 Georgetown    Entire home/a…       189.               69.1      132.      189
##  8 Capitol Hill  Entire home/a…       181.               61.9      136.      168
##  9 Union Station Entire home/a…       169.               61.6      120       156
## 10 Dupont Circle Entire home/a…       164.               57.6      122.      160
## # ℹ 26 more rows
## # ℹ 2 more variables: Q_price75 <dbl>, IQR <dbl>
```

```
df_price %>%
  group_by(neighborhood, room_type) %>%
  summarize(avg_price = mean(price, na.rm = TRUE), standard_deviation = sd(price, na.rm = TRUE)
,
            Q_price25=quantile(price, 0.25), Q_price50=quantile(price, 0.5), Q_price75=quantile
(price, 0.75)
            , IQR = (quantile(price, 0.75) - quantile(price, 0.25))) %>%
  arrange((avg_price))
```

```
## `summarise()` has grouped output by 'neighborhood'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 36 × 8
## # Groups:   neighborhood [10]
##    neighborhood     room_type   avg_price standard_deviation Q_price25 Q_price50
##    <fct>            <fct>           <dbl>              <dbl>     <dbl>     <dbl>
##  1 Capitol Hill     Shared room      54.3              28.8       29        51
##  2 Kalorama Heights Shared room      63                NA         63        63
##  3 Edgewood         Private ro…      73.0              29.8       49.5      71
##  4 Columbia Heights Private ro…      79.6              25.4       64.2      78
##  5 Downtown         Shared room      86.7               8.33      82        84
##  6 Kalorama Heights Unknown          86.7              44.8       61.5      72
##  7 Shaw             Shared room      93.5              17.9       86.2      90.5
##  8 Capitol Hill     Private ro…     102.               35.2       79        97.5
##  9 Union Station    Private ro…     104.               40.8       71        97
## 10 Capitol Hill     Hotel room      113                NA        113       113
## # ℹ 26 more rows
## # ℹ 2 more variables: Q_price75 <dbl>, IQR <dbl>
```

```
df_price %>%
  group_by(neighborhood, room_type) %>%
  summarize(avg_price = mean(price, na.rm = TRUE), standard_deviation = sd(price, na.rm = TRUE)
,
            Q_price25=quantile(price, 0.25), Q_price50=quantile(price, 0.5), Q_price75=quantile
(price, 0.75)
            , IQR = (quantile(price, 0.75) - quantile(price, 0.25))) %>%
  arrange(desc(standard_deviation))
```

```
## `summarise()` has grouped output by 'neighborhood'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 36 × 8
## # Groups:   neighborhood [10]
##    neighborhood     room_type   avg_price standard_deviation Q_price25 Q_price50
##    <fct>            <fct>           <dbl>              <dbl>     <dbl>     <dbl>
##  1 Downtown         Private ro…      163.               95.1      71.8      172.
##  2 Shaw             Unknown          199.               90.9     148.       171
##  3 Georgetown       Private ro…      141                88.4      87        100.
##  4 Capitol Hill     Unknown          130.               78.1     104.       118
##  5 Columbia Heights Unknown          128.               76.9      80        125
##  6 Dupont Circle    Unknown          154.               75.4     112.       154.
##  7 Downtown         Entire hom…      226.               74.9     154        241
##  8 Union Station    Unknown          216.               71.2     172.       190.
##  9 Georgetown       Entire hom…      189.               69.1     132.       189
## 10 Capitol Hill     Entire hom…      181.               61.9     136.       168
## # ℹ 26 more rows
## # ℹ 2 more variables: Q_price75 <dbl>, IQR <dbl>
```

```
df_price %>%
  group_by(neighborhood, room_type) %>%
  summarize(avg_price = mean(price, na.rm = TRUE), standard_deviation = sd(price, na.rm = TRUE)
,
            Q_price25=quantile(price, 0.25), Q_price50=quantile(price, 0.5), Q_price75=quantile
(price, 0.75)
            , IQR = (quantile(price, 0.75) - quantile(price, 0.25))) %>%
  arrange((standard_deviation))
```

```
## `summarise()` has grouped output by 'neighborhood'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 36 × 8
## # Groups:   neighborhood [10]
##    neighborhood     room_type   avg_price standard_deviation Q_price25 Q_price50
##    <fct>            <fct>           <dbl>              <dbl>     <dbl>     <dbl>
##  1 Downtown         Shared room      86.7               8.33      82        84
##  2 Edgewood         Unknown         131.               12.1      126       135
##  3 Dupont Circle    Hotel room      136.               13.8      128.      138
##  4 Shaw             Shared room      93.5              17.9       86.2      90.5
##  5 Columbia Heights Private ro…      79.6              25.4       64.2      78
##  6 Capitol Hill     Shared room      54.3              28.8       29        51
##  7 Edgewood         Private ro…      73.0              29.8       49.5      71
##  8 Dupont Circle    Private ro…     132.               31.6      114.      135
##  9 Capitol Hill     Private ro…     102.               35.2       79        97.5
## 10 Union Station    Private ro…     104.               40.8       71        97
## # ℹ 26 more rows
## # ℹ 2 more variables: Q_price75 <dbl>, IQR <dbl>
```

```
df_price %>%
  group_by(neighborhood, room_type) %>%
  summarize(avg_price = mean(price, na.rm = TRUE), standard_deviation = sd(price, na.rm = TRUE)
,
            Q_price25=quantile(price, 0.25), Q_price50=quantile(price, 0.5), Q_price75=quantile
(price, 0.75)
            , IQR = (quantile(price, 0.75) - quantile(price, 0.25))) %>%
  arrange(desc(IQR))
```

```
## `summarise()` has grouped output by 'neighborhood'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 36 × 8
## # Groups:   neighborhood [10]
##    neighborhood     room_type   avg_price standard_deviation Q_price25 Q_price50
##    <fct>            <fct>           <dbl>              <dbl>     <dbl>     <dbl>
##  1 Downtown         Private ro…      163.               95.1      71.8      172.
##  2 Downtown         Entire hom…      226.               74.9     154        241
##  3 Union Station    Entire hom…      169.               61.6     120        156
##  4 Georgetown       Entire hom…      189.               69.1     132.       189
##  5 Kalorama Heights Private ro…      142.               53.2     100        144
##  6 Georgetown       Private ro…      141                88.4      87        100.
##  7 Shaw             Unknown          199.               90.9     148.       171
##  8 Dupont Circle    Unknown          154.               75.4     112.       154.
##  9 Dupont Circle    Entire hom…      164.               57.6     122.       160
## 10 Capitol Hill     Entire hom…      181.               61.9     136.       168
## # ℹ 26 more rows
## # ℹ 2 more variables: Q_price75 <dbl>, IQR <dbl>
```

```
df_price %>%
  group_by(neighborhood, room_type) %>%
  summarize(avg_price = mean(price, na.rm = TRUE), standard_deviation = sd(price, na.rm = TRUE)
,
            Q_price25=quantile(price, 0.25), Q_price50=quantile(price, 0.5), Q_price75=quantile
(price, 0.75)
            , IQR = (quantile(price, 0.75) - quantile(price, 0.25))) %>%
  arrange((IQR))
```

```
## `summarise()` has grouped output by 'neighborhood'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 36 × 8
## # Groups:   neighborhood [10]
##    neighborhood     room_type   avg_price standard_deviation Q_price25 Q_price50
##    <fct>            <fct>           <dbl>              <dbl>     <dbl>     <dbl>
##  1 Capitol Hill     Hotel room      113                NA        113       113
##  2 Downtown         Hotel room      231                NA        231       231
##  3 Foggy Bottom     Unknown         241                NA        241       241
##  4 Kalorama Heights Shared room      63                NA         63        63
##  5 Downtown         Shared room      86.7               8.33      82        84
##  6 Edgewood         Unknown         131.               12.1      126       135
##  7 Shaw             Shared room      93.5              17.9       86.2      90.5
##  8 Dupont Circle    Hotel room      136.               13.8      128.      138
##  9 Columbia Heights Private ro…      79.6              25.4       64.2      78
## 10 Capitol Hill     Private ro…     102.               35.2       79        97.5
## # ℹ 26 more rows
## # ℹ 2 more variables: Q_price75 <dbl>, IQR <dbl>
```

```r
## Question 3

conf_test = function(level , name , dataframe) {
  # the function will take three arguments
  # level: numerical confidence level
  # name of the numerical variable (string)
  # name of the dataframe

  # need to only grab all the columns and work with that data
  # since double [[]] for dataframes will pull only the data from that column
  var = dataframe[[name]]
  avg = mean(var , na.rm = TRUE)
  std = sd(var, na.rm = TRUE)
  alpha = 1 -level
  d_f = length(var) - 1
  # one tailed
  t_val = abs(qt(  alpha, d_f ))

  conf_int = c( avg - (t_val * (std / sqrt(length(var))))
                ,
                avg + (t_val * (std / sqrt(length(var)))))
  # two tailed
  t_val2 = abs(qt(  alpha/2 , d_f ))
  conf_int2 = c( avg - (t_val2 * (std / sqrt(length(var))))
                 ,
                 avg + (t_val2 * (std / sqrt(length(var)))))

  return(list(one_tailed = conf_int , two_tailed = conf_int2))

}

conf_test(0.95, "price" , df_price)
```

```
## $one_tailed
## [1] 152.8294 157.7591
##
## $two_tailed
## [1] 152.3568 158.2317
```

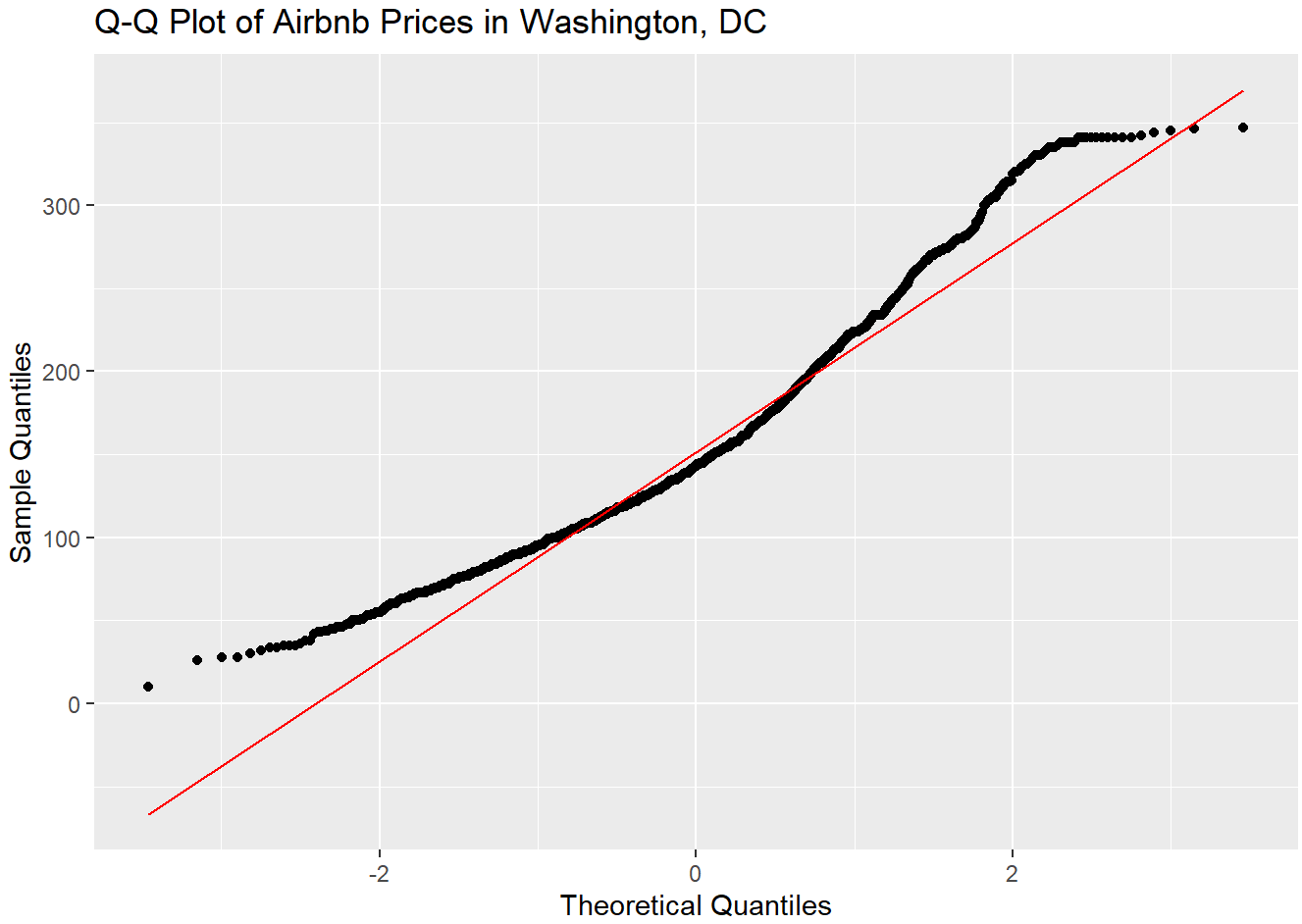
```
confInt = function(level , var, data) {

  x = data[[var]]
  n = length(x)

  alpha = 1-level
  avg = mean(x)
  stdev = sd(x)
  sterr = stdev / sqrt(n)

  cvT = qt(1-alpha/2, df = n-1)

  c(lower = avg-cvT * sterr,
    mean = avg,
    upper = avg + cvT * sterr,
    n = n)

}

confInt95 = confInt(.95, "price", df_price)

round(confInt95, 2)
```

```
##   lower    mean   upper       n
##  152.36  155.29  158.23 1842.00
```

```
#Q4 : A T-Test was performed.

t.test(df_price$price, mu = 200, alternative = "greater", conf.level = 0.95)
```

```
##
##  One Sample t-test
##
## data:  df_price$price
## t = -29.848, df = 1841, p-value = 1
## alternative hypothesis: true mean is greater than 200
## 95 percent confidence interval:
##  152.8294      Inf
## sample estimates:
## mean of x
##  155.2942
```

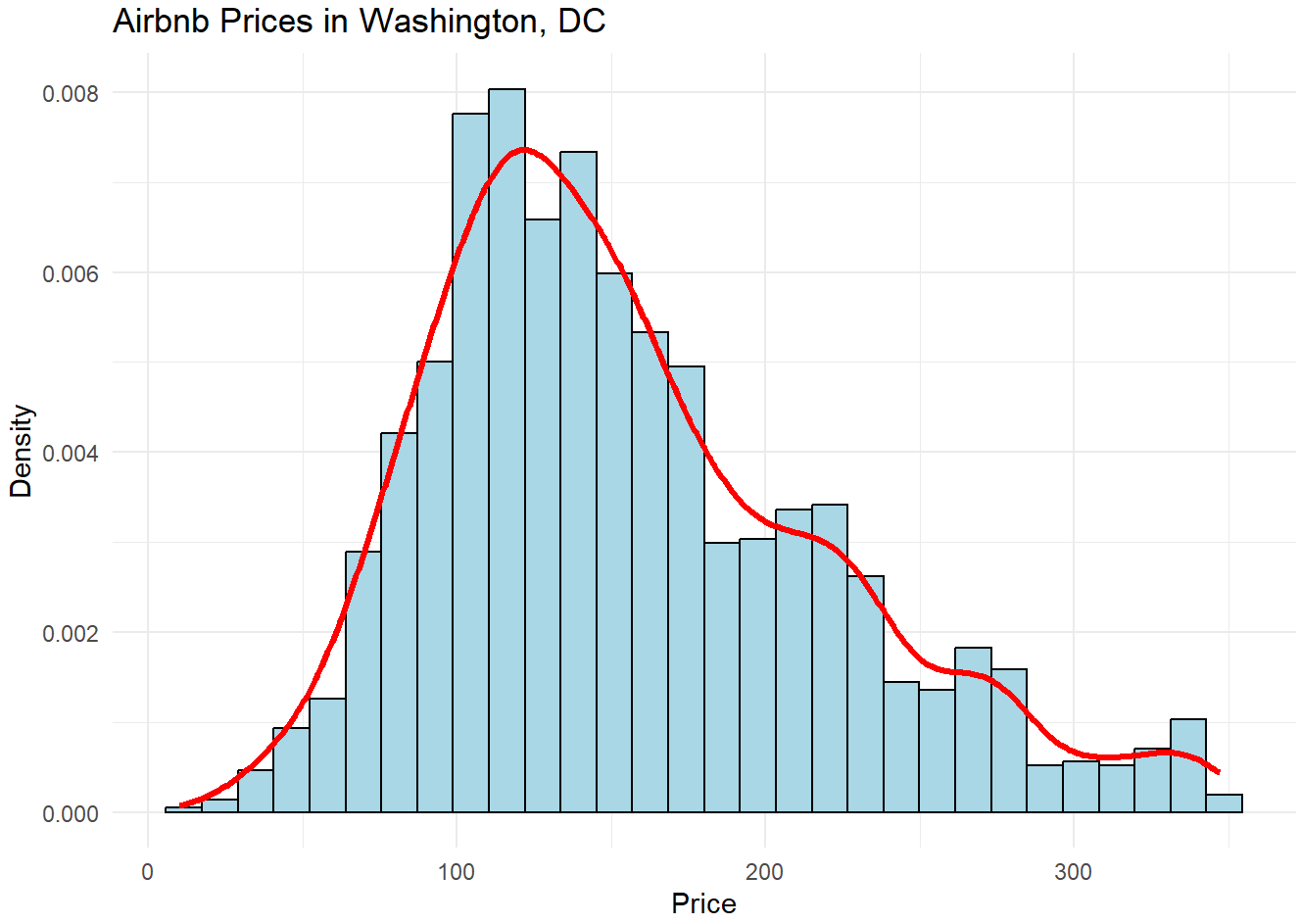
```
#Q5. Visualize price to test for normality.

#Use Q-Q Plot to test for normality.

ggplot(df_price, aes(sample = price)) +
  stat_qq() +
  stat_qq_line(color = "red") +
  labs(x = "Theoretical Quantiles",
       y = "Sample Quantiles",
       title = "Q-Q Plot of Airbnb Prices in Washington, DC")
```



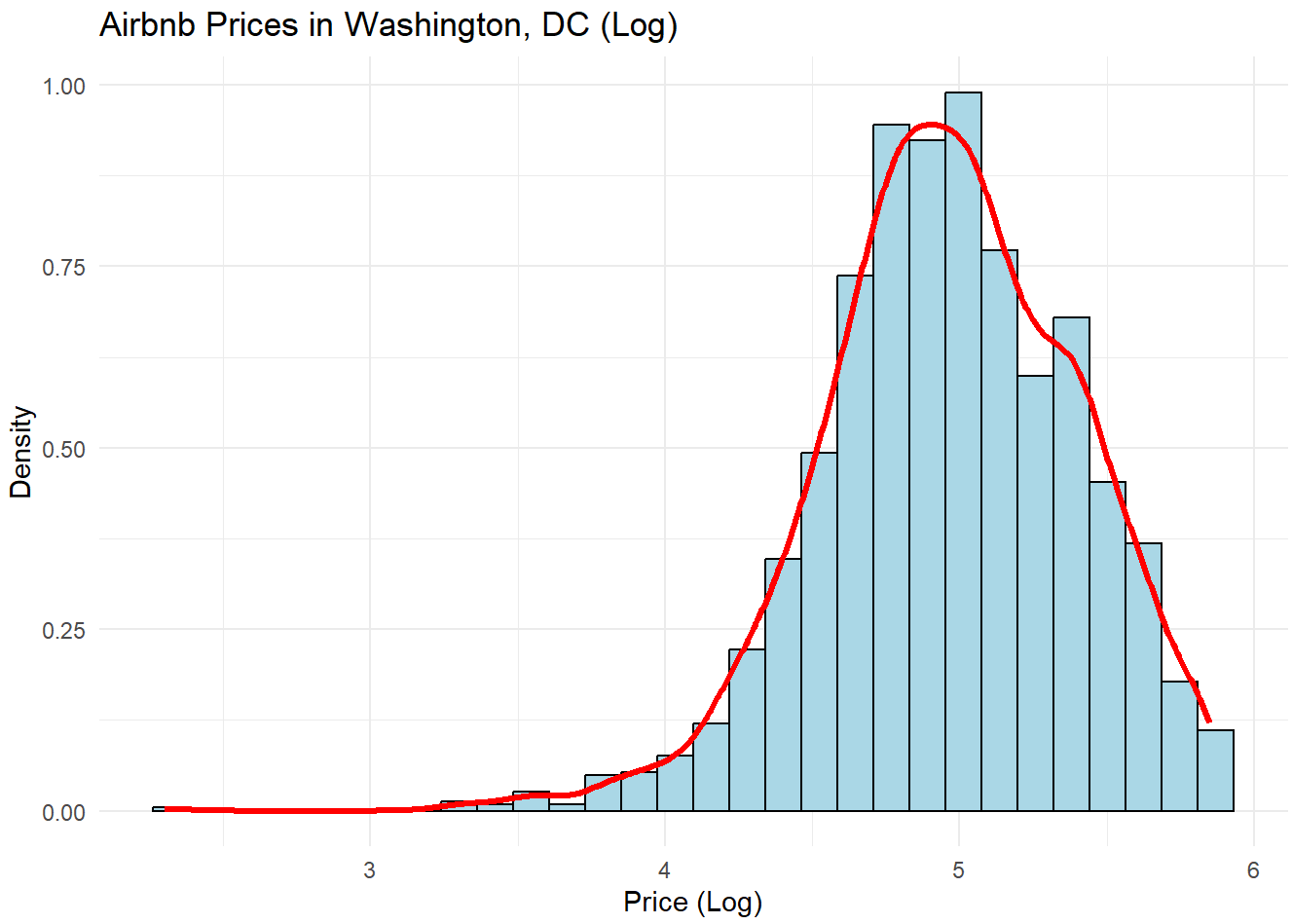

```
#Use Histogram to test for normality

ggplot(df_price, aes(x = price)) +
  geom_histogram(aes(y = ..density..), bins = 30, fill = "lightblue",
                 color = "black")+
  geom_density(color = "red", size = 1.2) +
  labs(x = "Price",
       y = "Density",
       title = "Airbnb Prices in Washington, DC") +
  theme_minimal()
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## ℹ Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

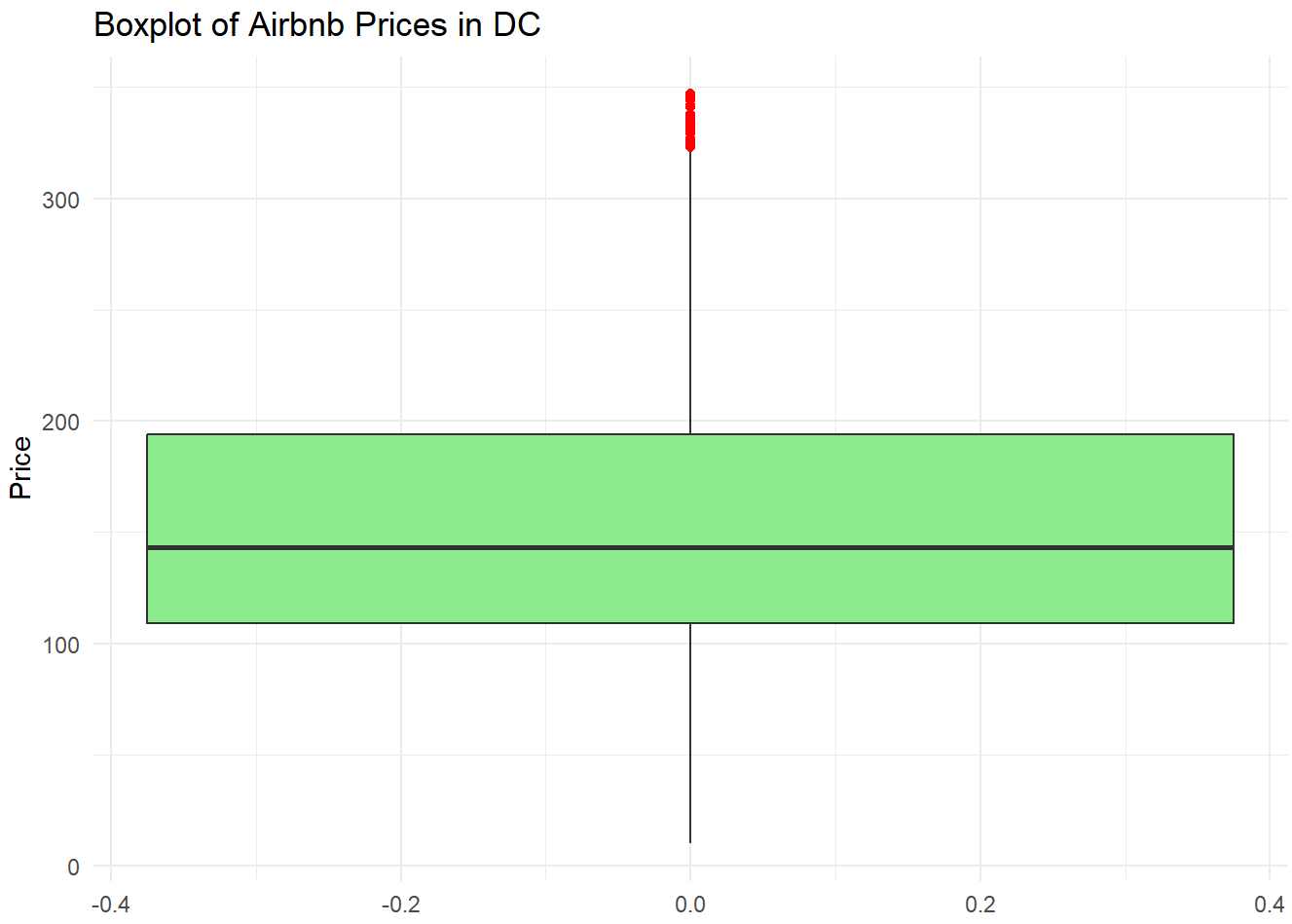
```
## Warning: The dot-dot notation (`..density..`) was deprecated in ggplot2 3.4.0.
## ℹ Please use `after_stat(density)` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



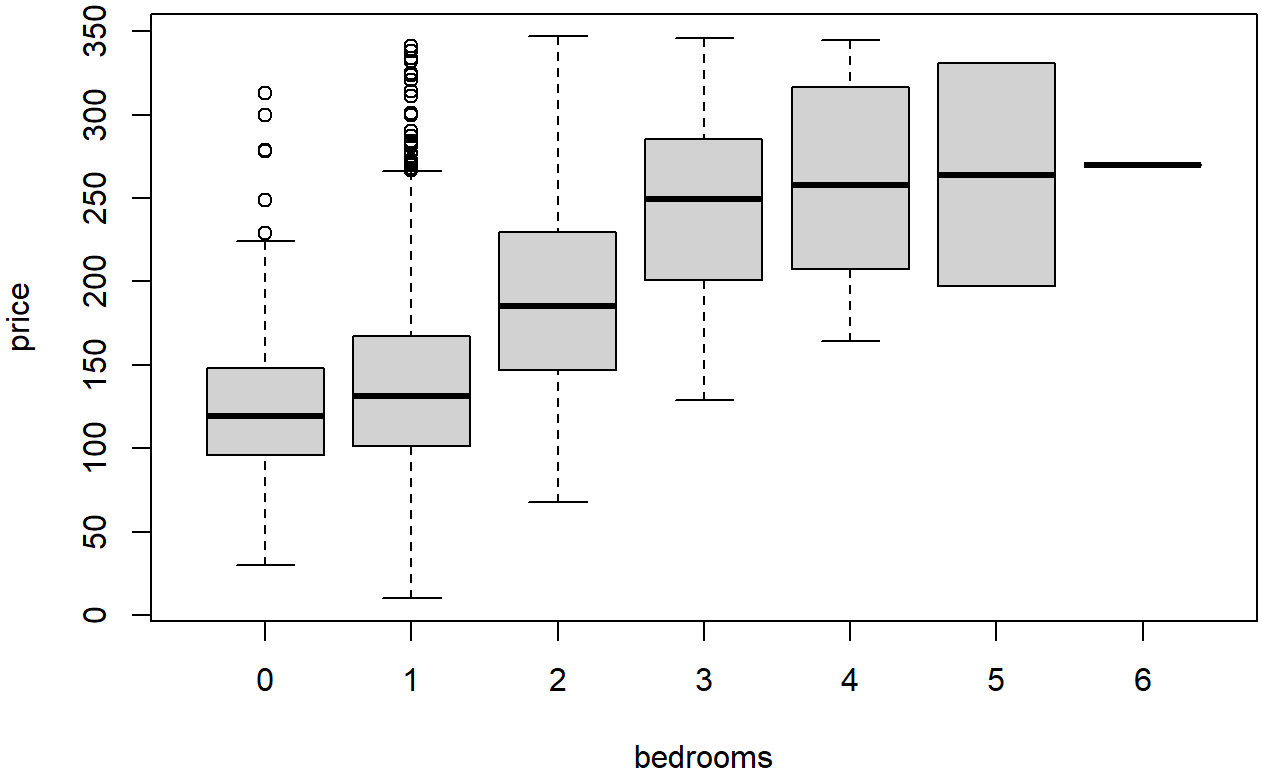

```
#Use log(price) to try to normalize and make less skewed
#Log(price) did not work

ggplot(df_price, aes(x = log(price))) +
  geom_histogram(aes(y = ..density..), bins = 30, fill = "lightblue",
                 color = "black")+
  geom_density(color = "red", size = 1.2) +
  labs(x = "Price (Log)",
       y = "Density",
       title = "Airbnb Prices in Washington, DC (Log)") +
  theme_minimal()
```



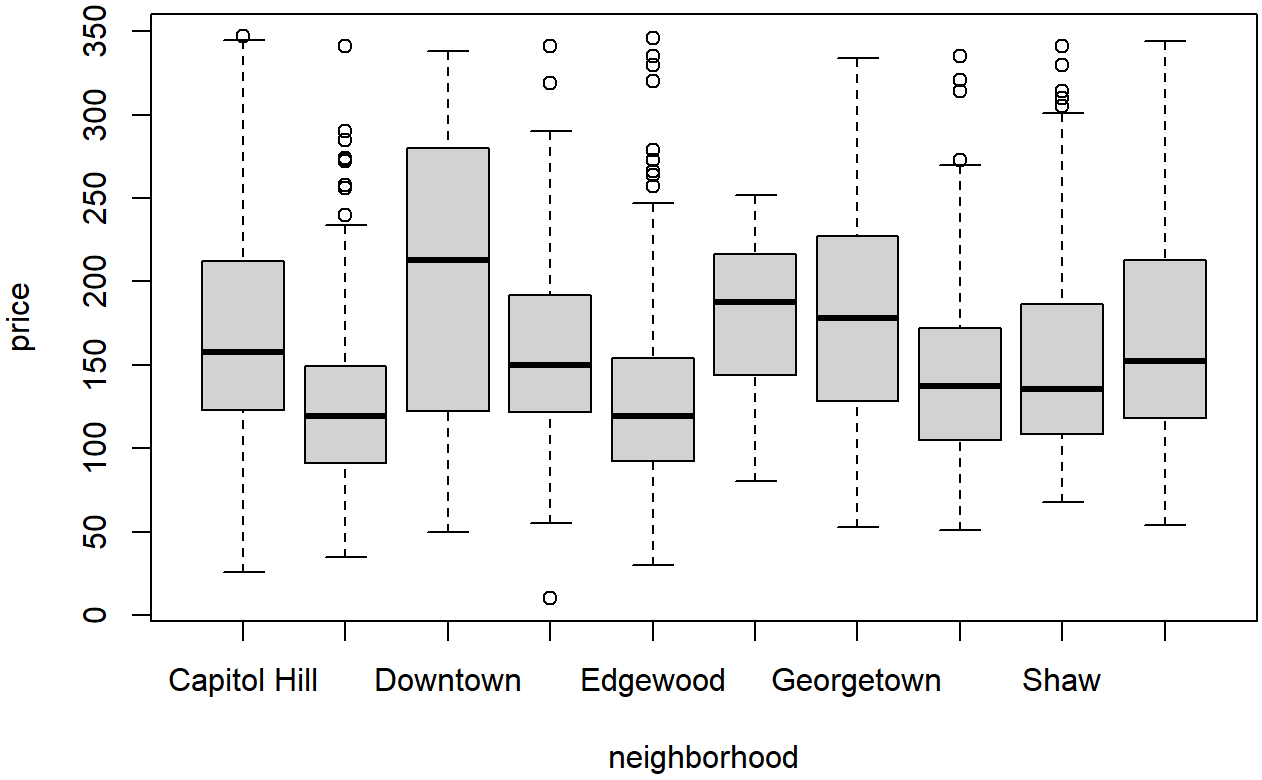

```
#Boxplot (can removed since it does not answer #5)

ggplot(df_price, aes(y = price)) +
  geom_boxplot(fill = "lightgreen", outlier.color = "red") +
  labs(y = "Price",
       title = "Boxplot of Airbnb Prices in DC") +
  theme_minimal()
```




```
#Boxplot to compare price and bedrooms

boxplot(price~bedrooms, df_price)
```




```
#Boxplot to compare price and neighborhood

boxplot(price~neighborhood, df_price)
```


price
0
50
100
150
200
250
300
350
Capitol Hill
Downtown
Edgewood
Georgetown
Shaw
neighborhood

```
#Q6. What's the best simple linear regression model for the "price" of the listings based on R-squared and residual standard error?
#Compare/present the results of the tested models as a table in your report.

#Simple linear regression of price using neighborhood as predictor.
RegNeighborhood = lm(price~neighborhood, df_price)

#Simple linear regression of price using bedrooms as predictor.
RegBedrooms = lm(price~bedrooms, df_price)

#Simple linear regression of price using room type as predictor.
RegRoomType = lm(price~room_type, df_price)

#Simple linear regression of price using accommodates as predictor.
RegAccommodates = lm(price~accommodates, df_price)

#Simple linear regression of price using beds as predictor.
RegBeds = lm(price~beds, df_price)

#Simple linear regression of price using bathrooms as predictor.
RegBathrooms = lm(price~bathrooms, df_price)

#Simple linear regression of price using minimum nights as predictor.
RegMinNights = lm(price~min_nights, df_price)

#Simple linear regression of price using total reviews as predictor.
RegReviews = lm(price~total_reviews, df_price)

#Simple linear regression of price using host acceptance rate as predictor.
RegHostAcceptRate = lm(price~host_acceptance_rate, df_price)

#Simple linear regression of price using Superhost as predictor.
RegSuperhost = lm(price~superhost, df_price)

#Simple linear regression of price using host total listings as predictor.
RegHostListings = lm(price~host_total_listings, df_price)

#Simple linear regression of price using average rating as predictor.
RegAvgRating = lm(price~avg_rating, df_price)

#Compare the regression models of R-squared and residual standard error.

#Summaries
sumNeighborhood = summary(RegNeighborhood)
sumBedrooms = summary(RegBedrooms)
sumRoomType = summary(RegRoomType)
sumAccommodates = summary(RegAccommodates)
sumBeds = summary(RegBeds)
sumBathrooms = summary(RegBathrooms)
sumMinNights = summary(RegMinNights)
sumReviews = summary(RegReviews)
sumHostAcceptRate = summary(RegHostAcceptRate)
```

```
sumSuperhost = summary(RegSuperhost)
sumHostListings = summary(RegHostListings)
sumAvgRating = summary(RegAvgRating)

#Comparison table for all the models.

modelComp = data.frame(
  Model = c("Req with Neighborhood", "Reg with Bedrooms", "Reg with Room Type",
            "Reg with Accommodates", "Reg with Beds", "Reg with Bathrooms",
            "Reg with Min Nights", "Reg with Total Reviews", "Reg with Host Acceptance Rate",
            "Reg with Superhost", "Reg with Host Total Listings", "Reg with Average Rating"),
  SER = c(sumNeighborhood$sigma, sumBedrooms$sigma, sumRoomType$sigma,
          sumAccommodates$sigma, sumBeds$sigma, sumBathrooms$sigma,
          sumMinNights$sigma, sumReviews$sigma, sumHostAcceptRate$sigma,
          sumSuperhost$sigma, sumHostListings$sigma, sumAvgRating$sigma),
  R2 = c(sumNeighborhood$r.squared, sumBedrooms$r.squared, sumRoomType$r.squared,
         sumAccommodates$r.squared, sumBeds$r.squared, sumBathrooms$r.squared,
         sumMinNights$r.squared, sumReviews$r.squared, sumHostAcceptRate$r.squared,
         sumSuperhost$r.squared, sumHostListings$r.squared, sumAvgRating$r.squared)
)

modelComp
```

```
##                            Model      SER           R2
## 1          Req with Neighborhood 61.25694 0.0963342077
## 2              Reg with Bedrooms 57.45585 0.2015308358
## 3             Reg with Room Type 60.89715 0.1044808891
## 4          Reg with Accommodates 56.91354 0.2165326508
## 5                  Reg with Beds 61.01476 0.0995503403
## 6             Reg with Bathrooms 55.16945 0.2686162663
## 7            Reg with Min Nights 63.72875 0.0176632197
## 8         Reg with Total Reviews 63.78832 0.0158257530
## 9  Reg with Host Acceptance Rate 63.44086 0.0265182301
## 10            Reg with Superhost 64.25538 0.0013607687
## 11  Reg with Host Total Listings 62.63565 0.0510729702
## 12       Reg with Average Rating 64.28685 0.0003821928
```

```
#Model with lowest residual standard error
modelComp$Model[which.min(modelComp$SER)]
```

```
## [1] "Reg with Bathrooms"
```

```
#Model with highest residual standard error
modelComp$Model[which.max(modelComp$SER)]
```

```
## [1] "Reg with Average Rating"
```

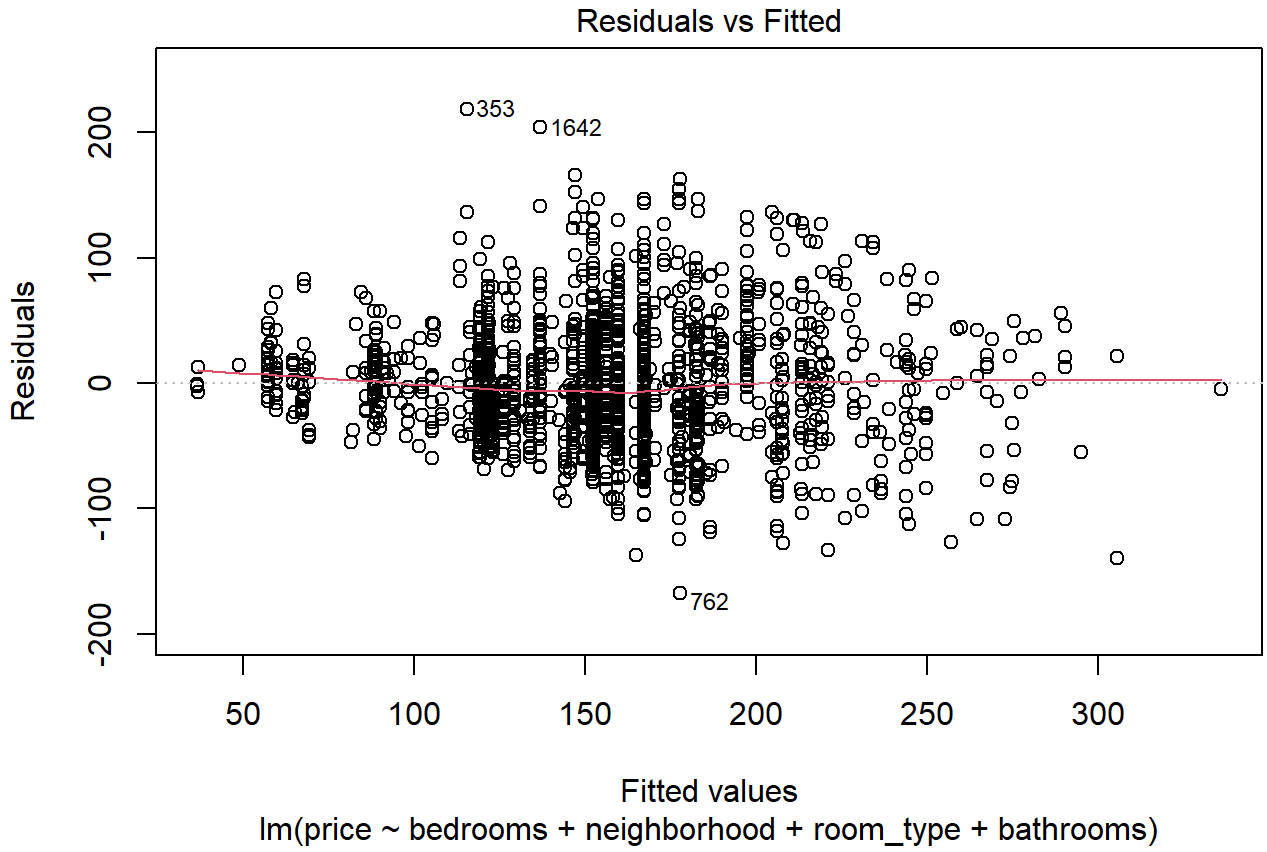
```
#Q7:
# multiple linear model
# predictors: bedrooms + neighborhood + room_type + accommodates only analysis
filtReg = lm(price ~ bedrooms + neighborhood + room_type + bathrooms,df_price)
summary(filtReg)
```

```
##
## Call:
## lm(formula = price ~ bedrooms + neighborhood + room_type + bathrooms,
##     data = df_price)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -167.880  -31.579   -5.529   25.815  218.595
##
## Coefficients:
##                              Estimate Std. Error t value Pr(>|t|)
## (Intercept)                   136.913      3.644  37.574  < 2e-16 ***
## bedrooms                       30.278      2.206  13.724  < 2e-16 ***
## neighborhoodColumbia Heights  -47.954      4.425 -10.837  < 2e-16 ***
## neighborhoodDowntown           38.984      6.639   5.872 5.12e-09 ***
## neighborhoodDupont Circle      -7.559      4.499  -1.680 0.093081 .
## neighborhoodEdgewood          -45.564      4.147 -10.987  < 2e-16 ***
## neighborhoodFoggy Bottom       10.324     11.359   0.909 0.363530
## neighborhoodGeorgetown         10.237      6.374   1.606 0.108403
## neighborhoodKalorama Heights  -20.573      6.234  -3.300 0.000985 ***
## neighborhoodShaw              -11.009      4.965  -2.217 0.026733 *
## neighborhoodUnion Station     -15.004      3.679  -4.079 4.72e-05 ***
## room_typeHotel room           -10.960     14.343  -0.764 0.444868
## room_typePrivate room         -19.258      7.206  -2.673 0.007594 **
## room_typeShared room          -55.093     16.360  -3.367 0.000774 ***
## room_typeUnknown               -2.340      8.528  -0.274 0.783787
## bathrooms1 private bath       -13.912      8.394  -1.657 0.097627 .
## bathrooms1 shared bath        -42.766      9.498  -4.503 7.14e-06 ***
## bathrooms1.5 baths             18.247      6.075   3.004 0.002705 **
## bathrooms1.5 shared baths     -34.622     11.614  -2.981 0.002911 **
## bathrooms2 baths               31.058      5.012   6.196 7.14e-10 ***
## bathrooms2 shared baths       -35.178     11.926  -2.950 0.003223 **
## bathrooms2.5 baths             36.829      7.519   4.898 1.05e-06 ***
## bathrooms2.5 shared baths      11.765     17.894   0.657 0.510963
## bathrooms3 baths               62.529     13.705   4.562 5.39e-06 ***
## bathrooms3 shared baths       -65.797     21.413  -3.073 0.002152 **
## bathrooms3.5 baths             -6.552     25.485  -0.257 0.797136
## bathrooms4 baths              -26.298     35.845  -0.734 0.463250
## bathroomsShared half-bath     -66.929     49.678  -1.347 0.178070
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 49.1 on 1814 degrees of freedom
## Multiple R-squared:  0.4252, Adjusted R-squared:  0.4166
## F-statistic: 49.69 on 27 and 1814 DF,  p-value: < 2.2e-16
```

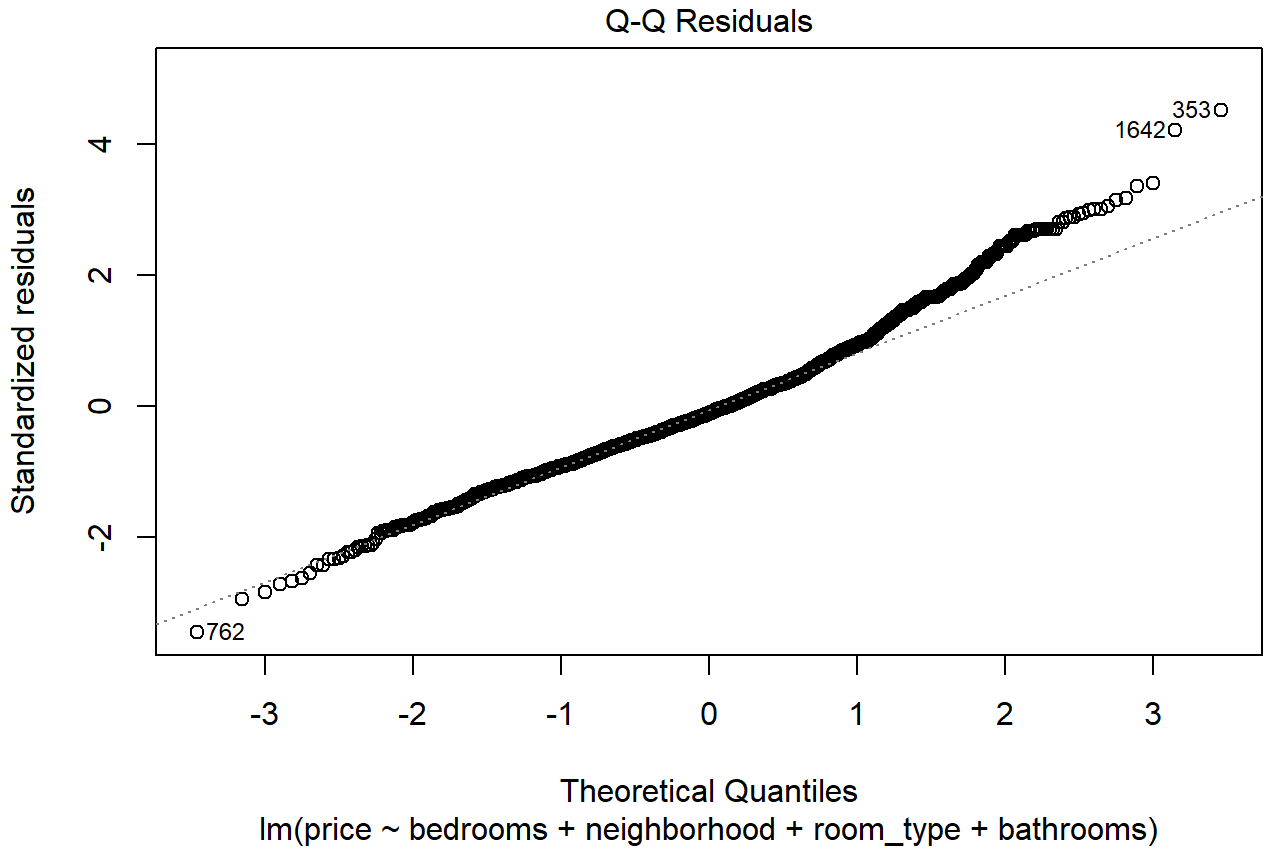
```r
# coefficent table
coef =  coef(summary(filtReg))
coef
```

```
##                                Estimate Std. Error     t value      Pr(>|t|)
## (Intercept)                  136.913466   3.643874  37.5736003 5.064628e-229
## bedrooms                      30.277802   2.206155  13.7242379  7.411175e-41
## neighborhoodColumbia Heights -47.954089   4.424933 -10.8372460  1.463895e-26
## neighborhoodDowntown          38.983961   6.639195   5.8717907  5.115536e-09
## neighborhoodDupont Circle     -7.558652   4.498532  -1.6802484  9.308117e-02
## neighborhoodEdgewood         -45.563815   4.146886 -10.9874776  3.103711e-27
## neighborhoodFoggy Bottom      10.323697  11.358598   0.9088884  3.635298e-01
## neighborhoodGeorgetown        10.237417   6.373672   1.6062040  1.084031e-01
## neighborhoodKalorama Heights -20.573244   6.233747  -3.3003015  9.845337e-04
## neighborhoodShaw             -11.008809   4.965202  -2.2171927  2.673348e-02
## neighborhoodUnion Station    -15.004493   3.678726  -4.0787196  4.724493e-05
## room_typeHotel room          -10.960337  14.342893  -0.7641650  4.448683e-01
## room_typePrivate room        -19.257899   7.205703  -2.6725914  7.594204e-03
## room_typeShared room         -55.092738  16.360196  -3.3674864  7.744231e-04
## room_typeUnknown              -2.340214   8.527619  -0.2744276  7.837873e-01
## bathrooms1 private bath      -13.912133   8.394341  -1.6573227  9.762717e-02
## bathrooms1 shared bath       -42.765915   9.498027  -4.5026104  7.139478e-06
## bathrooms1.5 baths            18.246978   6.075214   3.0035122  2.705343e-03
## bathrooms1.5 shared baths    -34.622358  11.614172  -2.9810441  2.910910e-03
## bathrooms2 baths              31.057706   5.012446   6.1961173  7.144449e-10
## bathrooms2 shared baths      -35.177655  11.926248  -2.9495995  3.222599e-03
## bathrooms2.5 baths            36.829294   7.519079   4.8981124  1.053567e-06
## bathrooms2.5 shared baths     11.764692  17.893908   0.6574691  5.109627e-01
## bathrooms3 baths              62.528593  13.705030   4.5624556  5.394724e-06
## bathrooms3 shared baths      -65.796856  21.412718  -3.0727932  2.152204e-03
## bathrooms3.5 baths            -6.552119  25.485380  -0.2570932  7.971360e-01
## bathrooms4 baths             -26.297980  35.844889  -0.7336605  4.632505e-01
## bathroomsShared half-bath    -66.928875  49.678416  -1.3472425  1.780704e-01
```

```r
#Diagnostic plots
plot(filtReg, which = 1)
```




```
plot(filtReg, which = 2)
```




```
#Summary Of Data Model" sigma, R2, AdjR
ModelName = c("filtReg")
Sigma = c(summary(filtReg)$sigma)
RSqrd = c(summary(filtReg)$r.squared)
AdjRSqrd = c(summary(filtReg)$adj.r.squared)

SummaryTable = data.frame(ModelName,Sigma,RSqrd,AdjRSqrd)
SummaryTable
```

```
##   ModelName    Sigma    RSqrd  AdjRSqrd
## 1   filtReg 49.09875 0.425156 0.4165999
```

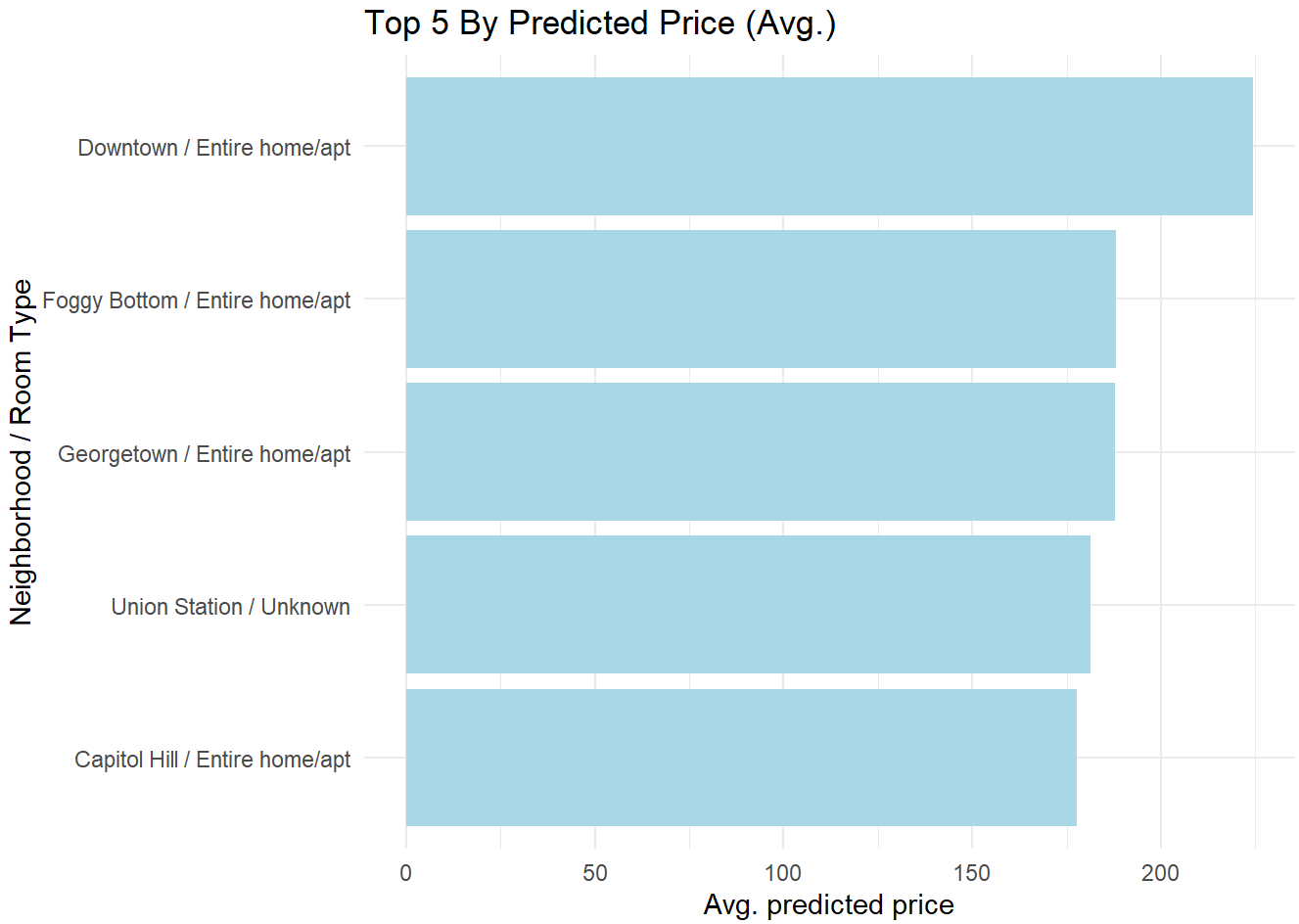
```
# predicted price
df_price$prdctPrce = predict(filtReg)

#Group, summarize, and keep top 5 groups
topFiveGroup = df_price %>%
  group_by(neighborhood, room_type) %>%
  summarize(
    numLstngs = n(),
    avgPred = mean(prdctPrce))%>%
  ungroup() %>%
  filter(numLstngs >= 5) %>%
  arrange(desc(avgPred)) %>%
  slice_head(n = 5)
```

```
## `summarise()` has grouped output by 'neighborhood'. You can override using the
## `.groups` argument.
```

```
topFiveGroup
```

```
## # A tibble: 5 × 4
##   neighborhood  room_type       numLstngs avgPred
##   <fct>         <fct>               <int>   <dbl>
## 1 Downtown      Entire home/apt        45    224.
## 2 Foggy Bottom  Entire home/apt        15    188.
## 3 Georgetown    Entire home/apt        64    188.
## 4 Union Station Unknown                 6    181.
## 5 Capitol Hill  Entire home/apt       271    178.
```

```
#ggplot top 5

ggplot(topFiveGroup,
       aes(x = avgPred,
           y = reorder(paste(neighborhood, "/", room_type), avgPred))) +
  geom_col(fill = "lightblue") +
  labs(title = "Top 5 By Predicted Price (Avg.)",
       x = "Avg. predicted price", y = "Neighborhood / Room Type") +
  theme_minimal()
```




```
#Q8: Ran correlations against bedrooms and accommodates. mild correlation. Still mulling this on
e over.

cor(df_price$bedrooms, df_price$accommodates)
```

```
## [1] 0.6860678
```